kayfabe

A Mode-2 NVIDIA GPU forwarding stack for KVM/QEMU guests

An architecture whitepaper, written to be attacked

STATUS: SUPERSEDED, 2026-10-27. This paper describes the pre-v3 architecture at its historical source pins. Read `kayfabe_v3.pdf` for the v3 single-process architecture, current evidence limits and outside-reader questions. The historical text and corrections below are preserved; its counts, performance results and open questions are not current v3 verdicts.

Subject. Two repositories on one machine. `/workspace/nvidia-gpu-passthrough` — the C research artifact, 44 783 lines of C, developed 2026-05-24 to 2026-08-01; it works on real hardware, and it is not history: it is maintained as this project's *standing differential oracle*. `kayfabe` — the clean-slate Rust rewrite that is the product, **1 391 commits**, **198 122 lines of implementation** across 23 crates, **158 170 lines of test code**, and a C QOM shim that links the whole thing into a stock QEMU. Developed 2026-07-24 to today. **This document is pinned to Rust HEAD** `5c367a38` **(2026-08-20).**

⊘⊘⊘ **RE-STAMPED 2026-09-11 AT** `5f323dd5`**, AND SEVEN CLAIMS BELOW ARE NOW FALSE. 477 commits** landed between the pin and this pass. Structural counts re-derived here, at `5f323dd5`: **1 898 commits**, **231 993 lines of implementation**, **163 998 lines of test code**, **23 crates** (24 workspace members counting `tests/`). **Every other number in this document is still [measured] at** `5c367a38` unless a correction block beside it says otherwise. The seven reversals, each folded in place at the section that carries it:

1. ***"No LLM has ever produced a token in this guest"* is FALSE, twice over.** [measured] 2026-09-06 (`w383llmgate`): `LLM_TOKENS=16`, *degenerate* text, 45 s/token. [measured] 2026-09-09, rev `c8fe67d6`: `W392_OUTCOME=(P)`, **16 tokens whose text is byte-identical to the same-boot CPU oracle**, host `Xid` **0**. The garbage-text cause was our own: `reach.rs` carried `qualifies = … && !is_remap`, so a re-mapped joined row was never revoked and the stale binding kept translating — the guest read the *previous* tensor. ⊘ **It is not a stable pass**: §8.5's correction block has the scope.
2. **The blocking model is three owner rules now, and only two of them are met.** No blocking work in a vCPU trap; **no blocking work under a lock a vCPU takes** (transitive blocking counts); and a trap may not exceed **1 ms**. §4.4 is new and states what is built, what is only counted, and what is not enforced at all.
3. **BAR1 and BAR2 default to UNTRAPPED** (`w431`, `qemu/hw/misc/nvkvm/nvkvm.c :3519/:3521`, both props now `true`). [measured] `w393`, `fa9d6395`: BAR1 traps **88 193** → **91**, a 969× reduction, client `(P)` on all arms, $n = 2$, `Xid` 0. §4.4.
4. **The doorbell defaults to DEFERRED, and a doorbell is a HINT rather than a barrier.** Owner, 2026-09-06: *"there is no guarantee that a real GPU gives either — the doorbell runs immediately in-line, it's a schedule."* Nothing may gate on one.
5. ***"Both invalidate transports measured zero on the compute path"* was a CENSUS GAP, not a property of the path**, and this paper built a normative rule on it (§6.1). The census had no row for the transport RM actually uses.
6. **§11 Q2 is ANSWERED** — the GSP command queue is serviced off the vCPU entirely, with no inline fallback. ⊘ And the wait it was asked about *did not move*.
7. `gpga.rs` **(1 391 lines) has ZERO production callers**, and this paper listed it in a **BUILT** crate without saying so (§5). So does `refresh.rs` (583).

⚠ **The reversals are not the whole delta and this pass does not claim they are.** It re-read the tree for the seven items above and for what §4.4 describes; it did not re-derive §7, §7.7, §8.19 or §8.18, whose sources report no change since the pin.

⊘ **SUPERSEDED IN PART BEFORE ITS FIRST READING — 2026-08-20 20:32, at** `56dc01f3`**, and the superseded claim is this document's own headline blocker.** Twelve commits landed after this pin (boots `w357`–`w367`, all on the same real GA106). Two of them retire claims made below and both are corrected in place rather than deleted: **(a) sequential multi-process no longer fails** — three consecutive guest CUDA processes in one boot all return `TORCH_OK`, with identical doorbell counts and 8/8 completions each (§8.11); **(b) the "object graph" wall named two paragraphs below does not exist** — every refusal in it was re-measured against the boot that *passes*, at the same revision and the same binary, and every one is *more numerous there* (§8.5). ⚠ **Every structural count in this paper — lines, crates, dependency edges, test counts, the claim-ledger numbers — is [measured] at** `5c367a38` **and has NOT been re-derived at** `56dc01f3`**.** Read those as pinned; read every behavioural claim through the corrections marked in place.

★★★★★ **What moved since the previous pin** (`f33e1c8`**, 2026-08-03**) **is the whole thesis of the document.** The previous four revisions were each built around a sentence of the form *"the Rust port has not reached compute"*, and the fourth's specific version was: *"the boot dies in the memory manager's scrubber"*, *"there is no* `cuInit`*, no* `cuCtxCreate` *and no compute in Mode 2 anywhere in this codebase"*, and *"on every boot to date the device reports* `doorbells: 0 arrived`*"*. **All**

three are now false, and retiring them is the single largest edit in this revision. What replaced them, on a real GA106 (RTX 3060) with host driver 580.159.04 open, stock unpatched guest driver:

- `cup3`**: FIRST COMPUTE.** [measured] 2026-08-14, rev `0655e6aa`, `traces/w297_cup3/`. A JITed PTX shader computing `out = in*3 + 1` returned `^CUP3_VAL=43` from `in = 14`. **43 cannot be copied, filled or forged**: a copy engine copies and memsets, our emulator has no arithmetic, and a forged completion carries a payload *we* chose. ⇒ the host GA106 **GR** engine executed the guest's shader. Reproduced **6/6** across subsequent rungs.
- `cup8`**: byte-exact matmul at scale.** `CUP8 RESULT N=2048 bad=0 maxerr=0 C[0]=2048 -> PASS`, `traces/w308_cup8/`, and again at **N=3072** (36 MiB operands) over 12 iterations, `traces/w327_cliff/`.
- **Doorbells are rung, routed and served.** The `cup3` boot's device census, verbatim: `480 arrived, 480 served, 0 REFUSED by name, GrCompute=125 Ce=355 unrouted=0`. Completions are hardware-written — a GPU-authored report semaphore with the engine's *own* clock in it, against a control that stayed at zero for 174 s.
- **The isolate plane is real and is on the path.** A capability-less, six-namespace, unprivileged child process holding `/dev/nvidiactl` and `/dev/nvidia0` is what issues the host RM ioctls that carry the compute.

★★★ **And the blocker is now one layer above everything this project used to worry about: it is in closed CUDA userspace.** For this whole campaign `libcuda` — the CUDA *driver* API — answered every call correctly in the guest while `libcudart` — the *runtime* API — returned **3** from its first call, which is what stops PyTorch. **That wall broke on 2026-08-20**: `cudaGetDeviceCount`, `cudaSetDevice`, `cudaMalloc` and `cudaDeviceSynchronize` all return 0, and `torch.cuda.is_available()` is `True`. ⊘ **And the model still does not run** — `torch._C._cuda_init()` raises *"CUDA unknown error"* and `LLM_TOKENS = 0`. Host `Xid` is 0 and guest `NVRM` sits at the green baseline, so it is a refusal we issue, not a fault.

⊘ **CORRECTED at `129dd316`, 2026-08-20 15:39 — three minutes after this document's pin.** This paragraph used to continue: *"the failure moved one rung into a different subsystem: our own object graph, where a handle resolves to the wrong kind of object on the context-promotion path."* **That wall does not exist.** Its every element was re-measured on the boot that *passes*, same revision and same binary (`PromoteFault::UnknownContextObject` 4 vs 3, `RmGraphError::FreeUnknown` 14 vs 10, unmapped alloc classes 6 vs 4), and the census line called *"the sharpest on the board"* turned out to be an eleven-day-old baseline line describing a route that legitimately does not apply (`dbc6c953`). ⇒ **What `torch._C._cuda_init()` fails on is unidentified**, and the paper's account of it is §8.5 read with that block on top. ⚠ **The LLM workload has not been re-run since `3887be37`**, so it has not been retested across the multi-process fixes of §8.11 either.

⚠ **This paragraph was rewritten three times in the hours it took to write this document**, as five commits landed; §8.5 keeps each version and why each was wrong, because that is the drift rate (§8.21) observed live rather than described. ★ Two findings out of it are methodological, and both are posed for outside review in §11: **the minimal fatal set was a PAIR of controls each of which is innocent alone**, so no one-at-a-time bisection could have found it — and one did not, six times, correctly; and **one control in the chain appears zero times in a healthy reference capture**, so no experiment driven by that reference could ever have covered it. ⊘ **And the milestone was graded on a proxy**: `is_available()` is not tokens, the two disagreed, and *the proxy was the optimistic one*.

★★ **The C artifact's fifth oracle limit still stands and is still the sharpest warning in the document.** Its captured control table has 56 rows: **29 complete, 16 truncated, 11 empty**. All eleven empty rows were asked of a real GA106 and **nine are contradicted** — and the two that are not are the instructive ones, because **one of them is genuinely zero and nothing about the row says so**: it is byte-identical to the nine that are wrong. *Believing the empty rows would have been right once and wrong nine times for the same reason.* The truncated class — the *larger* one — is unmeasured and passes the gate written to catch the empty one. ⊘ **An empty capture is evidence of nothing, not evidence of emptiness.** §3.1.

Stance. Every number here was re-measured from the tree while writing, not carried across from the previous revision and not restated from a design document. Where a claim is a reading of NVIDIA source rather than an observation it is marked [inferred]; where it is neither it is marked [assumed] or [unverified]. **Roughly half of this document is about what does not work, is not built, or is not known**, and it ends (§11) with the open questions stated so that a reader with no access to this tree can answer them.

Contents

1 What this document is, and how to attack it **1**

2 The idea **2**

2.1 The problem . . . 2
2.2 Vocabulary . . . 2
2.3 Mode 1 and Mode 2 . . . 2
2.4 The correctness criterion, and why it is not "replay everything" . . . 3
2.5 Why this is hard . . . 3

3 Provenance: the C artifact, and what it actually proved **3**
3.1 ★★ The C artifact is a standing oracle — and it has FIVE measured limits, the fifth different in kind . . . 5

4 The architecture **6**
4.1 Layers and ports . . . 6
4.2 The crate dependency graph . . . 9
4.3 The data path . . . 11
4.4 ★★★★★ NEW 2026-09-11: the blocking model, and the three synchronisation points . . . 13
4.5 The isolation model . . . 17
4.6 The lifecycle . . . 19

5 The crates **21**
5.1 `kayfabe-core` — 14 617 lines, **BUILT** and mutation-hardened . . . 21
5.2 `kayfabe-abi` — 42 511 lines, **BUILT** — now the largest crate in the tree . . . 21
5.3 `kayfabe-linux-raw` — 14 208 lines, **BUILT** — the *first* of two `unsafe` crates . . . 22
5.4 `kayfabe-gsp` — 5 497 lines, **BUILT** (S0–S5) — **and a real guest driver now writes into it** . . . 22
5.5 `kayfabe-rmrpc` — 6 098 lines, **BUILT** — **the bridge, and no longer an island** . . . 23
5.6 `kayfabe-rt` — 10 079 lines, **BUILT** (L1-M1) . . . 23
5.7 `kayfabe-fwd` — 8 509 lines, **BUILT** — and **part of it has now moved bytes on silicon** . . . 23
5.8 `kayfabe-mocks` — 4 609 lines, **BUILT**, test-only — **and where an invariant hid** . . . 23
5.9 `kayfabe-vmm-kvm` — 2 724 lines, **BUILT** — **defect #57 CLOSED** . . . 24
5.10 `kayfabe-isolate` — 4 246 lines, **BUILT** traits — **with real implementations now** . . . 24
5.11 `kayfabe-isolate-host` — 21 557 lines, **BUILT** — **the real unprivileged host worker** . . . 24
5.12 `kayfabe-trace` — 1 497 lines, **BUILT** vocabulary; **still not threaded** . . . 24
5.13 `kayfabe-shell` — 995 lines, **BUILT** (L1-M2, in progress) . . . 24
5.14 `kayfabe-vmm` — 943 lines, **BUILT** traits — **and the agnosticism claim is now settleable** . . . 24
5.15 `kayfabe-device` — 19 595 lines, **BUILT** (L2 stage Q4) — the chip table and register plane . . . 25
5.16 `kayfabe-chips` — 3 202 lines, **BUILT** (L3) — **the adapter contract, finally measured** . . . 25
5.17 `kayfabe-vmm-qemu` — 5 852 lines, **BUILT** (L2) — the QEMU memory plane . . . 25
5.18 `kayfabe-qemu-raw` — 18 720 lines, **BUILT** (L2) — **the composition root and the second** `unsafe` **crate** . . . 25
5.19 `kayfabe-crec` — 2 041 lines, **BUILT**, harness-only — the C-trace differential . . . 25
5.20 `kayfabe-completion` — 756 lines, **BUILT** . . . 25
5.21 `kayfabe-util` — 1 796 lines, **BUILT** . . . 25
5.22 `kayfabe-arch` — 2 942 lines, **BUILT** traits — **with three production implementations** . . . 26
5.23 `kayfabe-mmu` — 5 128 (**7 006** at `5f323dd5`) lines, table **BUILT**; **walker built**; **GPGA and refresh are DESIGN-ONLY** . . . 26

6 The invariant catalogue **26**
6.1 The two-layer trust model . . . 28

7 What is tested, how, and what that does and does not prove **30**
7.1 The shape of the suite . . . 30
7.2 The CI gates — and the gate that GitHub cannot be . . . 30
7.3 The claim ledger — a gate on the paper's own vocabulary . . . 31
7.4 Exercised versus merely present — the project's own core doctrine . . . 32
7.5 The mean test, and composed versus isolated runs . . . 33
7.6 The conservation ledger . . . 33
7.7 The mutation gate — **five** lies deep, and the fifth is the best one . . . 33
7.8 The OS-free configuration, and why it exists . . . 34
7.9 What the suite does not prove . . . 35

8 Discussion: the good, the bad, and the risky **35**
8.1 What is genuinely good . . . 35
8.2 ★★★ The execution plane was the wall — and it was crossed . . . 36
8.3 ★★★★★ The ladder to first compute — `cup2` → `cup3` → `cup8` . . . 36
8.4 ★★★ Performance: the binding constraint moved twice, and every number here needs a scope . . . 38
8.5 ★★★★★ The LLM wall: a ladder of controls, a fatal PAIR, and a milestone graded on a proxy . . . 41
8.6 ★★ A trace of a boot that SUCCEEDS — and the ladder, enumerated . . . 44
8.7 ★ The bridge, and the two NVIDIA findings that survived being right . . . 46
8.8 ★★ A live-memory-free defect — and the independent oracle had the identical bug . . . 47
8.9 ★★ The quality instruments were wrong — three of them, in one day . . . 48
8.10 `kayfabe-gsp`: boot is modelled, reboot is modelled and never exercised, and no teardown has ever been run 49
8.11 ★★★★★ Sequential multi-process: the wall fell, three processes pass, and the fourth does not . . . 52
8.12 ★★★★★ The fifth device open wedges the GPU — and the code predicted it, then filed it as a trade-off 53
8.13 What contact with hardware has and has not bought . . . 55
8.14 The quadratic control plane — an open, measured DoS . . . 56
8.15 Reclamation is the least finished plane . . . 57
8.16 arm64 is measured for the core and unmeasured for the shell . . . 57
8.17 ★★★ R1 was broken on the one path the suite could not see . . . 57
8.18 Multi-tenancy: two shapes contained, one that bites, and no tenant axis at all . . . 58
8.19 The QEMU $\geq$ 10.2 floor buys less than the decision that took it assumed . . . 59
8.20 The bench — rented, serialised, and now more than one box . . . 60
8.21 The recurring failure mode: claims that were true *once* . . . 61
8.22 Risks this document adds . . . 62

9 How to read the code **64**
9.1 Entry points . . . 64
9.2 Conventions . . . 64
9.3 Which design documents to read, in order . . . 65

10 Claims corrected while writing this paper **66**
10.1 What this paper could not verify . . . 73

11 Open questions — posed for a reader with no access to this tree **74**

A Measurement method **80**

1 What this document is, and how to attack it

This is a review document. Its job is to describe kayfabe accurately enough that a competent systems engineer who has never seen it can (a) explain what it does and why it is hard, (b) find any component in the tree, (c) judge whether the claims are believable, and (d) find the weak points without help. It is not a status report and it is not a pitch.

Four consequences shape everything below.

Numbers trace to something. Every count, score and measurement names its source. Where a number is an estimate it says so; where a claim could not be verified it is marked [unverified] rather than softened. Appendix A gives the exact commands used.

Unbuilt is stated as unbuilt. Most of the planned system does not exist. The build order is deliberate, but a document that blurs "designed" and "running" is worthless for the stated purpose. The layer diagram (Figure 1) and the data-path diagram (Figure 3) both carry explicit **NOT BUILT** bands.

The uncomfortable section is the longest. §8 is where to start if you only read one section.

This document applies the project's own doctrine to itself. kayfabe's testing doctrine opens with the rule *"a green instrument on an unexercised path is worse than no instrument — it reads as evidence, and nobody re-checks a zero"* (`docs/design/testing_doctrine.md` §1). The same rule applies to a whitepaper. So where this document says a thing is tested, it also says what that test would have to be wrong about for the claim to fail.

The single most important paragraph in this paper. Each of the five revisions of this document has led with a claim about *what the critical path is*, and **four of the five were falsified by the work they named, usually within days**. Revision one said the critical path was a disconnected crate; two said it was the bridge; three said it was the guest driver reaching the bridge; four said it was the execution plane, which "cannot be faked". **All four have been crossed.** So read this revision's version knowing the base rate. ⊘ **UPDATED 2026-08-20 20:32: make that FIVE of five, and the fifth did not survive the afternoon it was written in.** This revision's version — *"the remaining wall is our own object model, reached through closed CUDA userspace"* — was refuted **three minutes** after this document was committed (§10 row 44), and its headline blocker was fixed **four hours and fifty-five minutes** after it (row 43, §8.11). *The base rate is now 5/5 and the interval is measured in minutes.*

What is established, and it is a real result. A stock, unpatched NVIDIA 580.159.04 open kernel module, in a KVM guest, against our emulated GA106 and faked GSP, runs a CUDA program that produces a numerically correct answer computed by the *host* GPU's graphics/compute engine — `cup3`'s `43`, and `cup8`'s 2048^2 and 3072^2 matmuls at `bad=0 maxerr=0`. The guest driver is unmodified. The host work is issued by an unprivileged, sandboxed process. That is the thing this project exists to do, and it is done for one workload shape on one part.

What is not established, and it is the larger half.

- ⊘⊘⊘ **CORRECTED 2026-09-11 — the model RUNS, and the honest replacement bullet is about rate and reproducibility rather than capability.** [measured] 2026-09-09, rev `c8fe67d6`: 16 tokens, **text byte-identical to the same-boot CPU oracle**, host `Xid 0` (§8.5). **What is not established** is that it holds: **45 s per token**, a harness grade that is a *count* rather than a content assertion (the project's own requirements mark that RED and demand $n \geq 3$ boots), $n = 1$–2 boots, no native baseline on the same box, a [measured] race under other arms, and **no run at all under the three blocking constraints of §4.4**. The superseded bullet follows. *"Nothing above a hand-written CUDA kernel runs.* The CUDA runtime now initialises and `torch.cuda.is_available()` is `True` — **and the model still does not run**: torch's full CUDA init raises *"CUDA unknown error"* and produces **zero tokens** (§8.5). The LLM workload is this project's own definition of a useful v1 and it has never produced a token. ⚠ **Last measured at `3887be37` (`w355llm1`); it has not been re-run across the multi-process fixes of §8.11**, so its current status is [unverified] rather than known-failing. ⊘ And the *cause* this bullet used to name — our object graph — was refuted the same afternoon (§8.5): the cause is unidentified.
- **Multi-process works for two shapes now, and the general claim is still not made.** Two *concurrent* guest CUDA processes both computed `43` ([measured] 2026-08-14, w299, rev `f459cffa`), including on a staggered arm where the second started while the first was demonstrably *inside* `cuCtxCreate` — so the C artifact's #14 shape did not reproduce. ⊘ **The rung scoped itself**: two identical short workloads whose CE work is emulated, and the underlying serialisation (the QEMU BQL plus a device-global unranked mutex held across the host verb) is real, structural, and *latent rather than absent* — it arrives when the inline span grows.
  ⊘ **CORRECTED 2026-08-20 at `56dc01f3`, and this bullet used to end with the paper's headline blocker.** It read: *"and a second context sequentially in one boot still hung at `cuCtxCreate` (w311)."* **That is now false.** [measured] `w367`, rev `cca2eb4b`, one boot: a first torch process, a second with no `nvidia-smi` between, and a third after `nvidia-smi` **all return** `TORCH_OK`, with **8/8** completions and *identical* doorbell counts (104 `Ce` + 403 `GrCompute`) each, and host `dmesg` at 0 `NVRM` / 0 `Xid` (§8.11). ⚠ **A *fourth* process still fails**, above the `ioctl` boundary, cause unknown; $n = 1$ boot. *Nothing here is a multi-tenancy claim.*
- **There is no tenant axis at all.** No `VmId`, `TenantId` or `GuestId` exists anywhere in `crates/`. Cross-VM separation is the incidental consequence of two VMs being two host processes, and at the one place two tenants meet — the

host NVIDIA driver — RM's own cross-client check is defeated by a shared euid. **No two-VM run has ever been attempted.** §8.18.

- **Performance is 22–81× off native for large kernels and dominated by our own doorbell handler for small ones** (§8.4). The operands sit in host system memory reached at **2.51 GB/s** against VRAM's 313.5.
- **Everything is one part, one driver, one guest kernel, one guest at a time.** GA106, 580.159.04 open, and n is small: this project has measured a single-boot 43 to be wrong **1 time in 5**, so a single green boot is weak evidence and is treated as such throughout.

★ **And one instrument this paper holds itself to is currently RED.** The repository's claim ledger (§7.3) — a gate that asks whether every block using the word *measured* names what is behind it — fails at this pin: **1 166 unattributed against a bar of 381, 90 conflated against 66, 36 bare-hardware against 17**. The bars have not been moved since 2026-07-31 while the tree tripled. That is reported here rather than in a footnote because a document that quotes a gate is obliged to say when the gate is failing.

2 The idea

2.1 The problem

Giving a virtual machine a real GPU normally means one of two things. **IOMMU passthrough** hands the whole physical device to exactly one guest: simple, fast, and it gives every other guest nothing. **Vendor virtualization** (SR-IOV, NVIDIA vGPU) shares the device properly, but it exists on datacenter parts under licences, not on the consumer cards most people own.

kayfabe is a third option: **share one commodity GPU among several untrusted virtual machines, with per-guest-process blast-radius containment, using only unprivileged host processes**. The multi-tenant part is the point. A single cross-tenant leak, or a hostile guest that can wedge the box, would be fatal to the thesis.

2.2 Vocabulary

The rest of this paper leans on five terms.

- **RM** — NVIDIA's *Resource Manager*, the object model inside the driver. Everything a GPU program touches is an RM object with a handle: clients, devices, address spaces, channels, memory, engine contexts. Guest software allocates and frees them through ioctls and RPCs.
- **GSP** — the *GPU System Processor*: on modern NVIDIA hardware a real CPU core on the GPU die, running a firmware copy of much of the resource manager. The host driver no longer programs most of the hardware directly; it sends RPC messages to the GSP over a ring buffer in memory.
- **VAS / PDB** — a GPU virtual address space, and its *page directory base*: the physical address of the root of that address space's page tables. The GPU's MMU keys page tables by PDB, which makes **the PDB the real memory boundary** — not the client handle.
- **Channel / vChid** — channels are the GPU's work-submission queues; the *virtual channel id* is how a submission is routed to one. **vChid is the execution boundary.**
- **Doorbell** — the MMIO register write that says "channel N has work waiting". The value written is a token that encodes the vChid.

2.3 Mode 1 and Mode 2

The predecessor project defines two modes, and kayfabe is only the second.

Mode 1 forwards the guest's *userspace* NVIDIA ioctls to a host stub that replays them against the real `/dev/nvidia*`. It needs a guest kernel module and a virtio transport, so the guest is modified, and it caps you at "a Linux guest running our module". It is mature: 22 real GPU applications at host parity, Vulkan, OpenGL, a working desktop present path, five security audits.

Mode 2 inverts it. The guest runs the **stock, unmodified NVIDIA kernel driver** against an *emulated* PCI device that presents a plausible GPU and a *faked GSP*. We implement the RPC ring the driver talks to and answer as the firmware would. The guest driver believes it owns hardware: it allocates RM objects, builds GPU page tables, creates channels, and rings doorbells. We are on the other side of that conversation, and from the protocol traffic we reconstruct what the guest program was actually trying to do — then do the equivalent thing on a real host GPU through ordinary unprivileged userspace operations, inside a sandbox.

Mode 2 needs *nothing* from the guest, which is the entire reason it is worth the difficulty.

2.4 The correctness criterion, and why it is not "replay everything"

The naive reading of Mode 2 is "intercept the guest's privileged operations and perform them on the host". That is wrong, and the project's forwarding model says so explicitly:

> *"The job of Mode-2 is **not** to replay those low-level steps on the host. It is to **recover the original userspace-level intent and re-express it as the normal, unprivileged host userspace operations** — exactly the operations Mode-1 forwards directly."*
> [C: docs/design/mode2_forwarding_model.md]

The guest's kernel RM has *already* decomposed an unprivileged userspace intent (`cuCtxCreate`, `cuMemAlloc`, a kernel launch) into privileged GSP-internal steps. Replaying those steps on the host would require privilege we deliberately do not have — and the project has a named lesson about this: an `NV_ERR_INSUFFICIENT_PERMISSIONS` from the host stub *never* means "we lack a privilege we should have", it means "we forwarded at the wrong layer". So the operational split is:

- **Case 1** — the RPC essentially *is* the userspace operation. Re-issue it roughly 1:1 on the host, through the isolate, unprivileged, and let the host's own kernel RM legitimately re-derive the privileged steps for itself.
- **Case 2** — a GSP-internal control with no userspace equivalent (`PROMOTE_CTX`, `GET_CTX_BUFFER_INFO`). **Acknowledge the guest; do nothing on the host.** The host has already done the equivalent for its own context.

The correctness criterion that licenses this is **observable end-state equivalence**, not step fidelity. It is perfectly correct for some guest-kernel operations to complete instantly as fakes, *as long as* the one operation that actually carries the work triggers the real host-side chain. The hard boundary on the other side is equally explicit: a completion that gates guest *userspace* must be a real host write. Forging one would make the whole project a lie, and the design forbids it by name.

2.5 Why this is hard

1. **The protocol is not documented and not stable.** It varies on two independent axes: the *driver version* (wire layouts) and the *GPU generation* (silicon behaviour). kayfabe separates these as "Axis A" and "Axis B" and gives each its own trait seam, because conflating them is its own bug class.
2. **Some things cannot be faked at all.** GR's golden context is produced by FECS/GPCCS microcode on real silicon. There is no way to fabricate one. The design's response is to never emulate an engine — forward the guest's own pushbuffer and let real silicon produce the context.
3. **Address translation is a trust problem, not a lookup problem.** The guest builds GPU page tables in memory it controls. Reading them at an arbitrary instant means acting on bytes an adversary can change under you.
4. **Identity collides by construction.** Guest handles and guest VAs are per-process namespaces. Two processes using identical values is normal. Anything that keys on them is wrong (Figure 4).
5. **The failure modes are global.** A mis-resolved address is not a wrong answer; it is a host GPU MMU fault that takes out a channel, and in the worst case a state the guest driver cannot recover from without a full VM restart.
6. **Everything is concurrent.** N guest vCPUs, several guest processes, several host isolate processes, an interrupt path, and a host RM that *serializes every ioctl per client* so that one wedged verb blocks every isolate in D-state.
7. **★★★ And the last one, which this project learned by hitting it: the driver does not only ask questions, it runs experiments.** A protocol can be answered. A *functional test* cannot. Partway through initialisation the guest driver writes a sentinel to framebuffer, asks a copy engine to move it, reads it back and compares — unconditionally, with no code path that skips it on the hardware we target. **There is no reply that satisfies a read-back comparison.** Everything above is about recovering intent from a conversation; this is the point at which the conversation stops and something has to actually happen. §8.2.

3 Provenance: the C artifact, and what it actually proved

kayfabe is a rewrite, not a greenfield project. The C artifact at `/workspace/nvidia-gpu-passthrough` is the working prior implementation and the differential oracle for everything the rewrite claims about NVIDIA's behaviour. Being precise about what it did and did not prove is the foundation of every other claim here.

Claim	Mode	What was actually measured
22 real GPU apps at host parity	1	**True.** `tests/perf/realapp_matrix.md`, 2026-06-01. Same binary, host and guest, strictly serially on one RTX 3060, with a correctness check alongside throughput; parity gate ≥ 0.90. 10 CUDA benchmarks, 7 PyTorch, 2 LLM, 3 graphics. **Caveat the doc states itself:** NVENC is *partial*, not parity (`InitializeEncoder` fails); NVDEC excluded (broken on the host baseline too).
7B LLM at 63 tok/s	1	**True but mis-framed.** 63.4 tok/s is a Mode-1 A/B endpoint after a caching fix, not a headline result. The audited same-day parity figure is **host 67.8 → guest 66.0 tok/s, 0.97×** (Qwen2.5-7B Q4_K_M, `-ngl 99`, RTX 3060). Quote the ratio.
Mode-2 bare-metal parity	2	⊘ **Weaker than it has been quoted as, and the correction is the project's own. "20 → 60 tok/s" is the TITLE OF A PLAN, not a result**, and its 60 was borrowed from *Mode 1* — a different architecture with no doorbell trap. *"I read a completed task's title as a result."* What the C *measured*, with logs, is **19.5–26.6 tok/s at** $\pm 40\%$ **run-to-run** (best single observation 38.3). A separate figure of **49.9 vs 47.5 tok/s host-native** on the same RTX 3050 does exist and is credible — **but its commit is documentation-only: no log and no trace was committed**, unlike the matmul beside it. ⇒ *treat 49.9 as plausible and uncorroborated; treat 20–38 as measured.* The earlier 20→50 gap was **nested-virt vmexit tax**, not design, and the project retracted its own "~21% residual" claim as a comparison across two different GPUs.
Mode-2 CUDA correctness	2	**True.** Full fake-boot + GSP RPC + GMMU + CE + IRQ; `cuInit`, `cuCtxCreate`, byte-exact matmul; PyTorch 2.5.1 *single-process*. Bugs #12 (second-context hang) and #13 (multi-iteration hang) resolved on hardware.
What the C artifact never proved		
Mode-2 multi-process	2	**Never.** Bug #14 is open. Root-caused on hardware 2026-07-24 by a host Xid 31 `FAULT_PDE` naming the loser's exact GR channel: the second process's identical guest VAs were never published into *its own* host address space. ★ **The Rust port has since done both halves, and this row is about the C.** Two *concurrent* processes computed on 2026-08-14 (w299); *sequential* processes — three of them — passed on 2026-08-20 (`w367`, §8.11).
Mode-2 sequential processes	2	**Broken at the C artifact's HEAD**, and this row is about the C rather than the port. Measured 2026-07-25: exactly *one* CUDA process works per QEMU lifetime; the second fails `cuInit`→999 regardless of how the first exited. The "fresh boot per clean run" bench discipline *is* this bug, not a precaution. ⊘ **SUPERSEDED FOR THE PORT, 2026-08-20 (`56dc01f3`):** three sequential guest CUDA processes now pass in one boot with **8/8** completions and identical doorbell counts each (`w367`, §8.11). ⚠ The *fourth* still fails. The C's limitation was **not** inherited after all — the port's cause was its own: a framebuffer join orphaned by an exited process.
Mode-2 graphics / display	2	**Never.** All Vulkan/OpenGL/present results are Mode 1.
Mode-2 on the 22-app matrix	2	**Never run.** It is Mode-2's definition of done.
Multi-GPU, MIG, non-Ampere	—	**Never.** Ampere GA106 only. Turing/Ada/Hopper/Blackwell rows in the ABI doc are all marked *assumption — verify*.
Mode-2 security posture	2	**Not audited.** The five security audits are Mode-1-era. The 2026-07-25 bench found live guest-reachable Mode-2 defects, including parsing arbitrary guest RAM as GSP RPC elements after a guest-triggered driver reload.

The structural reason for the rewrite. #14 is not a bug that could be patched. The architecture document states it in one line: *"the emulator's completion, execution, isolate, and GPA-arena planes are each a single-process singleton, and they fail in dependency order."* Four rounds of investigation each found a different plane; they were

one structural fact observed four times. Making per-process separation structural rather than retrofitted is the rewrite's founding requirement, and it is why the diagram in Figure 4 is the centre of the design.

One provenance correction worth carrying. Two C design documents state the C baseline encodes *"~14 months of quirk capture"*. Git says the first commit is 2026-05-24 and the last is 2026-08-01: **ten weeks, 753 commits**. Either the figure means agent-hours, or it is wrong. Use the verifiable number.

3.1 ★★ The C artifact is a standing oracle — and it has FIVE measured limits, the fifth different in kind

Two virtualizers facing the same guest driver see identical MMIO iff they behave identically, so a divergence at guest access N localises the fault to our reply at $N-1$: one message, not one layer. That is why the C was promoted from history to a *standing differential oracle*, and why it is still maintained. Five committed captures exist (`traces/mode2_c_reference/`), all dense with `n_errors=0`; two of them are committed *decompressed* into this repository so the harness needs no decoder and no third-party crate.

Capture	Records	Hermetic	What it is for
`cap1_coldboot_hermetic`	359 062	yes	cold GSP bring-up to `GSP_INIT_DONE`; 139 821 PROM/VBIOS reads
`cap1b_..._d6`	360 725	yes	★ the same experiment re-taken with the continuation elements *witnessed* — **the only trace a replay can be closed over**
`cap2_stalequeue_negative`	886 999	no	the `WPR2-already-up` chain
`cap2b_stalequeue_nofn47`	862 940	no	★ the real negative: **378 GSP elements parsed out of arbitrary guest RAM and answered** `NV_OK`
`cap3_matmul_forwarding`	532 824	no	the decision planes; `cuCtxCreate` → matmul at `bad=0 maxerr=0`

The four limits of a C *recording*, all pre-registered before the capture. **(i)** On the **copy plane** the completion has *no* C oracle: there the C *forges* completions, and a grep over its isolate verbs finds seventeen call sites and zero poll/event verbs. **A green diff says nothing about that plane**, and it is the likeliest source of false confidence. ⊘ **But see the scoping box below — this limit was stated too broadly for four rungs and the over-statement was expensive.** **(ii)** The diff will never be green end-to-end, and a green diff would itself be the bug: the C has no refusal vocabulary, so every must-differ row is a position where it emits a positive event and we emit a refusal. **(iii)** Any forwarding-mode trace is non-hermetic by construction — `pci_dma_map` is an uninstrumented channel and the host GPU DMAs into guest RAM behind every recorder. **(iv)** The archived logs are unusable: the C's sequence counter is incremented *inside* its log statement, so it counts logged BAR0 accesses only. ★ The load-bearing good news is that a single-counter total order is nonetheless *sound* — the device creates no thread, bottom half or timer, and all four `MemoryRegionOps` run under the BQL — which upgrades a previously [inferred] assumption to an evidenced one.

★★★ SCOPE LIMIT (i) BEFORE USING IT — it was over-drawn, and the over-draw cost a day. *"The C forges its completions"* is **true of the copy plane and FALSE of the GR/compute plane**, and the passing trace says so in its own self-describing header: `m2fwd=1 m2exec=1 m2hostsem=0 m2cefwd=0 m2cexec=0`.

Read those flags. `m2exec=1`: **the execution plane was ON.** `m2hostsem=0`: **the host-semaphore forge was OFF.** The forge exists in the C's source and **wrote nothing to the completion page on the green runs** — it is scoped to the *kernel* scrubber channels and explicitly excludes user GR and CE channels. **The real host GR engine executed the guest's context-init pushbuffer and wrote the completion itself. ⇒ the C *is* an oracle for GR engine execution**, which is precisely what limit (i) as previously written declared it to be *"no oracle at all"* for. ⊘ And `m2cefwd=0` contradicts, from the trace's own header, a sentence this project repeated for weeks: that the flag was *"on every green run"*.

★★★★★ What the mis-scoping cost. Reading the C as *"it forged everything"* **retired the only question that mattered** — *how did the C's host GR engine get a complete address space?* The answer was in the C's own source the whole time, one line, in a comment: *"fault-safe: a mapping is always backed before the engine that uses it runs."* That sentence is the invariant §8.3 describes, and a whole class of faults chased one at a time was one instance of it.

⚠ **And the correction itself needed a correction**, which is why this box is worded the way it is: the first version of it listed *two* address-table population sources, both derived from page tables, while a second document in the same tree listed a **third** — an RPC capture — and *the two documents gave opposite answers to the question the fix turned on*. The RPC capture is real and is the one that matters. *Citing the oracle is not the oracle being right, and citing a correction is not the correction being complete.*

★★★ **The FIFTH limit, and it is different in kind: here the oracle is POSITIVELY WRONG, not blind.** The C's captured control table `mode2_initctrl_ga106.h` has **56 rows**. A census (`scripts/census_initctrl.py`, pinned by a test) splits them: **29 complete**, **16 truncated** ($0 <$ `dlen` $<$ `psize`) and **11 empty** (`dlen = 0`, 19.6%). All eleven empty rows were asked of a real GA106 — an RTX 3060 running the open 580.159.04 modules rebuilt with a probe, 2026-08-01, `traces/real_ga106/` — and **nine of the eleven are contradicted**.

The sharpest is `0x20802a08`, `CE_GET_FAULT_METHOD_BUFFER_SIZE`: its empty row decodes as size **0**; the part answers **20 480**. RM DMAs CE fault records into a buffer of exactly that size, so trusting the empty row was **a buffer overrun with a hardware writer**, not merely a wrong number.

★ **And the two that are *not* wrong are the important ones.** `0x20800a70` has `psize = 0`, so nothing could have failed to be captured — checkable from the row itself. `0x20800a4c` genuinely answers zero on this part because SMC is disabled — and **nothing about the row says so**. It is byte-identical to the nine that are wrong. ⊘ **An empty capture is evidence of nothing, not evidence of emptiness**: believing it would have been right once and wrong nine times *for the same reason*. The gate that now enforces this refuses the predicate `psize` $> 0 \wedge$ `dlen` $= 0$ rather than a list of eleven ids, because a list is a smaller true statement that goes stale at the 57th row.

⊘ **And the third class is a live hole this paper will not paper over.** The 16 truncated rows are *larger* than the empty class and **have not been measured**. A truncated row *passes* a `dlen == 0` test because it has a body, and its uncaptured tail arrives as zeros — the exact failure `0x20802a08` cost four rungs, with the one signal that caught it removed. All sixteen kept-prefixes end in zero bytes, so the trim hypothesis is dead and the tail is genuinely *unknown*.

★ **This scopes the oracle rather than devaluing it.** It remains the only implementation a real NVIDIA driver has ever accepted end to end, and its non-empty bodies are corroborated by hardware where they have been checked. What is demoted is one move: reading an empty `ctl_` array as *"hardware says zero"*.

★★★ **How it survived four rungs, which is the part that generalises.** A gate demanding a `C:` source citation was *satisfied* by a row that cited the empty body *as corroboration*. **Citing the oracle is not the oracle being right. A citation gate checks that a claim is sourced; it can never check that the source says what the claim says.** The sibling was found from the opposite side in the same week: three shipped rows cited `mode2_initctrl_ga106.h` lines belonging to *different* controls — every value right, every address wrong, the citation gate satisfied throughout. And one turn deeper still: the replacement gate's own first version accepted the *phrase* `"real GA106"` as evidence, which a row satisfied with a claim about hardware manufactured out of the empty capture. It now demands a path to a committed artifact. **A gate that checks for the vocabulary of evidence can be satisfied by writing the vocabulary down.**

Has any replay been closed? No — and the wall changed kind, which is the result. The `cap1` replay stops at transaction **978**, on a *missing observation*: the C never recorded the continuation elements, so the run dies at a read the capture does not contain. `cap1b` witnesses them (32/32, proven against a control not to have changed the reply stream) and the limit moves to **1028** — where every read is *observed* and the refusal is `QueueFull`, because the C's only accesses to the status-queue header are *writes*. **That is oracle blindness, not a Rust defect**: the guest really is advancing a pointer we are right to consult, and no capture taken from this C can ever contain it. ⊘ **Do not treat** `QueueFull` **as a bug to be tuned away** — relaxing our flow control to get a greener diff would delete a correctness property the oracle simply cannot see. *The diff is the instrument; the instrument does not get to edit the subject.*

And the oracle is perishable. Recording a C trace needs a running C deployment: a built QEMU, a guest pinned to 6.8.0-117, and host driver 580.159.04. That deployment has died with a rented box before and will again. **Recorded traces are the durable artifact; a bootable C on a rented box is not.**

4 The architecture

4.1 Layers and ports

The one defining constraint. The logic core is a pure state machine over guest-supplied bytes: no OS, no syscalls, no hypervisor types, no wall clock, no `#[repr(C)]` NVIDIA struct layouts, and no concrete GPU-generation or driver-version name anywhere. Everything effectful crosses a trait seam. `unsafe_code = "forbid"` is a workspace lint; every core type is compile-time-asserted `Send + Sync`.

This is enforced, not intended. Four CI greps run on every push: a *hexagonal boundary* grep that fails if `eventfd|epoll|timerfd|rawfd|libc|O_NONBLOCK` appears in any of the 11 pure crates; a *VMM-vocabulary* grep that

fails if any QEMU concept (`BQL`, `MemoryRegion`, `qemu_bh`, `iothread`, …) appears in those 11 plus `kayfabe-rt`; a *generation-name* grep over 10 of them that fails on any concrete chip or GPU-generation name outside a comment; and a *GPA-accessor* gate that asserts `.gpa_read(`/`.gpa_write(` appears **exactly once** in the entire forwarding crate, and **nowhere** in the other nine.

The claim that pays for this constraint is the **adapter contract**: bringing up a new GPU generation is `impl Arch for <Gen>` in an adapter crate with *zero edits to any logic crate*. The standing proof is that the whole suite runs the real core against `MockArch`, whose encodings are deliberately non-NVIDIA — if the core had absorbed a real encoding anywhere, the mock would not work.

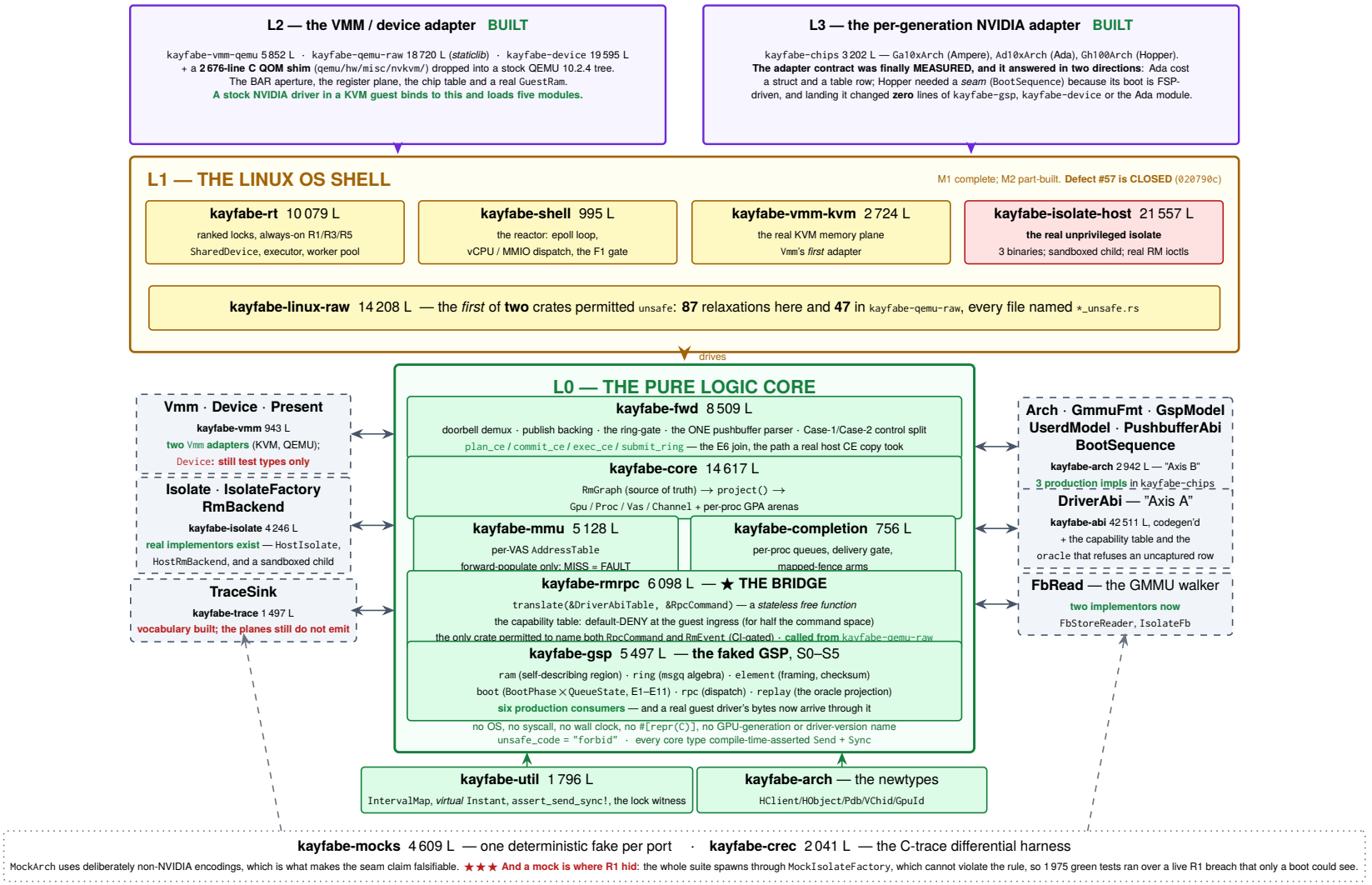


Figure 1: The layer model and the hexagonal ports. Line counts are measured at `5c367a38` (`crates/<name>/src/**/*.rs`; `kayfabe-abi`'s figure excludes its detached offline generator, 3 284 lines). ★★★ **The bands inverted at this revision**: L2 and L3 were drawn as dashed **NOT BUILT** boxes at the previous pin and are now built, shipped and booted against. Implementor census, counted from `impl <Trait> for` across the tree and re-derived rather than carried over: `Arch` has **three** production implementations (`Ga10xArch`, `Ad10xArch`, `Gh100Arch` in `kayfabe-chips`); `Isolate`, `IsolateFactory` and `RmBackend` have real ones in `kayfabe-isolate-host`; `Vmm` has *two* real adapters (`KvmVmm`, `QemuVmm`); `FbRead`, which the previous revision recorded as having *no implementation of any kind*, has two; `GuestRam` has a production implementor (`MachineRam`, in `kayfabe-qemu-raw`). `Device` **is the one port on this diagram whose only implementors are still test types** — the QEMU shell reaches the core through `GuestRam` and the register plane instead.

4.2 The crate dependency graph

Figure 2 is derived from every `Cargo.toml` in the tree and cross-checked against `Cargo.lock`; it is not drawn from the design documents. Three findings come straight off it.

The external dependency surface is still essentially zero, and that survived a 33% growth in members. In the whole workspace there is exactly one non-dev external crate: `libc`, a real dependency of `kayfabe-linux-raw` only, and a *dev*-dependency of `kayfabe-vmm-kvm` and `tests` for asserting exact errnos — plus `trybuild` and `proptest`, both dev-only. **Twenty-three of the twenty-four** members have no non-dev external dependency of any kind. *Six crates were added since the last pin and not one of them added a dependency*, including the two that talk to QEMU and the one that talks to the host driver. Whatever else is true, the supply-chain surface is not a place to look for problems.

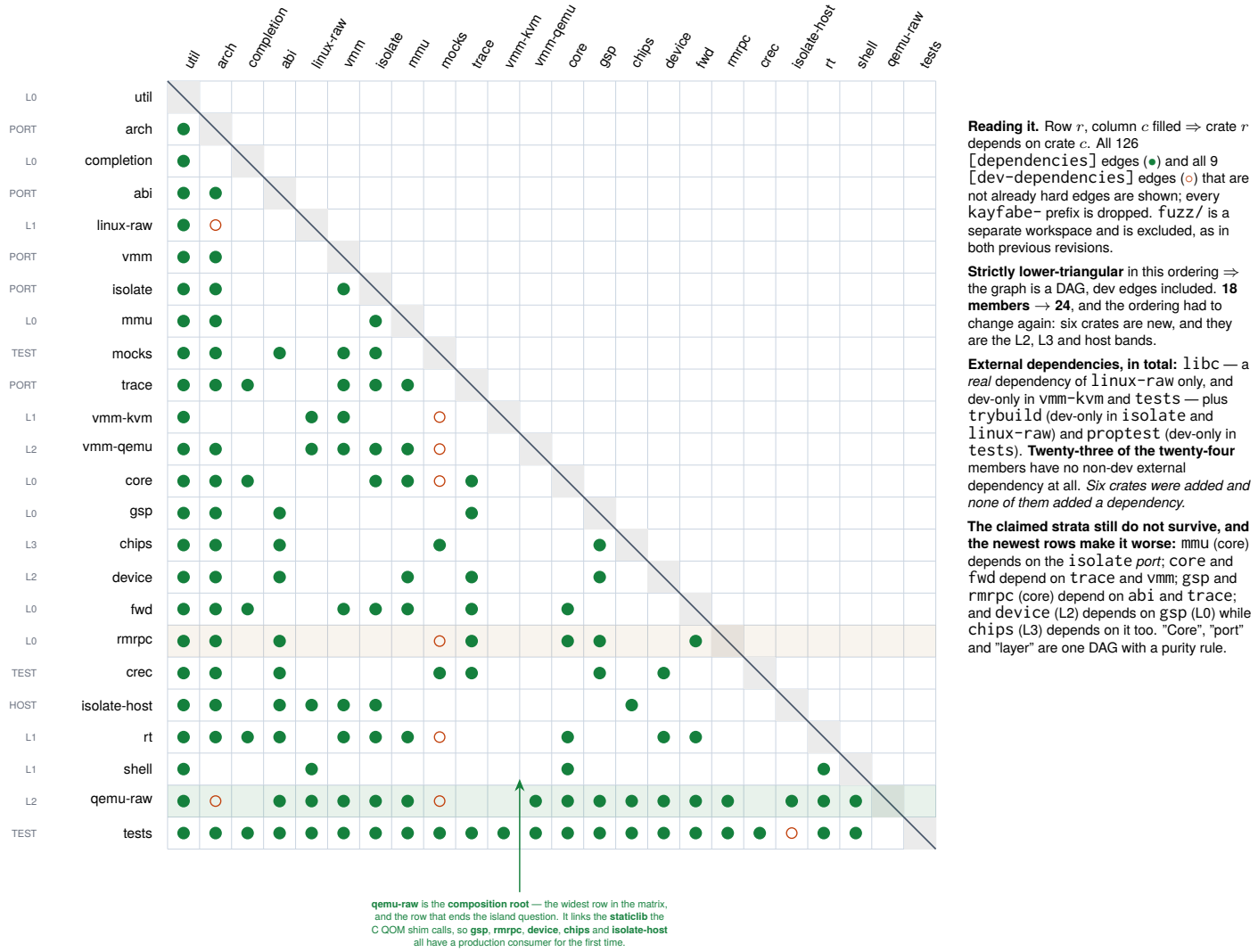
The graph is acyclic, including dev edges, which the strictly lower-triangular shape of the matrix demonstrates directly. *The ordering that makes it lower-triangular had to change again*, for the third revision running, and this time by more than a row: **18 members became 24**. The new rows are the bands the previous matrix could not draw because they did not exist — `vmm-qemu`, `qemu-raw` and `device` (L2), `chips` (L3), `isolate-host` (the host worker) and `crec` (the C-trace harness).

But the claimed strata are not disjoint, the documents imply they are, and the newest rows make it worse rather than better. `kayfabe-mmu` — a core crate — depends on the `kayfabe-isolate` *port* (it needs `HostHandle`). `kayfabe-core` and `kayfabe-fwd` depend on the `trace` and `vmm` ports. `kayfabe-gsp` — also a core crate — depends on `abi` and `trace`. And now `kayfabe-device` (L2) depends on `kayfabe-gsp` (L0) while `kayfabe-chips` (L3) depends on it too. "Core", "port" and "layer" are one DAG with a purity rule, not three tiers. That is a defensible design, but a reader who takes the hexagon picture literally will be surprised by the manifests.

★★ This paper has now been wrong about the island question three times, and the fourth answer finally closes it. Revision one called `kayfabe-abi` and `kayfabe-gsp` "disconnected islands". Revision two sharpened that into its headline — *"the critical path is the bridge, not the crate"* — and revision three recorded that the bridge had been built and **the island had simply moved up one crate**: *"the only manifest that lists* `kayfabe-rmrpc` *is* `tests/Cargo.toml`*."* **That sentence is now false too, and this time the property does not move anywhere.**

Re-measured against the manifests, not inherited. `kayfabe-rmrpc` is named by `crates/kayfabe-qemu-raw/Cargo.toml`, and `kayfabe-gsp` by *six* production manifests (`chips`, `device`, `qemu-raw`, `completion`, `crec`, `rmrpc`). `kayfabe-qemu-raw` is a `staticlib` that a C QOM shim links into a real QEMU binary, so the chain bytes → FSM → `RpcCommand` → `RmEvent` → `Gpu::apply` now runs inside a hypervisor a guest driver is talking to. **The island question is answered, and answering it took three revisions and one commit each time to invalidate the answer.**

★★★ And the successor property this paper printed at the previous revision has been falsified too, in the direction the project wanted. It read: *"on every boot to date the device reports* `doorbells: 0 arrived`*"* — offered explicitly as *"a sentence a single boot can falsify, which is the only kind worth printing here."* **A single boot falsified it.** On the `cup3` boot the device routes **480** doorbells, serves **480** and refuses none (`GrCompute=125`, `Ce=355`) with `unrouted = 0`, and the host GR engine executes what they announce. *The mechanism worked exactly as designed: the claim was stated so that evidence could kill it, and evidence killed it in eleven days.* That is the shape every claim in this document is meant to have, and §11 is the same discipline applied to what is still open.



Reading it. Row r, column c filled $\Rightarrow$ crate r depends on crate c. All 126 `[dependencies]` edges (●) and all 9 `[dev-dependencies]` edges (○) that are not already hard edges are shown; every `kayfabe-` prefix is dropped. `fuzz/` is a separate workspace and is excluded, as in both previous revisions.

Strictly lower-triangular in this ordering $\Rightarrow$ the graph is a DAG, dev edges included. **18 members → 24**, and the ordering had to change again: six crates are new, and they are the L2, L3 and host bands.

External dependencies, in total: `libc` — a *real* dependency of `linux-raw` only, and dev-only in `vmm-kvm` and `tests` — plus `trybuild` (dev-only in `isolate` and `linux-raw`) and `proptest` (dev-only in `tests`). **Twenty-three of the twenty-four** members have no non-dev external dependency at all. *Six crates were added and none of them added a dependency.*

The claimed strata still do not survive, and the newest rows make it worse: `mmu` (core) depends on the `isolate` *port*; `core` and `fwd` depend on `trace` and `vmm`; `gsp` and `rmrpc` (core) depend on `abi` and `trace`; and `device` (L2) depends on `gsp` (L0) while `chips` (L3) depends on it too. "Core", "port" and "layer" are one DAG with a purity rule.

Figure 2: The crate dependency matrix, re-derived from scratch by script from the manifests at `5c367a38` — **24 members**, **134** `[dependencies]` edges and 9 `[dev-dependencies]` edges that are not already hard edges. `kayfabe-qemu-raw` (row 23) is the row that matters: it is the composition root, the `staticlib` a C QOM shim links into a real QEMU, and it depends on *eighteen* of the twenty-two crates above it. ★ **The previous revision's row ordering is no longer topological** — `qemu-raw` has since acquired a dependency on `shell`, which sat below it — so `shell` and `qemu-raw` are swapped here and the strictly-lower-triangular property (which *is* the acyclicity proof) holds again. Drawn in the old order the matrix would have carried one dot above the diagonal and nothing in the build would have said so.

4.3 The data path

Figure 3 traces one guest GPU operation from a CUDA call to real silicon and back, naming the symbol each step lives at. Three things about it are worth stating in prose.

The plan/execute/commit shape is the whole concurrency design. A host RM verb can block for a long time — the host RM serializes every ioctl per client, and it waits *uninterruptibly*, so one wedged verb parks every isolate that shares the client in D-state. Invariant **R1** therefore forbids issuing a verb while any ranked lock is held. That forces a three-phase shape: *plan* under locks, *execute* with no lock held, *commit* after re-acquiring. And because the locks were dropped in the middle, invariant **R5** requires the commit to re-validate everything the plan assumed. This is not free — §8 covers what it costs.

The ring gate is the #14 fix, and it is now structural in the type system. Before any host operation, a doorbell submission's entire working set must already be published into *this* `Vas`'s own host address space. If it is not, the call refuses with `NoVas` having issued zero host operations. The failure it prevents is exactly the one the C artifact hit on real hardware: the losing process's identical guest VAs were never published into its own host GR address space, so the host GPU faulted past the shared prefix. At the previous pin this was a call-graph property — "every path happens to go through `plan_doorbell`". At `634b253` it stopped being one: `VerbPlan::Doorbell` is `#[non_exhaustive]`, so no struct expression for it exists outside `kayfabe-isolate`, and the sole constructor `VerbPlan::gated_doorbell` *runs* the gate. Constructing an ungated doorbell is now compile error E0639, pinned by a `trybuild` row (`crates/kayfabe-isolate/tests/ui/`).

The diagram is now driven end to end by a guest, and the previous revision's replacement headline died with it. Three revisions ago there was no register model, no GSP transport and no RPC decode; two revisions ago all three existed and nothing consumed them; one revision ago the bridge consumed them and nothing fed the bridge; *last* revision the guest fed the bridge and never reached the execution plane, and this paper printed *"the diagram is now two working halves that have not been joined."* **They are joined.**

★★★ **One guest CUDA call, all the way down.** On the `cup3` boot ([measured] 2026-08-14, rev `0655e6aa`, RTX 3060 GA106, host driver 580.159.04 open, stock guest driver) a guest `cuLaunchKernel` traverses every numbered step of Figure 3: the guest's MMIO doorbell store is trapped, decoded to a token, routed to `(GpuId, VChid)` and then `(ProcId, ChanId)`, passed through the #14 ring gate, planned, executed by a real unprivileged host isolate against a real GA106, and the **host GR engine** — not a copy engine, not our emulator — runs the guest's JITed PTX and produces `43`. The device's own census on that boot: `480 arrived, 480 served, 0 REFUSED by name, GrCompute=125 Ce=355 unrouted=0`.

⊘ **What "joined" does not mean, and the three things it hides.** (**1**) The path is green with **eleven relaxations armed by the bench runner**, and **five of them are measured load-bearing** — `VAS_PUBLISH`, `GR_ROUTE`, `GUEST_RING`, `CE_EXECUTOR` and `PT_WITNESS_EXEC` each kill the 43 when returned to their other value. ★★★ **And** `KAYFABE_GR_ROUTE` **still defaults to** `refuse` **in the shipped code**, so *the configuration that computes is not the configuration that ships. A relaxed green is a map, not the milestone.* (**2**) The relaxation *census* itself was wrong once and corrected: two arms measured "inert for `cup3`" turned out to each kill a second workload, so *inert for one workload was read as inert.* (**3**) **Every program that has traversed this path was written against the CUDA *driver* API directly.** The runtime API initialises as of 2026-08-20 and **no model has yet produced a token through it** (§8.5).

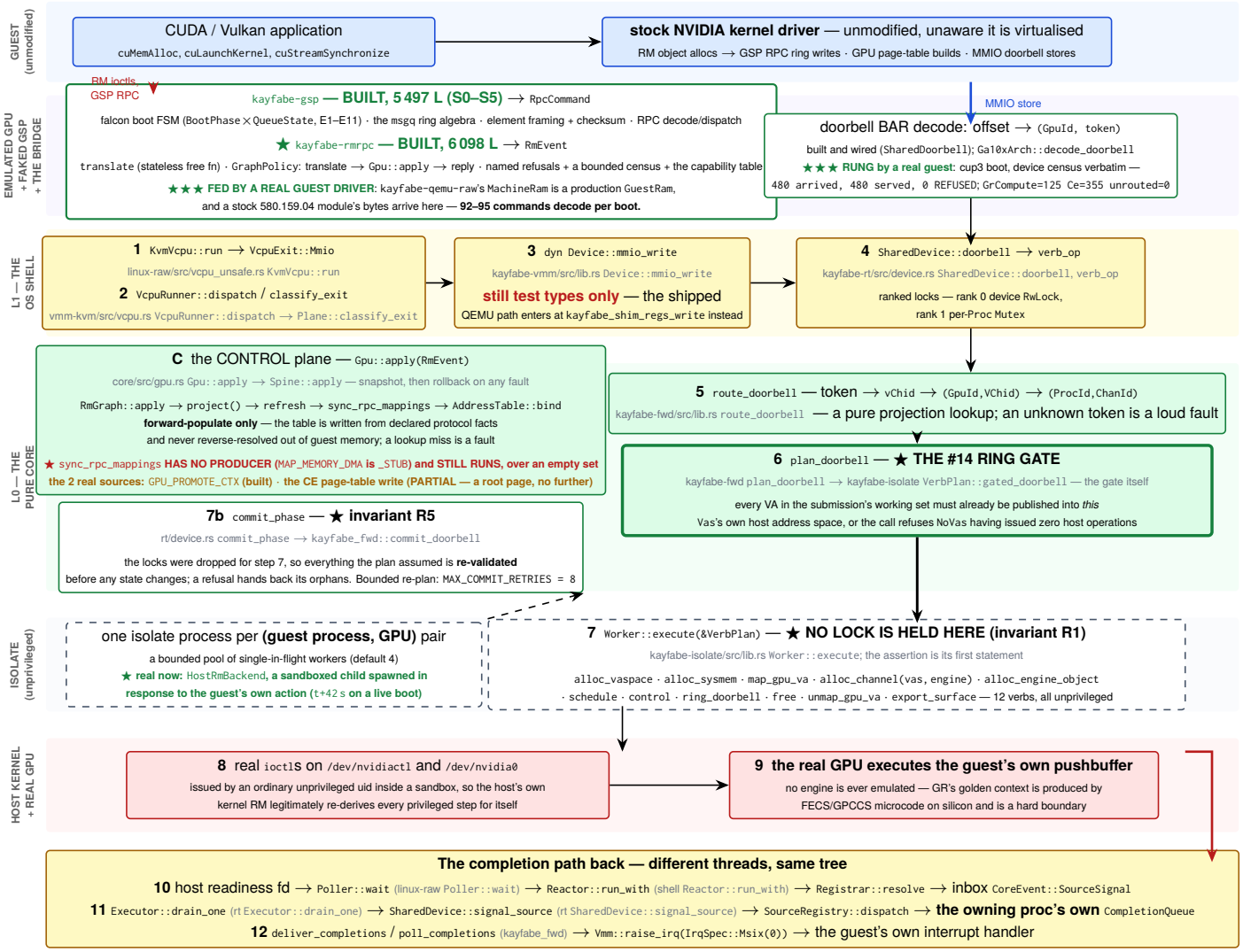


Figure 3: The data path, verified function by function against the tree at `5c367a38`. Numbered steps are the execution plane; **C** is the control plane, which populates the address table from declared protocol facts. Boxes name *symbols*, not line numbers: the repository converted its own 105 pins to symbols in `3fd1455` after they drifted. **What changed at this revision is the joint**: the previous revision's red box was the doorbell — built, wired, and never rung by a guest. **A guest rings it 480 times on the `cup3` boot, every one is served, none is refused, and the host GR engine at step 9 returns a value only it could have computed.** The whole numbered chain runs. Red text marks what does not exist or has not happened, and what is left of it is upstream (`Device` has only test implementors; `sync_rpc_mappings` has no producer because `MAP_MEMORY_DMA` is not on the wire — see §8.7) or downstream (§8.5).

4.4 ★★★★★ NEW 2026-09-11: the blocking model, and the three synchronisation points

★★★★★ This subsection did not exist at the pin, and it replaces the design the rest of §8.4 measures. Everything above describes a device whose work happens *inside* the guest's MMIO traps: a doorbell trap that publishes page tables, a GSP reply produced synchronously inside the trap that asked for it, BAR1 and BAR2 fully trapped. **All three have been restructured since 2026-08-21.** What follows is the current model, what is actually built of it, and the three places where it is stated but not enforced.

The three rules

1. **No blocking work in a vCPU trap.** Owner, 2026-09-06: *"Never do a blocking call during a doorbell write, quickly re-enter the VM. Instead if you need to do mappings, run the doorbell asynchronously. **There is no guarantee that a real GPU gives either** — the doorbell runs immediately in-line, it's a schedule."*
2. **No blocking work under a lock a vCPU takes.** Transitive blocking counts, and the owner states it as a mechanism rather than a principle: *"a blocking call in vcpu also counts if the lock its waiting on to acquire is hold by a thread that has a blocking call"* — and, on the register plane specifically, *"big lock for work should never be taken on the vcpu thread"* and *"the mmio trap should return long before the refresh finished"* (2026-09-11). A worker that holds the vCPU's lock across a slow call has moved the work and kept the wait.
3. **A trap may not exceed 1 ms.**

Rule 1 makes the doorbell a hint rather than a barrier, and that is the important half. A real GPU gives no guarantee that work has not already been scheduled before the doorbell write lands, so **nothing in this system may gate on a doorbell.** The tree had already said so in a type before the ruling arrived: `GuestChannelKind::Emulated` declares `TrapContract::ScheduleAndReturn`, whose own doc reads *"the handler must not run on the vCPU thread"*, and `SharedDoorbell::try_ce_submission` *reported that contract as violated in the same breath* rather than enforcing it. ★ The distinction is now written into the code that does the work: *a barrier finishes its VAS; a hint samples it* — the doorbell-driven pass takes a row-count sample, the barrier paths drain to completion. ⇒ The owner's publish-trigger preference ordering (2026-08-14) puts the doorbell **third**: *"exact GPU boundary (TLB invalidate) > trap the PTE write, as little as possible > deferred publish on doorbell > work under the BQL."*

★★★ Why there are THREE synchronisation points, and not one

This is the load-bearing correction to §6.1, and it took two wrong answers to reach. Page-table state can only change at a point where an honest guest is holding its own locks, and there are exactly three such points. The obvious unification — *"all three eventually lower to a hardware TLB invalidate, so there is really one barrier with three transports"* — was written down, verified as far as RM's `vaspaceInvalidateTlb` lowering to `NV_PFB_PRI_MMU_INVALIDATE`, and then **refuted by the owner on the same day**:

> **Owner, 2026-09-11, correcting both the author and their own earlier speculation:** *"no the driver doesn't unif[y] them in one call, thats why we have 3 entrypoints. dont take that assumption. this is what I thought. my suspicion is that a real bare metal gsp does that in microcode, but since we impersonate that we must implement all 3 entrypoints"* [docs/design/PLAN_barrier_driven_refresh_w434.md]

The reason is the whole thesis of this project pointed at itself. On a GSP part the mapping work happens *inside the GSP*, and the GSP **is us**. A real GSP does its invalidate in microcode; nothing crosses the guest boundary for it. **So for that path no barrier will ever arrive, because there is nobody to hear from: we serviced the map, so we are the one who knows it happened.** A design that waits for a barrier on that path would look correct on any workload whose first synchronisation point is the register write, and would silently never refresh anything the GSP mapped on the guest's behalf.

#	entry point	who tells us	our obligation, and its state
1	**TLB invalidate** — a BAR0 store to `0x00B8_30B0`	the guest, by writing the register	handled; wants scoping to what the register names. ⚠ We *must* clear bit 31 (`_TRIGGER`) or the guest wedges: CPU-RM spin-polls it with only a timeout check in the loop.
2	**RM call** — a map/unmap RPC we service	**nobody. We are the GSP.**	refresh inside our own handler, before the reply. **BUILT** for `GPU_PROMOTE_CTX` (`w441`) and **generalised to every GSP RPC that changes an address space** (`w456`): latch under the lock, refresh off it, drain before the command drain so a posted reply is the guest's licence to use the space.
3	**UVM kernel channel** — `MEM_OP MMU_TLB_INVALIDATE`	the guest, as a pushbuffer method	consume it and refresh *before* we forward. ⊘ **Wired** (`w437`) **and NEVER EXECUTED**: `MEMOP-CENSUS seen=0`, `refresh passes: 0` on both runs of the `w447` milestone.

★ **Point 3 needs no new wait object, and that is the cleanest argument in the plan.** The `MEM_OP` and the completion semaphore are in the *same* pushbuffer, we parse it before we forward it, and the engine cannot start until *we* forward. **Ordering our own refresh before our own forward IS the block**, and it needs no guarantee from the guest. That is the C artifact's invariant stated from the other side (`src/qemu/nvkvm_gpu_emul.c:582`): *"a mapping is always backed before the engine that uses it runs."*

And none of the three blocks a vCPU. Owner, 2026-09-11: *"ensure all 3 entrypoints block until the refresh is done … I dont mean with block inline in vcpu, all three have semaphores/wait bits the guest is spinning on … those only fire if the refresh thats off vcpu completes."* The guest's own wait primitive blocks; the off-vCPU refresh is the only thing that releases it. Each point already has one: the invalidate's trigger register, the message queue's reply, and the pushbuffer's own semaphore. ⚠ Every lane therefore needs a path that releases *on refusal* too, or a refused refresh parks the guest forever.

⊘ The three points are NOT sufficient, and the gap was found in NVIDIA's source rather than in a census

[measured] by source audit of `ogkm-580.159.04`, 2026-09-11 (`docs/design/uncovered_page_table_writes_w452.md`). ★ The useful result is a *predicate* rather than a list, so it cannot go stale the way an enumeration does: routing to us is generic, so **the complete set of controls we can ever receive is exactly those whose NVOC export flags carry** `RMCTRL_FLAGS_ROUTE_TO_PHYSICAL` (`0x40`). `GPU_PROMOTE_CTX` has it; **every uncovered write below lacks it.** The sharpest of the uncovered set: a VA-space *destruct* releases page-table levels with no invalidate anywhere in it; `VASPACE_RELEASE_ENTRIES` is a downgrade with no invalidate, freeing PDE memory for reuse while our host mappings stand; `DMA_GET_PDE_INFO` is a *read* control that materialises page tables; a PDB swap invalidates *before* the write and never after, and its hardware-commit callback is a no-op on a GSP client; and page tables in sysmem are written as plain guest-RAM stores that emit nothing at any level.

⚠ **And the audit's second finding points the opposite way from how it reads.** The BAR2 bootstrap writes uninitialised PTE bytes that arrive as PRAMIN writes we trap. **A sweep triggered on the raw page-table byte write would decode framebuffer residue as PTEs and make mappings real on the host that the guest never authored** — manufacturing an illegal mapping out of a benign guest transient. ⇒ **Publish on the invalidate or the doorbell. Never on the raw page-table byte write.**

★ **The generalising rule the audit produced, and it retired one of its own rows.** Sparse ranges were first listed as an uncovered gap and then **withdrawn**: a sparse PTE maps *nothing*, so there is no backing to make real and no work to miss. What matters is the mirror image — an unmap that *sparsifies instead of unmapping* is a **revocation**, and a host mapping outliving it is a real defect. ⇒ *What we must observe is not "a page-table entry changed" but "real backing appeared or disappeared."*

What is built, what is only counted, and what is not enforced at all

mechanism	state	evidence
Doorbell deferred to a worker	**BUILT, default on**	`DoorbellAsyncArm` maps an unset variable to `On`; the trap validates, enqueues and returns, and a publication thread runs the identical body. ★ **There is no inline fallback**: a full lane counts itself and arms a sweep rather than running on the vCPU.
GSP command queue off the vCPU	**BUILT, no inline fallback**	`NV_PGSP_QUEUE_HEAD` writes record and return; a worker services the RPCs and raises the interrupt. [measured] `w447`, $n = 2$: **561** RPCs serviced off the vCPU.
BAR1 / BAR2 untrapped	**BUILT, default on**	Both QEMU props default `true`. ⊘ **The mechanism is indirect and worth knowing**: the regions are still constructed as pure MMIO, and the untrapping is a demand-driven first-touch memslot mirror placed over the range; anything that still reaches the trap is counted as a named miss. [measured] `w393`: BAR1 traps **88 193** → **91**.
Host RM verb off the vCPU	⊘ **NOT DONE — counted, not prevented**	`SharedDevice::verb_op`'s execute phase still blocks the calling thread on an isolate worker. `OffTrap::at_a_host_verb` takes *"the honest branch"*: a claim on the worker, or a **counted** inline exception on a vCPU. It cannot panic in production. [measured] `w449`: `inline_exceptions=24`.
The 1 ms trap budget	⊘ **MEASURED, NEVER ENFORCED**	`SLOW_TRAP_US = 1_000` feeds counters in the outermost trap guard (`SLOW_TRAPS`, `WORST_TRAP_US`, `WORST_TRAP_SITE`, and a per-site census). **No abort, no panic, no cap.** A budget that only counts is a metric, not an invariant. ★ **2026-09-11: the invalidate lane's own constant was re-derived and renamed** from `INVALIDATE_HOLD_BUDGET_US = 40_000` to `INVALIDATE_HOLD_CEILING_US = 1_000` — *"a budget says this much is fine, a ceiling says past this is a bug"*.

★★★★★ **Moving the work off the vCPU did not move the WAIT, and the metric built to watch it stopped measuring its own purpose.** [measured] `w447`, $n = 2$: with the GSP queue deferred and `gsp_off_vcpu = 561`, `worst_trap = 1 779 094 µs at bar0+0x110c00` — which is `NV_PGSP_QUEUE_HEAD(0)`, *the exact register that had just been deferred* — with `slow_traps(>1000µs) = 194`. The cause was rule 2: the vCPU's queue-head write takes the register plane's state lock, and the worker took *the same* lock and held it across the whole drain. **The work moved into the worker and the wait moved into the lock.**

⊘ **Two fixes, neither of which moved it.** Bounding the hold to one command per acquire (`w448`) gave `1 884 058 µs`, `slow_traps` 194 → 178. Giving the shared queue its own small mutex instead of riding the big state lock (`w457`) is landed and unmeasured. [measured] `w449`: **no single command even reached 1 ms**, so the 1.8 s is a wait, not a computation.

★★★ **And a stack sampler then moved the question rather than answering it.** [measured] `w460`, eight samples across 16 s of steady-state client running: the publication worker is parked in its queue's blocking take *every time*, and **no vCPU thread is anywhere in the MMIO write path.** ⇒ `worst_trap = 1.87 s` **is not a steady-state property; it**

does not happen while the workload runs. Where it *does* happen is unknown; the live hypothesis is GSP bring-up, once, before the client starts. ⊘ **A by-decade histogram was built for exactly this question** (`w459`) **and no boot has run with it, so any bucket figure quoted today would be invented.**

⚠ **The instrument lesson is the transferable part, and it bit twice.** First, `inline_exceptions` fell from 166 while the thing it proxied for did not move at all — *a metric that stopped measuring its purpose*, findable only because `worst_trap` carries its *site*. Second, the figure it fell *to* was wrong: the ledger's `grep -o 'inline_exceptions=[0-9]*'` piped to `tail -1` was matching the harness's own goal string (`target: inline_exceptions=0`). **The real value is 24, not 0**, and the `0` was reported to the owner twice and still appears in the milestone document's own table.

★★★★★ ⊘⊘⊘ **UPDATED LATER THE SAME DAY, AND THE ANSWER IS THE WORST POSSIBLE ONE: THE SECOND-WORST TRAP WAS INTRODUCED BY THE COMMIT THAT MOVED THE WORK OFF THE vCPU.** A *per-site* slow-trap census replaced the sampler and named both offenders outright: `bar0+0x110c00` (**12** slow traps) and `bar0+0xb830b0` — **the TLB invalidate trigger** — with **48 slow traps, worst 54 053 µs** [measured] `w462`. **The invalidate arm's own path was clean**: it touches only a small dedicated lock whose doc says *"the guest polls this register in a tight loop, so answering it must never take the FSM's lock"*, and it returns before any big lock. ⊘ **Then the GSP deferral put one back after it, on every MMIO write**, by reading a pending-count through the very lock the worker holds for whole RPCs. ⇒ *Moving work off a thread and then reading its state through a lock that thread holds puts the wait straight back*, which is rule 2 stated as a consequence rather than a principle. The fix mirrors the count into a lock-free atomic; ⊘ **it is not yet measured**, and the falsifier is `bar0+0xb830b0` leaving the slow-site list entirely. ⚠ And `RegPlane::write` still takes that lock **five times** on the vCPU path.

★★ **Note which instrument found it, because the paper's own doctrine is on trial here.** It was invisible to `inline_exceptions` (nothing is inline), invisible to `worst_trap` alone (the 1.86 s at the other register dominated it), and invisible to the stack sampler, which failed **six times** — five of them harness defects and the sixth a timeout too short for symbol loading on a 100 MB binary — before being abandoned. **A per-site count with 48 hits on one register is what made it findable.** ⇒ *The cheaper instrument answered the question outright, and was reached last.*

The threat model this narrows, and it is narrower than the paper's

> **Owner, 2026-09-11:** *"thats only safe with locks in a honest guest, still remember that a malicious guest can write data underneath so for them protect a breakout/illegal action not a self corruption of the guest."*

Reading page-table memory only inside a barrier's handler is safe **for an honest guest**, because that is the one window in which the guest is holding its own locks. A malicious guest can write page-table memory underneath us and there is no window that prevents it. **The obligation is therefore asymmetric, and §6.1 does not state it this way:** validate that what we read cannot make us touch memory we do not own or act on behalf of another `Proc`; **do not try to make a lying guest's own mappings coherent.** A guest that corrupts its own page tables gets its own work faulted, which is self-harm and not our problem — and it gains nothing by it, because an unpublished mapping means *our* host address space lacks the row and *the guest's own* work faults.

⇒ **The genuinely security-relevant boundary is object lifetime and our retire path, which we mediate directly. It is not the TLB.** That is a real narrowing of I1–I4's scope and it should be read alongside them, not instead of them.

Two standing directives that have no design-doc home in this tree

Stated here because they constrain every design above and a reader will not find them in `docs/design/`. (**a**) **A VMM address may never be guest-chosen or guest-visible**, and may appear only inside the two audited `unsafe` crates, bounds-checked (owner, 2026-09-10, strict). **GPU VA identity with the guest is separately *required*** — the guest's VA is the key, and it is the one address that must match. (**b**) **Avoid blocking calls outside the isolate workers**, whose blocking is justified because NVIDIA's ioctls are blocking and most cannot be polled; elsewhere prefer a resumable call and a multiplexed wait, because *"each blocking call is an unusable thread"* (owner, 2026-09-10). ★ The refinement in (b) is the useful one: the question is not *how long did you sleep* but **could new work have woken you**, so *a wait must name what can wake it* — and a wait for one operation to complete and a multiplexed wait that includes it *look identical in a stack trace and behave oppositely*. [unverified] in this document: (a) is carried from the owner's ruling and is not recorded in any design document in this tree, which is itself a gap worth closing.

⊘ **What this model has NOT yet bought**

- **Synchronisation point 3 has never executed.** `MEMOP-CENSUS seen=0` on both milestone runs, *while* `P2 uvm-memop` *verified*. A green P2 is not evidence for it. Whether those channels emit anything we can observe is an open source question.
- **The sweep is still there.** The barrier plan's step 5 is to delete the pin-rate sampling path, most of the publication-arm machinery and the convergence logic *once* the three points hold. None of that deletion has happened, so the tree currently carries both designs.
- **The LLM has not been run under any of it.** The milestone that established the three constraints says so in its own words: *"The LLM. Untouched since this landed."*
- **Rule 3 is not enforced anywhere**, and rule 1's exception for host verbs is counted rather than closed.
- **The 1.86 s hold at the GSP queue head is still unexplained**, and the atomic-mirror fix for the 54 ms invalidate hold beside it is landed and **unmeasured**.

4.5 The isolation model

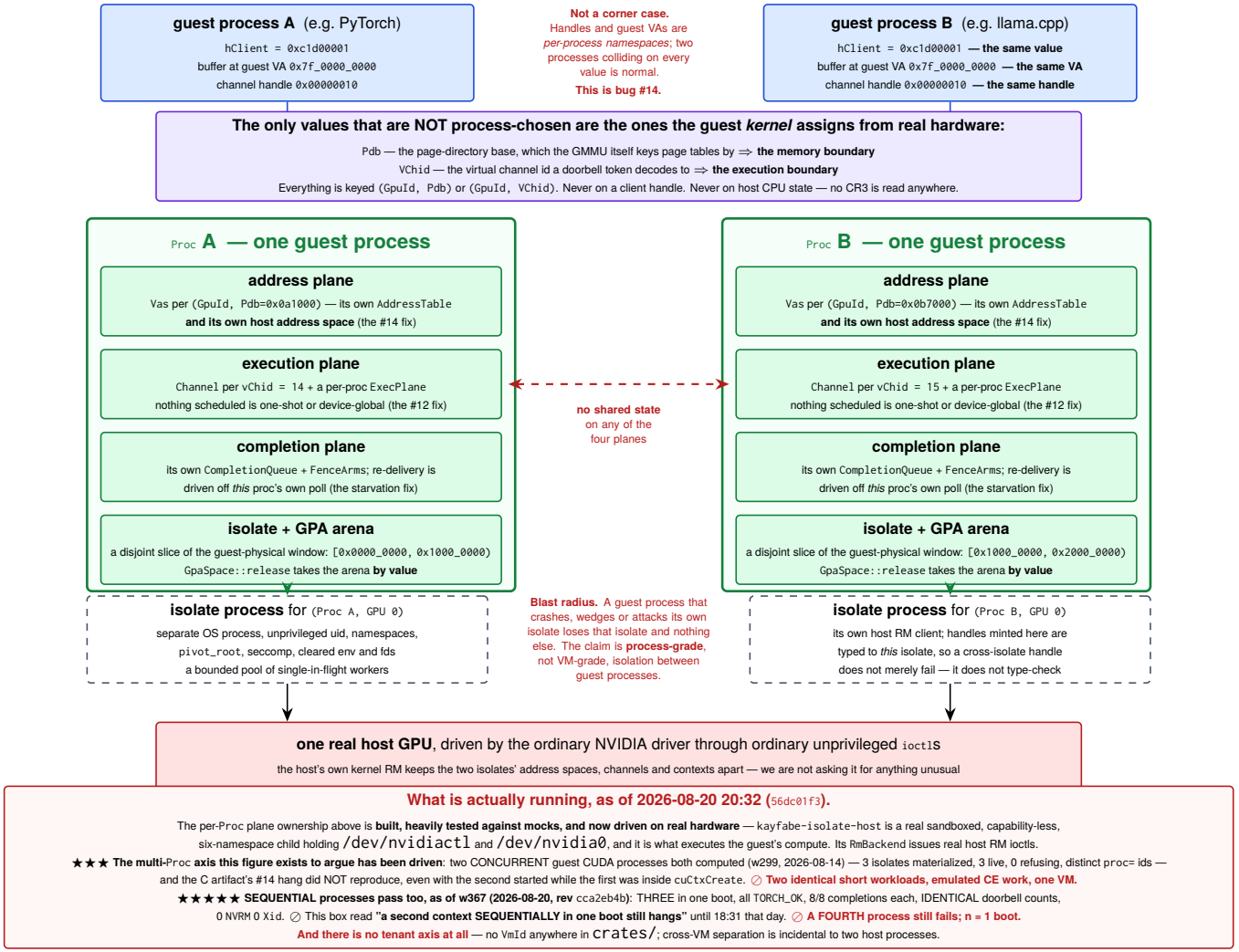


Figure 4: The process and isolation model, and the problem it exists to solve. ★ **Two concurrent guest CUDA processes have now been driven through this model on real hardware**, each with its own isolate, arena and address space, both computing correctly — so the founding #14 hazard did not reproduce. *One VM, two identical short workloads.* ★★★★★ **And as of 2026-08-20** (`w367`, §8.11) **so have three *sequential* ones**, in one boot, with identical doorbell counts and 8/8 completions each — the model's own cross-process refusal held throughout (a live peer's framebuffer backing was refused *by name* 132 times), and what was reclaimed was a join no live or dead process named. ⊘ **A fourth process still fails**; $n = 1$ boot. The per-`Proc` ownership of all four planes is built and heavily tested against mocks. ★ **The sandboxed isolate process at the bottom is now real** — capability-less, six fresh namespaces, holding `/dev/nvidiactl` and `/dev/nvidia0`, spawned in response to the guest's own action. ⊘ **What remains unexercised is the axis *above* `Proc`**: there is no tenant identity anywhere in the system, no two-VM run has been attempted, and the serialisation that made the two-process run safe is bounded today only by the hypervisor's big lock — a bound that weakens exactly when the forwarded work gets longer.

The rule that generates the whole model is one sentence: **a guest process is a grouping label, and it is never what an address or execution operation keys on.** Address operations key on `(GpuId, Pdb)`; execution operations key on `(GpuId, VChid)`. `Proc` exists only to own the isolate, the GPA arena and the lifecycle.

`Proc` membership is itself a **pure projection** of the RM graph — the dup-connected components of declared user clients — never inferred from timing, never from arrival order. That matters because it means two identical event streams delivered in different orders produce bit-identical process boundaries, which is a property the test suite checks directly by permuting streams.

4.6 The lifecycle

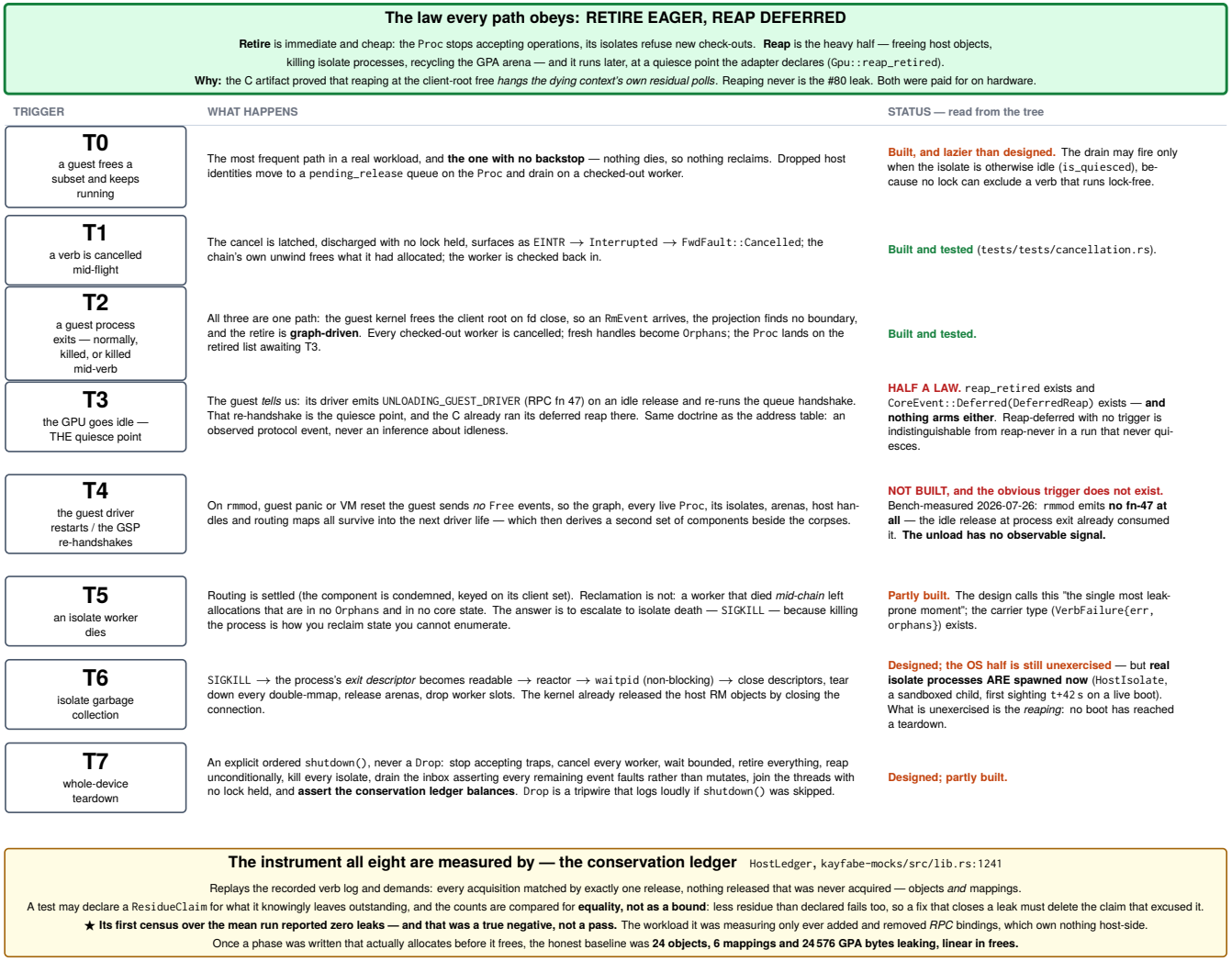


Figure 5: The eight teardown triggers and the retire/reap split. Two of the eight are honestly broken and say so: T3's quiesce trigger is designed but nothing arms it, and T4's obvious trigger was measured on hardware and does not fire.

The lifecycle is where the C artifact's scar tissue is thickest, and it is worth understanding why the split exists. Reaping resources at the moment the guest frees its client root *hangs the dying context's own residual polls* — bench-proven. Not reaping at all exhausts the shared guest-physical window after a handful of process generations — also bench-proven, as bug #80, and *re-introduced and re-found in the Rust core* by the regression test written for it. Hence: retire eagerly (refuse new operations, cheap), reap later at a quiesce point the adapter declares.

Figure 5 marks two of the eight triggers red, and the reasons are different in an instructive way. **T3** is a design that is not wired: `reap_retired` exists, `CoreEvent::Deferred(DeferredReap)` exists, and nothing arms either. **T4** is worse — the trigger the design chose *was measured on hardware and does not fire*. The design assumed `rmmod` emits RPC function 47; the bench showed it emits none, because the idle release at process exit already consumed it. The problem is not that two triggers are indistinguishable; it is that the driver unload has *no* observable signal.

5 The crates

Sizes are `crates/<name>/src/**/*.rs`, measured at `5c367a38`. "Maturity" is this document's reading of the tree. ⚠ **It is no longer cross-checked against** `docs/design/crate_maturity_map.md`**, because that document is now wholly stale** and this paper will not launder a number through it: it is pinned at `d65eb75` ("496 tests green"), lists **16** crates where the tree has 24, calls `kayfabe-gsp` a **34-line skeleton** where it is 5 056 lines, and records defect #57 as open five days after it was closed. It is a worked instance of §8.21 and is listed there (§10 row 29).

★ **The shape of the growth inverted at this revision, and that is the most informative thing in this section.** The previous pin added *six new crates* and grew 2.7×. This one added **zero** — the member list is byte-identical at 24 — and still grew **1.6**×, from 120 979 to 198 122 lines. *All of it went into crates that already existed*, and almost all of it into four: `kayfabe-qemu-raw` 3 435 → 18 720 (5.5×), `kayfabe-isolate-host` 9 939 → 21 557, `kayfabe-device` 9 832 → 19 595, `kayfabe-abi` 28 630 → 42 511. ⇒ **The architecture stopped being extended and started being filled in** — which is what the interval between "a driver binds" and "a guest computes" actually cost, and it is concentrated entirely in the three *adapter* crates plus the wire-format crate, i.e. exactly the layers the hexagonal design predicted would absorb the contact with reality. *The pure logic core grew least in proportion*: `kayfabe-core` 10 528 → 14 617, `kayfabe-gsp` 5 056 → 5 497 (+8.7%). ⊘ That is a favourable reading of a number, so state the unfavourable one beside it: the growth landed in the crates with the *least* test coverage and the *only* `unsafe`.

5.1 `kayfabe-core` — 14 617 lines, **BUILT** and mutation-hardened

Owns the spine. `RmGraph` is the source of truth: a refcounted resource/handle split with `DUP_OBJECT` aliasing, order-tolerant "parked" facts (a mapping that arrives before its target is held, not rejected), mapping refcounts and capacity bounds. `project()` is a pure function from the graph to process boundaries and the two routing tables. `Gpu::apply` is transactional — snapshot, mutate, re-project, roll back on any derivation fault, so a hostile event earns only its own refusal. `gpa.rs` owns the guest-physical window and carves per-process arenas; `GpaSpace::release` takes the arena *by value*, which makes releasing a live process's arena unrepresentable rather than merely wrong.

Deliberately not here: anything about wire formats, any NVIDIA constant, any OS call.

The sharpest single defect the project has found in its own core is still the one described in §8.8 — a live-memory-free in `RmGraph::apply` whose independent oracle carried the identical bug. Nothing since has displaced it, and the parked-fact machinery that produced it is still where the difficulty in this design lives.

5.2 `kayfabe-abi` — 42 511 lines, **BUILT** — now the largest crate in the tree

Axis A: the per-driver-version wire layouts. Contains an offline generator (`crates/kayfabe-abi/gen`, a separate cargo project with zero third-party dependencies) that parses NVIDIA's own FINN-emitted C headers from the open kernel modules and emits Rust; 13 generated structs plus one hand-transcribed variant; a version-dispatch table keyed on full `major.minor.patch`; and a large oracle suite. The `#[repr(C)]` mirrors are a *layout oracle only* — decode is `from_le_bytes` out of a byte slice, never a cast, so `forbid(unsafe_code)` still holds.

Its test design is the best in the repository: layouts are pinned against **three independent oracles** — NVIDIA's own headers, gVisor's independent transcription, and the C artifact — with a `file:line` citation per number, and a test whose name is `nvos46_the_three_oracles_disagree_and_here_is_exactly_how`. That test found that the C artifact's *parity test* was weaker than the C code it was supposed to guard.

It tripled in size and acquired the two mechanisms this revision leans on hardest. **First,** `capability.rs` — the guest-ingress allowlist ported from the C's nvproxy-derived table: 162 control rows and 91 allocation classes at

580, each carrying NVIDIA's own name and an `Origin`, six controls and two classes refused *by name*, resolved denial-first (which is stricter than nvproxy). ★★★ **And "default-deny" is true of half the command space, not all of it**, which the module says in its own header: the GSS-legacy rule passes every command with bit 15 set — 2^{31} values, no table row, no review — because that is nvproxy's rule and the C's, and this is a port. A test named `the_gss_legacy_rule_passes_half_the_command_space` pins it. ⊘ It also does *not* carry the host-side security property: all six denied controls are `NON_PRIVILEGED` in RM, and deleting the table tomorrow would leave that property standing (§8.18).

Second, `oracle.rs` **and** `fmbsize.rs` — the machinery that refuses an uncaptured C oracle row, which is §3.1's fifth limit turned into a predicate rather than a list.

It now has six production consumers, and it is still the crate that keeps every NVIDIA constant quarantined: the bridge names none, by design and by review. *The previous revision's surviving criticism is now expired too*: it said the version-dispatch machinery was "a built capability with a test-only interrogator" because `gsp_element_wire()` had no caller in `crates/*/src`. It has one now — the shipped register plane selects a wire layout to answer a real guest through.

5.3 `kayfabe-linux-raw` — 14 208 lines, **BUILT** — the *first* of two `unsafe` crates

One of exactly two crates permitted `unsafe`, and one of the two that do not inherit the workspace lints (because `forbid` cannot be relaxed by an inner `allow`). It restates `unsafe_code = "allow"` plus `clippy::undocumented_unsafe_blocks = "deny"`. Every file that uses the keyword is named `*_unsafe.rs`, so `ls` enumerates the audit surface: 12 files, **87 relaxations**. It also carries a `trybuild` compile-fail matrix covering things like a bare `MAP_FIXED` address, a page-size literal, and a view outliving its region.

★ **The ratchet moved from a constant to a named two-element list, exactly as §8.19 predicted it would have to** — `AUDITED="kayfabe-linux-raw:91 kayfabe-qemu-raw:49"` — and the acceptance criterion the gate protects is now "read two crates". The movement is itemised per step in the workflow with an argument each: 7 → 25 (first consumers) → 37 (a reactor and a guest) → 47 (a host GPU and a child process) → 50 (a machine it did not create) → 57 (**closing a measured host-filesystem escape**) → 60 (closing the privilege half) → 73 (the embedded isolate) → 87.

> ★★★ **And the ratchet's own counting pattern was wrong, found by being its first FFI consumer.** `unsafe ({|fn |impl |trait )` **cannot match** `unsafe extern "C" fn` — the dominant form in a foreign-function crate. **It counted 23 of 31 and called it a complete audit.** The fix was not to bump the bar: `kayfabe-linux-raw`'s count was *re-measured* under the corrected pattern rather than carried forward, because *"it was 57 when I looked" would have been a claim about a tree that no longer exists.* ★ A second rule came out of the same review: **thirteen relaxations are the surface, not a style** — a macro emitting the call would collapse all thirteen into one textual relaxation, because the ratchet counts source tokens and a macro body is one token. *That is gaming the instrument.*

5.4 `kayfabe-gsp` — 5 497 lines, **BUILT** (S0–S5) — **and a real guest driver now writes into it**

The faked GSP: the guest driver's entire picture of its GPU arrives through this crate. Eight modules. `ram` owns the guest-RAM port and walks the shared region's *self-describing page table* (the region is allocated `NV_MEMORY_NONCONTIGUOUS`, so the C artifact's `base + offset` arithmetic is a latent bug this crate does not reproduce). `ring` is NVIDIA's `msgq` algebra — geometry, cursors, and the `msgqRxLink` acceptance predicate reproduced code-for-code so "would the guest link this header?" is answerable offline. `element` is framing, the 64-bit XOR-fold checksum and multi-element split/join. `boot` is `BootPhase` × `QueueState` with transitions E1–E11, and it deletes all three of the C's one-shot latches — two by construction, one by making the queue GPA exist only inside a `Bound` variant. `rpc` classifies functions into three dispositions (must-reply-synchronously, reply-expected, must-*not*-reply). `replay` is the differential projection its oracle compares, plus a ledger in which a deliberate divergence is admissible only with four things attached, one of them an independent oracle.

Five things it is deliberately not tied to, each with a test that pushes on it: a GPU architecture (every offset is behind `kayfabe_arch::GspModel`; the suite drives the same FSM through two deliberately-different models), a driver version (the element layout is an `ElementLayout` *value*, not a constant), a hypervisor (the only way in is an abstract register access and the `GuestRam` port; CI greps for every VMM type name), a single GPU lifetime (`GspFsm::device_reset` rebuilds the value and teardown drops the queue binding *by value*, so driver-restart works in-process, repeatedly), and a CPU architecture (every wire access is explicitly little-endian).

What changed underneath it at this revision is that it stopped being a model. It has **six production consumers** (`chips`, `device`, `qemu-raw`, `completion`, `crec`, `rmrpc`), and — the part that is not a manifest fact — a stock NVIDIA 580.159.04 kernel module in a KVM guest now writes into this FSM through a real BAR. Its boot phases are driven by a driver that does not know it is being driven. **GSP-S1**, the guest-kernel memory-safety invariant an earlier

revision of this paper singled out as "the most security-relevant thing in this document that is a sentence rather than a mechanism", remains a mechanism and is now one a real guest is on the other side of. §8.10 is about what remains.

5.5 `kayfabe-rmrpc` — 6 098 lines, **BUILT** — the bridge, and no longer an island

Named by `crates/kayfabe-qemu-raw/Cargo.toml`, which is what retires the previous revision's headline about it. Its whole entry point is still one signature:

```
translate(&DriverAbiTable, &RpcCommand) -> Result<Translation, BridgeRefusal>
```

and it is a **free function** — no `&mut self`, no handle table, no memo, no seen-set, no guest-memory access and no lookup. That statelessness is the design's central claim rather than a tidiness preference: a per-handle cache here would re-open the whole recycled-handle bug class the core closed the same week, so *recycle-safety is structural rather than tested*. The suite pins it by mutation — a per-handle memo and a seen-set are planted as deliberate mutants at every stage, and by B3 they were visible to 20 and 24 tests respectively.

`policy.rs` holds the only thing in the crate that applies: `GraphPolicy`, the `kayfabe_gsp::CommandPolicy` the boot FSM calls, whose `respond` is translate → `Gpu::apply` → reply. A refusal answers with a non-zero status and an **empty** body — deliberately not the request echoed, because reflecting guest bytes under a failing status is `memcpy(resp, cmd, 4096)`, the C defect this deviates from by name. Refusals are counted by a `RefusalCensus` bounded by the finite tag set rather than by traffic, with `applied()` and `inert()` kept as *two* counters so a regression that turned every alloc inert cannot hide inside a sum.

What it deliberately is not: it is not in `kayfabe-fwd`, where both the port plan and `rpc.rs`'s own comment said the bridge would go — `fwd` depends on neither `abi` nor `gsp`, and B1 needed no `Worker`, `Vmm` or `RmBackend` at all. Three superseded placements are recorded in the design doc rather than quietly dropped.

Its honest limits, stated as the crate states them: it **still cannot emit a** `Translation::Forward`, because the control-classification table it would hand to does not exist, so the entire control long tail is refused by name; and it cannot reach the compute path at all (§8.7). ★ That second limit is quietly load-bearing for security: three descriptor-passing controls that a multi-tenant audit says should be denied are still on the allowlist, and the only thing stopping them mattering is that **the forward arm is unwritten** — *which is a schedule, not a control* (§8.18).

5.6 `kayfabe-rt` — 10 079 lines, **BUILT** (L1-M1)

The threaded shell. `LockRank{Device=0, Proc=1, Leaf=2}` with an always-on (not `debug_assert`) rank check that panics *before* the OS acquire, so an inversion is deterministic. `SharedDevice` is the plan/execute/commit driver, and it ships **both** lock modes — `Sharded` and `Degenerate` — from day one, tested as first-class configurations with a differential asserting a bit-identical end state, specifically so a late granularity change is never the untested mode.

5.7 `kayfabe-fwd` — 8 509 lines, **BUILT** — and **part of it has now moved bytes on silicon**

The entry points an adapter calls: `handle_doorbell`, `publish_backing`, `parse_pushbuffer` (the one parser: CE page-table-write capture, semaphore release, TLB invalidate, everything else opaque and passed through), `forward_engine_object` and `route_control` (the Case-1/Case-2 split), the fence arms, and the present route.

Its epistemic status changed when the copy plane was joined to a real engine. `plan_ce` / `commit_ce` / `exec_ce` / `submit_ring` are the three R1 phases for a partitioned copy request plus their composition. Before that, `PushbufferOutcome::ce_spans` — the partitioned request the parser had produced since #102 — **had no caller anywhere in the workspace**: every copy the core recovered from a guest ring was computed, asserted about, and dropped. `commit_ce` adopts nothing and is **never retryable**, because re-running a copy that executed performs it twice.

The honest scope. A real host copy engine has executed a request this crate decoded and planned (§8.2). **That is one path.** The doorbell path has been driven to `UnknownVchid` on a boot and to `IsolateRetired` in-process, and never further. The control long tail, the fence arms and the present route remain claims about NVIDIA's behaviour that nothing here has checked against NVIDIA.

5.8 `kayfabe-mocks` — 4 609 lines, **BUILT**, test-only — **and where an invariant hid**

One deterministic fake per port, plus the shared verb recorder and the conservation ledger. Excluded from the mutation denominator on the argument that mutating an instrument is a category error. *Strictly*, the crate is not confined to dev-dependencies: `tests/` and `fuzz/` list it as a normal dependency, so `cargo build --workspace` compiles it. Nothing shipping links it.

★★★ **And this crate is where invariant R1 hid for an unknown number of commits.** The assert that guards R1 at an isolate spawn lives in `kayfabe-linux-raw`, at the syscall — so it is reachable *only on the host plane*. The whole suite spawns through `MockIsolateFactory`, whose `spawn` blocks on nothing and asserts nothing. **1 975 green tests ran over a live R1 breach**, and only a real boot on a real GPU found it (§8.17). The lesson, stated generally in the design record: *an invariant asserted only in the production adapter is an invariant the test suite does not hold. Where the mock and the real implementation differ in* ***whether they can violate a rule****, the assert belongs at the seam they share.* It was moved there — `IsolateBox::new` now asserts, which turns the entire mock-driven suite into a live guard for it.

5.9 `kayfabe-vmm-kvm` — 2 724 lines, BUILT — defect #57 CLOSED

The real KVM memory plane: real `/dev/kvm` file descriptors, real memslots, real vCPU run loops, MMIO exit classification. **Defect #57 — which this paper twice printed as the reason the QEMU adapter had not been started — was closed at** `020790c`, and the shape of its resolution is the part worth keeping: *the violation was not the two-lock split, the lock order, or a syscall inside a critical section. It was* `Arc` *ownership* — a reader's clone became the last one and `munmap` ran on a thread legally holding rank 0. **R1 is about blocking calls, and a** `Drop` **is a call site.**

5.10 `kayfabe-isolate` — 4 246 lines, BUILT traits — with real implementations now

The port for the unprivileged host worker: `RmBackend` (12 verbs, all of which must be issuable unprivileged), `Isolate` (two-stage retire, cancellation, quiesce predicate), `IsolateFactory`. A `HostHandle` carries its `IsolateId`, so a cross-isolate handle does not merely fail — it does not type-check. *The previous revision's headline for this crate — "no real implementation" — is retired by* `kayfabe-isolate-host` *below.* And `Worker::execute` is no longer "the single door": `IsolateBox::new` and `IsolateBox::drop` are now R1 doors too, closing birth and death of an isolate respectively.

5.11 `kayfabe-isolate-host` — 21 557 lines, BUILT — the real unprivileged host worker

New, and it is what makes half of this paper's hardware claims possible. A sandboxed child process: capability-less, six fresh namespaces, holding `/dev/nvidiactl` and `/dev/nvidia0`, issuing real host RM ioctls. It carries three binaries — `kayfabe-isolate` itself, `kayfabe-rm-ladder` (a human-facing bring-up ladder, deliberately *not* part of the isolate, because the isolate faces a hostile guest), and `kayfabe-sandbox-probe` (a separate *process* by necessity: entering the sandbox rewrites this process's mount namespace and root, so the containment property can only be observed from outside). `CeEvidence::copied()` — the predicate the whole execution plane is accepted against — lives here.

★ **Building it closed a measured host-filesystem escape**, which is why the unsafe ratchet moved 50 → 57: the isolate's own `/dev` descriptor could open `../etc/shadow` on a real host. That is a defect the C artifact had found, fixed and documented, and the Rust rewrite reproduced before re-fixing it — a worked example of *port the C and subtract its named bugs* failing on a bug that was named.

5.12 `kayfabe-trace` — 1 497 lines, BUILT vocabulary; still not threaded

The typed `TraceEvent` vocabulary, a `TraceSink` port, one-counter total ordering, perf-budget counters and a projection differential. It exists because the GSP milestone's oracle *is* trace replay, so the replay format had to be defined before the harness that consumes it. Two findings from building it are worth carrying: it had to sit *below* `kayfabe-core` (every plane must be able to emit), so it cannot name `RmEvent` or `ProcId` — the bridge is a trait the owning crate implements with an exhaustive `match`, so a new variant upstream fails the build until the vocabulary names it. And no `&mut Trace` is threaded through any plane signature yet; the vocabulary is driven only from the conformance suite's seam observer.

5.13 `kayfabe-shell` — 995 lines, BUILT (L1-M2, in progress)

The reactor: an epoll loop, source resolution, the inbox, and the executor's wake. It carries the restated F1 rule — *assert a bound, never an absence* — as a concrete quantity: after 16 consecutive unproductive wakes it refuses with `ReactorFault::UndrainableSource` rather than spinning. That restatement came from the mutation gate: three spin-versus-park mutants survived precisely because "never busy-poll" is an absence, and an absence is not observable.

5.14 `kayfabe-vmm` — 943 lines, BUILT traits — and the agnosticism claim is now settleable

The hypervisor and display ports: `Vmm` (7 capability groups), `Device`, `Present`. An eighth group — the memory-lock primitive — was *removed* from the trait when it turned out to need no hypervisor cooperation on any

backend, which is a good sign about the seam's discipline. `Vmm` **now has two real adapters** (`KvmVmm`, `QemuVmm`) with a differential test asserting they produce identical operation logs for the same core run, which is the experiment the previous revision said only a second backend could run. [unverified] here: this paper did not execute it. `Device` **remains the one port on the layer diagram whose only implementors are test types** — the shipped QEMU path enters through `GuestRam` and the register plane instead, so the trait was routed around rather than implemented, which is worth noticing before treating it as a live seam.

5.15 `kayfabe-device` — 19 595 lines, **BUILT** (L2 stage Q4) — the chip table and register plane

New. Two things and nothing else: `ChipProfile`, one row per GPU generation carrying the PCI identity, the `GspModel` constructor, the silicon-constant registers and the VBIOS aperture; and `RegPlane`, the routing of one trapped register access into `GspFsm` and back out as a value. *It exists so there is exactly one description of a chip*: before it, the only real register map lived in the trace-differential harness, which a shipped archive cannot depend on, so wiring a guest's accesses meant either shipping the mocks or writing the offsets a second time — *two descriptions of one chip that can disagree*. The consequence is that the 359 062-record `cap1` replay now runs against **the same encoder the guest reads through**. Adding a generation costs a module, a VBIOS row and one appended table entry, and `tests/chip_table.rs` proves that by *doing* it with a chip whose register map is deliberately not GA10x.

5.16 `kayfabe-chips` — 3 202 lines, **BUILT** (L3) — **the adapter contract, finally measured**

New, and it exists to take one measurement. The port plan had named the home of an `Arch` implementation as "an arch-impl crate" for months; **no such crate existed**, so *"adding a generation is* `impl Arch for <Gen>` *in an adapter crate"* had never been done by anyone. It holds two generations that answer in **opposite directions**, reported separately because averaging them would hide the interesting one. **Ada** costs a struct and a `VBIOS_PROFILES` row and nothing else — ★ and that result *"is weak on its own and is labelled as such: an experiment that selects the easiest member of its universe produces a green with no red available to it."* **Hopper** is the hard member, and it was never the offsets: the boot ordering was spelled out in `match` arms inside a *logic* crate, so a generation whose boot is driven by an FSP command queue could only be expressed by editing the ordering Ampere boots on. The ordering now lives behind `kayfabe_arch::BootSequence`, and landing a second one changed **zero lines** of `kayfabe-gsp`, `kayfabe-device` or the Ada module. *Arch-specific boot code is legitimate and has to exist somewhere; the defect was that it was unhooked.*

5.17 `kayfabe-vmm-qemu` — 5 852 lines, **BUILT** (L2) — the QEMU memory plane

New. The second `Vmm` adapter: slot management, region classification, viewer installation, tiering. Under `forbid(unsafe_code)` — all the `unsafe` is quarantined one crate over.

5.18 `kayfabe-qemu-raw` — 18 720 lines, **BUILT** (L2) — **the composition root and the second** `unsafe` **crate**

New, and it is the widest row in the dependency matrix — it depends on eighteen of the twenty-two crates above it. A `staticlib` that a 2 676-line C QOM shim (`qemu/hw/misc/nvkvm/`) links into a stock QEMU 10.2.4 tree. It owns the FFI surface — `extern "C"` entry points, raw `MemoryRegion` handles, adoption of host pointers QEMU owns — at **49** audited `unsafe` relaxations, ratcheted separately from `kayfabe-linux-raw`. Its bar started at **0** deliberately, *"so that this had to be a reviewed commit rather than a manifest edit"*. `MachineRam` is the production `GuestRam`; `Regs::object_model()` is the single composition root that the object bridge declares into and the doorbell rings.

5.19 `kayfabe-crec` — 2 041 lines, **BUILT**, harness-only — the C-trace differential

New. Decodes the committed C captures and replays them against the real `GspFsm`. Two committed traces (`cap1`, `cap1b`) are stored *decompressed* so the harness needs no decoder and no third-party crate. §3.1 is its output.

5.20 `kayfabe-completion` — 756 lines, **BUILT**

Per-process queues (pending → in-flight → awaiting-ack → ack), a drain-gated delivery plane whose re-post is driven off *the owner's own poll* (the starvation fix), and mapped-fence arms with the #12 backwards-jump guard. Scored 100% on the mutation gate.

5.21 `kayfabe-util` — 1 796 lines, **BUILT**

`IntervalMap` (the address table's container, loud on overlap, empty and wrap), a *virtual* `Instant` so no wall clock exists anywhere in the core, the `assert_send_sync!` compile-time gate, and the lock witness that invariant R1

asserts against. The witness lives here rather than in `kayfabe-rt` specifically so that `kayfabe-isolate`, which cannot depend on `rt`, can still assert it.

5.22 `kayfabe-arch` — 2 942 lines, **BUILT** traits — **with three production implementations**

The identity vocabulary (`HClient`, `HObject`, `Pdb`, `VChid`, `ClassId`, `GpuVa`, `Gpa`, `GpuId`, `EngineKind`) and the Axis-B port set, which grew to include `GspModel`, `UserdModel`, `PushbufferAbi` and `BootSequence`. **The previous revision's standing line — "still zero production implementations ... that remains the standing proof that a real one will need no core edits" — has been settled by doing it.** There are three (`kayfabe-chips`), and the proof came out partly favourable and partly not: Ada needed no seam, Hopper needed one, and *the seam it needed was in this crate*.

5.23 `kayfabe-mmu` — 5 128 (**7 006** at `5f323dd5`) lines, table **BUILT**; **walker built**; **GPGA and refresh are DESIGN-ONLY**

`AddressTable` is complete and is the enforcement point for the project's most-cited directive: forward-populate only, unbind eagerly, and **a miss is a fault** — no reverse resolve, no heuristic pick, no MRU fallback exists anywhere in the codebase.

★ **The previous revision's headline for this crate is retired, and this paper named the gap that closed.** It read: *"the* `walker` *module is 41 lines containing one trait and one enum, with no walk function of any kind."* The module is now 863 lines with `translate`, `translate_from_entry`, `decode_page`, `decode_subtree`, `leaf_disposition` and `populate`, and `FbRead` has two implementors. It sits beside `gpga.rs` (1 391 lines), the guest-physical address space, and `reach.rs`, the reachability-on-transition machinery.

⊘⊘ **CORRECTED 2026-09-11, AND IT IS THE EXACT DEFECT THIS PAPER NAMES ELSEWHERE AS THE TREE'S FAILURE MODE.** The paragraph above lists `gpga.rs` inside a **BUILT** crate and says nothing about whether anything calls it. **It has zero production callers.** Its only non-test consumer is a `viewer_install` module that itself has zero production callers; the composition root depends on that crate and never names either. `refresh.rs` (583 lines) is the same: **no caller outside its own crate**, and its own doc says it is *"the **model** of that rule ... a pure fixpoint over mocked page tables."* ⇒ **Read both as DESIGN-ONLY.** `blockage.rs`, `reach.rs` and `walker.rs` do have production callers and are correctly described.

★ **The project's own assessment of this pattern is blunter than the paper's, and it is worth quoting against the paper.** *"Mechanism without consumers. Built, wired, unit-tested, never reached."* Counted in one night: a 634-line production backing path still named `measure_guest_ram_pin_rate` after the measurement it used to be; a hold that fires **0** times against **304** row-binding RPCs; a decoded guest barrier read by **no production code and four tests**; and **648 lines across two modules with zero production referrers**. ⊘ One of those two modules, `promotion.rs`, was **deleted** (2026-09-11, 332 lines); `dbtable.rs` (316 lines) survives, unwired. ⇒ **Every crate-size figure in this section counts lines, and a line count cannot see whether a boot can reach the code.** §4.4 states which mechanisms have fired on hardware and which have not.

`tests/` — the conformance suite

158 170 lines of test code against 198 122 lines of implementation. There are **106** files in `tests/tests/` and **96** more under `crates/*/tests/`, plus a `Scenario` DSL in `tests/src/`. The suite is a workspace member with a `[lib]`, which is why `kayfabe-mocks` compiles as a normal dependency. ★ **Note the ratio flipped**: at the previous pin test code exceeded implementation by 16% and this paper listed that as a risk; implementation has since overtaken it. That is not obviously an improvement — the growth is almost entirely in the L2/L3/host crates, which are the least tested part of the tree.

That ratio is deliberate and the reason is worth stating, because it is the crate's answer to an oracle bug found the same week. Every bridge message is transcribed **three independent ways**: an `ogkm`-derived builder in `tests/src/rpcwire.rs` that imports nothing from the decoder, hand-written offset-annotated hex arrays, and the decoders themselves — plus a differential against the C artifact's independently-transcribed offsets. The bug that motivated it was in `tests/src/gspworld.rs`: the independent guest model derived the element count *from the element under test* instead of reading the field, so the encoder's own write was unobserved by its own oracle. `250b78c` fixed the model; `gspworld.rs` is now 1 468 lines.

6 The invariant catalogue

These are the properties the design is willing to be judged on. Where an invariant has a mechanism, the mechanism is named; where it is only prose, that is stated, because the project's own doctrine is *"prefer a mechanism to a sentence — a rule stated only in prose has no reader."*

Invariant	Statement	Enforced by
Order-independence	Everything derived is a pure function of declared protocol facts, never of arrival order. "Protocol, not trace."	Permutation/interleave/dup proptests over the whole observable end state (`tests/tests/determinism.rs`).
MISS = FAULT	Tables are forward-populated only; a lookup that misses is a loud fault, never a guess. **Refined:** a ~28-site audit found the absolute had a second, correct answer — the real rule is decided *per site*: *not yet knowable ⇒ DEFER; never knowable ⇒ FAULT*. A mapping arriving before its PDB bind defers; a handle resolving to the wrong *kind* faults. Getting it wrong is asymmetric: fault-when-it-should-defer hangs the guest, defer-when-it-should-fault is a security question.	`AddressTable` has no reverse-resolve path to call. `tests/tests/miss_taxonomy.rs` makes the taxonomy executable. *No shipped site needed a behaviour change; three **doc** claims did, two of them stating the literal opposite of their own code.*
Per-target keying	`Pdb` and `VChid` are per-GPU namespaces. Cross-GPU identical ids are legal; same-target duplicates are loud collisions.	`project()`; `tests/tests/multi_gpu.rs`.
Per-`Proc` isolation (I1)	Identical guest VAs and handles in two processes reach disjoint arenas, disjoint host address spaces and disjoint isolates *by construction*.	Types (`GpaSpace::release` by value; `HostHandle` carries its `IsolateId`) plus a proptest against an injective oracle.
One gated ring path	A doorbell can only reach the host after the #14 working-set gate.	**Now structural, not a call-graph claim.** `VerbPlan::Doorbell` is `#[non_exhaustive]`, so its only constructor is `VerbPlan::gated_doorbell`, which runs the gate; an ungated one is compile error E0639, pinned by a `trybuild` row. The cardinality claim corrected in §10 is what this replaced.
GSP-S1	**We may never emit an element run wider than the guest's staging buffer.** At driver 580 the guest reads our `elemCount` field directly into an unbounded `portMemCopy` loop whose staging area is exactly `queueElementSizeMax / queueElementSizeMin` elements — 16 on the bench — with the live `msgq` metadata as the very next byte. An `elemCount` of 17 overwrites the metadata the guest is actively using; at the ring's reachable maximum it writes 188 416 bytes past a `portMemAllocNonPaged` allocation. This is a *guest kernel heap* corruption reachable from a value we choose.	★ **PAID OFF AT THIS REVISION** (`250b78c`), and it was the item the previous revision named as this project's highest-stakes prose-only rule. `max_elements` is derived from the layout, `encode_message` refuses `GspFault::ElementCountOutOfRange` before building a run, *unconditionally* rather than gated on the layout's field, and the receive side range-checks the guest's declared count and cross-checks it against the derivation (`ElementCountMismatch`), leaving the cursor unmoved on refusal. *With one honest footnote:* the originally briefed bite could not fire, because at a dividing element size the length bound already implies the count bound — the count bound is independently reachable only at a non-dividing `msgSize`, which is real guest-chosen input, and the test moved there.

Invariant	Statement	Enforced by
Completion integrity (I2)	A completion fires only from a genuinely-armed source in the *owning* process, at most once; the system forge path is typed so it cannot reach a user process's queue.	Type-level (`Traffic::System`) plus a differential against an independent reference fence model.
Refcount soundness (I3)	A resource is alive $\iff$ it has a live handle or map reference. No premature destroy, no leak, dup survives origin free.	Proptest over deep trees with cross-client dups and interior frees.
DoS containment (I4)	`Gpu::apply` is transactional; every guest-growable table is capacity-bounded with a loud refusal, never OOM.	**Corrected, and the correction is in the design doc:** "no $O(n^2)$ path on hostile floods" is **false and was measured**. The caps bound *memory*, not *time* — see §8.
R1	No blocking host verb — indeed no potentially-blocking syscall — may be issued while any ranked lock is held. Blocking calls take **zero** ranked locks.	★★★ **This is the invariant that was BROKEN on the production path, and the suite could not see it** (§8.17). `assert_lock_free` guards the backend doors, and a `Drop` is a call site — one violation was exactly an `Arc` ownership issue in a `Drop` (#57). But the spawn of an isolate is also a blocking call, it ran under rank 0, and **1 975 green tests could not reach the assert** because the suite spawns through a mock. The repair moved the assert to `IsolateBox::new` — the one door every plane shares — which turns the whole mock-driven suite into a live guard.
R3	Locks are acquired in strictly increasing rank, at most one per rank.	`check_acquire`, always-on, panics before the OS acquire.
R5	After re-acquiring, re-validate everything the plan assumed.	The four `commit_*` functions; bounded re-plan at `MAX_COMMIT_RETRIES = 8`.
F1	Three unrelated things are called F1 in this project: a fixed threat-model bug, the per-GPU collision guard, and the reactor's bounded-poll rule.	All three real; the overloading is a documentation hazard, not a code one.
GL1–GL13	The guest-memory-lock invariants.	**Documentation only. Zero occurrences in any `.rs` file.** They have no port to be enforced at, because capability group 8 was removed from the `Vmm` trait. Treat as a designed, unimplemented subsystem.

6.1 The two-layer trust model

One normative rule deserves its own space, because it decides designs and because it is the cleanest statement the project has produced:

> **Layer 1 (trap + lock) gives SECURITY against a guest that ignores the protocol. Layer 2 (NVIDIA's own synchronisation points) gives CORRECTNESS for a guest that follows it. Shared tables are NEVER authoritative for an immediate decision.** [docs/design/core_security_threat_model.md §2.1]

Neither layer is a weaker version of the other. Layer 2 alone is not security: a hostile guest can update a shared table *after* validation and *before* we act, and it can fake the handshake entirely. Layer 1 alone is not correctness: a trap says the bytes cannot change while you look; it does not say they are *finished*. A cooperating guest halfway through publishing a page-directory update has written something internally inconsistent and perfectly stable, and reading it under a perfect lock yields a perfectly-locked wrong answer.

The consequence — *a shared table is evidence, never authority* — is why the address table is forward-populated and why a miss is a fault.

★ **ADDED 2026-09-11: what Layer 1 is defending, stated narrowly.** The owner's ruling on the barrier-driven refresh scopes this model tighter than anything above says. **We defend against breakout and illegal action. We do not defend a guest against corrupting itself.** A malicious guest can write its own page-table memory underneath any window we read it in; the obligation is to validate that what we read cannot make us touch memory we do not own or act for another `Proc`, *not* to make a lying guest's own mappings coherent. §4.4 has the ruling verbatim and the consequence: the security-relevant boundary is object lifetime and our retire path, which we mediate directly, and it is not the TLB.

⊘⊘⊘ **CORRECTED 2026-09-11, AND THIS IS THE MOST CONSEQUENTIAL STALE CLAIM IN THE DOCUMENT.** The paragraph below reads two measured zeros as a fact about the GSP-emulated compute path. **Both zeros were real and both were useless**, and the reason is the same one this paper records elsewhere as its own recurring defect: *a census over transports is only as complete as its list.*

- `INVALIDATE_TLB` RPC `fn=200` measured **0** because it *cannot* fire: its only caller is gated on a vGPU predicate that requires `IS_VIRTUAL`, which is false for us by construction.
- The `MEM_OP` `MMU_TLB_INVALIDATE` pushbuffer method measured **0** because **that is UVM's transport, not RM's.** RM never reaches the copy-engine path for page tables at all.
- ⇒ **The census had no row for the transport RM actually uses**, which is a raw BAR0 store to `0x00B8_30B0`. The zero was a property of the instrument.

⇒ **"Read at the invalidate" is not refuted. It is one of THREE synchronisation points, and the current architecture is built on all three** (§4.4). ★★★ And the third point is the sharpest thing this correction buys: **on a GSP part the mapping work happens inside the GSP, which is us, so for RM's own map calls no barrier is coming at all** — we serviced the map, so we are the one who knows it happened, and the refresh has to live inside our own implementation of the RPC. ⚠ **Neither the old reading nor the new one is complete**: a source audit of the 580 open driver (2026-09-11) found page-table writes that none of the three points covers, and §4.4 enumerates them.

⊘ **The original paragraph is kept below, because a reader who acted on it needs to see which half was wrong.** Its general rule — *the Layer-2 edge is whichever edge the protocol commits at* — is right, and is exactly what forces the count to three. Its instance was wrong.

And the C's measurement sharpened it in a way worth carrying. The Layer-2 edge is *whichever* edge the protocol commits at, not a favourite one. On the kernel/UVM/RM paths that edge is the TLB invalidate. On the GSP-emulated compute path it is *not*: round 6 of the #14 investigation measured **zero** occurrences of both invalidate transports there. A design that had encoded "read at the invalidate" as *the* Layer-2 rule would have had no synchronisation point at all on the path that matters most. The general rule survived; one of its instances did not.

What the C artifact does not have is Layer 1. There is no lock of any kind between QEMU and an isolate in a system that runs 22 real GPU applications at host parity. It ships a copy-once snapshot instead, and the reasoning is written out in its source. The working system to date has been running on Layer 2 alone — which is correct for a cooperating guest and is not a security property.

And the boundary points both ways, which building the GSP made concrete. Everything above is about not trusting the guest. **GSP-S1** is the mirror: the guest kernel is inside the boundary we are defending, and the faked GSP is a device that can corrupt guest kernel memory from a width *we* choose. The mechanism is fully traced in `mode2_gsp_port_plan.md` §4.6 and reduces to one sentence — at driver 580 the guest reads our `elemCount` field into a copy loop bounded by nothing but the ring, and the byte after its staging area is the `msgq` metadata it is actively using, so the corruption is immediately self-amplifying. It is the same defect class as the C artifact's unguarded `% s->q_msgcount` `SIGFPE`, pointed at the guest instead of at QEMU. The obvious objection — "*the guest is the victim, so it is the guest's bug*" — is answered in the plan and the answer is that it is both. **At the previous revision this invariant was written down and not enforced, and this paper called that its most security-relevant sentence-rather-than- mechanism. It is now a mechanism** (`250b78c`; see the GSP-S1 row above and §8.9 for why the bite that proved it is not the bite that was predicted). The general point survives the specific repair: the boundary points both ways, and a faked device is a device.

7 What is tested, how, and what that does and does not prove

7.1 The shape of the suite

Measurement	Value	How
Tests reported passing	**2 926**	`cargo test --workspace --no-fail-fast` on the RTX 3060 bench, recorded at `9871d315` (2026-08-14). *Was 651, then 2 007.* [unverified] here — no `cargo test` was run for this revision
Tests standing RED at master	**7 on 4 targets**	**★★★ The previous revision printed a green count with no red count beside it, and there has been a standing red set the whole time.** It is enumerated in `docs/design/hardware_test_tier.md` §6 and re-verified at w329; the count itself *"moved and was mis-stated twice"* (6/3, then 7/3, then 7/4). *A pass count without a fail count is not a result*
`#[test]` attribute sites	**2 985**	grep over workspace members; excludes the detached generator and the `trybuild` ui fixtures. [measured] here
`proptest!` blocks	16	each is one test that runs many cases
Checked-in proptest failures	**20 seeds, 4 files**	Unmoved since the previous revision. ★ **This paper said "0" two revisions ago and that was wrong when printed** (§10 row 22)
KVM-gated tests	**56 reached**	up from 36; CI floors the *reached* count
GPU-gated tests	**1 reached**	the copy-engine hardware acceptance (`CeEvidence::copied()`). ⊘ **One test** — and note what that means now: the results in §8.3 are *not* carried by the test suite at all. They are graded by bench scripts against pre-registered outcomes, and **no** `cargo test` **run asserts that a guest can compute**
Sandbox-gated tests	10 reached	the containment family
Compiled `ogkm` oracles	**5 families**	VBIOS, GMMU-format, work-submit token, pushbuffer, USERD-chid — each compiles NVIDIA's *own* encoder as the oracle
TSan targets	4 files	nightly only
Fuzz targets	1	`parse_pushbuffer`; CI compiles it, does not run it
Implementation lines	**198 122**	`crates/*/src/**/*.rs`, excluding the detached generator. [measured] here
Test lines	**158 170**	`tests/`, `crates/*/tests/`. [measured] here
`cargo fmt --all --check`	**RED, 40 files**	[measured] here, at `5c367a38`, `rustfmt 1.9.0-stable`. ★ **And it cannot be attributed**: `rust-toolchain.toml` pins `channel = "stable"`, not a version, so the gate's own definition of correct moves underneath the tree — the project's bench notes record the same toolchain *"reformatting 62 files of untouched master"*. *This is a gate that can go red with no commit*
`cargo clippy -D warnings`	**RED, 5 errors**	Recorded red at master since at least `72f902f` (2026-08-14), at `proto.rs:118`, `rmgraph.rs:584` and three sites in `rm.rs`. [unverified] here — not run

★ **"The suite runs in about 20 seconds" has been deleted rather than updated.** It was [unverified] when printed and the tree has quadrupled since; the honest replacement is that the authoritative run is no longer a stopwatch figure at all but a *phase table* (`scripts/run_full_suite.sh`), whose mutation phase alone ran 33 374 s on its last outing. The OS-free property the 20-second figure was really claiming — no GPU, no OS dependency, no wall clock — does survive, and is still what makes the adversarial suites affordable on every change.

7.2 The CI gates — and the gate that GitHub cannot be

There is one workflow file with six jobs, and the `stable` job grew from fourteen steps to **twenty-four**. Seven of the nine new ones are *reached-count* floors for oracle families that only exist on a box with the vendored NVIDIA trees, plus an *ABI-quarantine* gate, an *ogkm-version-tag* gate (every NVIDIA citation names its tree) and **the claim-ledger gate** described below.

The bridge-exclusivity gate fired on its first run, on `kayfabe-gsp`'s own stale doc paragraph claiming the bridge would live in `kayfabe-fwd`. That is worth more than the gate itself: a gate that has never been observed to fire is not evidence, and this one earned its keep before it was five minutes old.

★★★ **GitHub CI is opportunistic convenience. It is not the definition of green**, and the project says so in an owner ruling rather than leaving it implied: *"this project cannot rely on gh for iterating for free hardware … ensure whole suite can still be run if you have real gpu."* The authoritative run is `scripts/run_full_suite.sh` on real hardware, whose exit 0 means *everything this box can run, ran*.

The failure it is written against is this paper's own founding doctrine. A test that is COUNTED but never PASSES is worse than a missing test. There are two live instances and both are honest about it: the 51 `/dev/kvm` tests and the 10 VBIOS-oracle tests are counted on GitHub and never run there, because `ubuntu-latest` has neither the device nor the vendored trees. Both print a marker on the *skipping* arm, straight to stderr, deliberately bypassing libtest's capture — because capture swallows output on the **passing** path, which is the arm that matters. ★ And the runner does not know those families by name: it **derives** them from the markers a run emits, so on a box whose profile promised the capability a single `SKIPPED` is a red run. *A gate quantified over a hand-list is a smaller true statement.*

7.3 The claim ledger — a gate on the paper's own vocabulary

The most unusual instrument in the repository, and the one this document is itself held to. `scripts/claim_ledger.py` sweeps every tracked Rust source and design document, finds every prose *block* making an epistemic claim (*measured*, *observed*, *verified*, *confirmed*, *proven*), and asks one question of each: **does the block name what is behind the claim?** It classifies into [measured] / [inferred] / [assumed] / UNATTRIBUTED, and enforces three bars: **381 unattributed (a ceiling), 66 conflated (exact), 17 bare-hardware (exact).**

The middle class is the named failure mode. CONFLATED means a block says *measured* or *proven* and the only thing behind it is a *citation into source* — `ogkm`, nvproxy, or the C artifact. *Reading the open kernel modules tells you what the driver does. Only a live boot with real work tells you what happens.* The bar is exact in both directions, because a gate that only catches growth leaves slack a later claim can be added into.

★ **Four things this gate says about itself, and they are the reason to trust it.** **(i) The universe is derived, never listed** — `git ls-files` plus untracked-but-not-ignored files, after a measured incident where a pre-commit run read 383 with three new modules unstaged and 385 the moment they were added. **(ii) It cannot check that the evidence exists**, that it says what the claim says, or that it was ever run: a block citing an imaginary capture passes. *Adjudicating a citation is human work* — and §3.1 is the worked example of a citation gate being satisfied by a source that said nothing. **(iii) The unattributed bar is a ceiling and is deliberately weaker than the other two**, "a worse gate honestly described, not a better one described optimistically", because an exact pin across a tree several agents write into at once turns an unrelated doc commit red — *which is how a ratchet decays into a formality.* **(iv) Two of its four firings were false positives and both moved the pattern, not the bar.** One site cited `ogkm` and then said, in the same block, *"No Ada card was measured for this number"* — the exact honesty the gate exists to encourage, scored as the defect it exists to catch. ⊘ *Absorbing that site into the bar would have been the cheaper edit and the wrong one: a gate that penalises the behaviour it is for teaches people to stop writing it down.*

★★★ **And at this pin the gate is RED, which the previous revision could report as green and this one cannot.** [measured] here, `scripts/claim_ledger.py --gate` at `5c367a38`, **exit 1**:

class	count	bar	
UNATTRIBUTED	1 166	381	ceiling — **3.1× over**
CONFLATED	90	66	exact
BARE-HARDWARE	36	17	exact
for scale: [measured] 2 167	[inferred] 857	[assumed] 136	

The bars have not been touched since `1227babe`**, 2026-07-31**, while the tree tripled and roughly forty directories of committed bench artifacts (`traces/**/*.md`) entered the gate's universe — which is derived from `git ls-files`, so committing a trace README containing the word *measured* moves the number with no source change at all.

⊘ **Two readings, and this paper will not choose between them without a measurement it did not take.** Either the tree really has accumulated 785 new unattributed claims, or the gate's universe grew in a way its bar was never re-derived for. **Both are failures of the same kind** — a ratchet whose denominator moves is not a ratchet — and the project's own doctrine names the second one exactly: *"a gate quantified over a hand-list is a smaller true statement"*, here inverted into *a gate quantified over a growing derived set is a bar that means something different every week.* What is *not* in doubt is that the authoritative local gate run (`scripts/ci_gates.sh --all`) cannot currently reach a clean exit.

⊘ **And one honest note that has not changed**: `.tex` is not in the gate's universe — it sweeps `*.rs` and `*.md` — so **the whitepaper is the one artifact in this repository the claim ledger does not police**, which is worth

knowing before trusting any *measured* in it more than the design document it came from.

Job	When	What it asserts
`stable`	every push + PR	**Twenty-four steps**: build; **test with KVM forced absent**; *six* reached-count floors (KVM, sandbox, VBIOS-oracle, GMMU-oracle, token-oracle, pushbuffer-oracle, USERD-chid-oracle); clippy `-D warnings`; `fmt --check`; the hexagonal-boundary grep; the VMM-vocabulary grep; **the generation-name grep** (no concrete chip or driver version in the logic crates); **the ABI-quarantine gate** (a C layout is quarantined, or provably own-wire); the `unsafe`-surface `ls` gate; the host-pointer gate; the GPA-accessor gate; the bridge-exclusivity gate; **the ogkm-version-tag gate** (every NVIDIA citation names its tree, ratcheted at 161 untagged); **the claim-ledger gate**; three unsafe-containment asserts including the **two-crate ratchet at 91 and 49**; and `fmt --check` for the fuzz workspace.
`aarch64`	every push + PR	`cargo check --workspace --target aarch64-unknown-linux-gnu`
`nightly-fuzz`	every push + PR	the fuzz harness *compiles*
`slow`	nightly	the full suite with `KAYFABE_SLOW=1`
`tsan`	nightly	ThreadSanitizer over four threaded targets, `-Zbuild-std` so std's own synchronisation is instrumented
`mutants`	weekly	`cargo mutants` over every production crate

★★ **Two of the steps in that table are RED on `master` and have been for at least a week, and the previous revision of this paper listed them as things CI asserts.** `cargo fmt --all --check` fails on **40 files** ([measured] here) and `cargo clippy --workspace --all-targets -- -D warnings` fails with **5 errors** (recorded at `72f902f`, not re-run here). The consequence is stated in the project's own words: *"`ci_gates.sh --all` runs both, so the authoritative local gate run cannot currently reach a clean exit for a reason that has nothing to do with any rung's change."*

★ **And the `fmt` red is not attributable, which is the more interesting half.** `rust-toolchain.toml` pins `channel = "stable"` — a *moving* channel, not a version — so what "formatted" means changes underneath the repository whenever the toolchain updates, and the bench notes record this exact toolchain *"reformatting 62 files of untouched master"*. ⊘ This paper therefore cannot say whether 40 files drifted or `rustfmt` did. **A gate that can go red with no commit is a gate whose red carries no information**, and it is the same species as everything in §8.21: not a wrong statement, a statement whose subject moved.

7.4 Exercised versus merely present — the project's own core doctrine

This distinction is the most useful thing in the repository, and it is worth reproducing the incident that generated it, because it is the template for every criticism in §8.

> The conservation ledger's first census over the mean run reported **0 leaked objects** — and that was a **true negative, not a pass**. A design document claimed the workload "exercises T0 thousands of times per run"; the churn it actually drives adds and removes *RPC* bindings, which own nothing host-side, and the script's two real subset-frees targeted channels that phase 0 deliberately leaves virgin. **The path had never been reached.** Reaching it took a new test phase. Once reached, the honest baseline was **24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees.** [docs/design/testing_doctrine.md §1]

Three rules follow, and the project applies them: a reclamation test that does not first allocate proves nothing; every instrument needs a non-vacuity assertion (a case where it is known to read non-zero); and **bite-check the instrument, not only the fix** — reverting the fix must move the number, and if it does not, the number is not measuring the fix.

Bite-checking is real practice rather than aspiration, and the evidence is that the repository records bite-checks that *did not bite*: at least six such notes are written into the source at the site they concern, each explaining what the neutering failed to move and what was added as a result. One has been promoted into a permanent test that plants a gap and a repeat in a trace and requires the order checker to fire. A commit message from this week reads, in part, *"the bite harness itself lied twice."*

The doctrine also produces two mechanisms directly. Slow tests are gated by `KAYFABE_SLOW`, and when unset they print `SKIPPED (slow): …` **straight to stderr**, bypassing libtest's capture; nothing in the suite is `#[ignore]`d, on the argument that `#[ignore]` is invisible in a summary and impossible to count. And the KVM gate prints on *both* arms — `KVM-GATE: RAN` or `KVM-GATE: SKIPPED` — with CI asserting a floor on the sum, because the reverse failure (all 36 gated tests silently vanishing while CI stays green) is equally silent.

7.5 The mean test, and composed versus isolated runs

`tests/tests/l1_mean.rs` is the largest single test file (35 tests) and is designated *the arbiter*. Its standing is: *pass = the design survived contact; fail = the doc changes, not the assert.* The rule it exists to enforce is an owner directive with four obligations — drive realistic multi-process × multi-thread × multi-GPU traffic end to end through the mocks and assert the observable end state; make it mean rather than happy-path (free mid-flight, race a re-declaration against a still-publishing generation, kill a worker partway, run both lock modes); wire every new milestone into it; and **never narrow a test to make it pass**.

The justification is measured, not asserted. One identity round landed 7 bite-checks; **none of the 363 pre-existing tests caught any of them**. Every real defect that day came from a composed run or a newly sharpened criterion. In the doctrine's words: *"the happy path found nothing, all day."*

A representative finding: a test named `worker_death_retires_the_proc_loudly_and_never_resurrects` was green — and green *only because it never issued an apply after the HUP*. The composed run put a worker hangup and an alloc/map-heavy workload in the same run and found that an out-of-band retire is undone by the next refresh: a guest able to crash its isolate worker got a **clean new isolate** on its next RM event. The first fix for the related leak class was a use-after-free of its own making.

7.6 The conservation ledger

The instrument all eight teardown paths are measured by. `HostLedger` replays the recorded verb log and demands every acquisition matched by exactly one release and nothing released that was never acquired — objects *and* mappings. A test may declare a `ResidueClaim` for what it knowingly leaves outstanding, and the counts are compared **for equality, not as a bound**: *less* residue than declared also fails, so a fix that closes a leak must delete the claim that excused it. There is deliberately no `allow_leaks` escape hatch.

Its stated limit, recorded in the source: the ledger increments only on *success*, so it says nothing about failed verbs. That is why a separate `VerbFailure{err, orphans}` type exists to carry handles out of a chain that died partway.

7.7 The mutation gate — five lies deep, and the fifth is the best one

The gate mutates the code and measures whether the suite notices. It has now been the most valuable instrument in the project *and* silently wrong **five** separate times, every one of them wrong *upward*. That asymmetry is not coincidence and it is the single most important thing to understand about this instrument: every way an instrument like this fails makes the code look better than it is, because a mutant that is not run, not built or not counted is a mutant that never survives.

Lie 1 — it tested nothing. `kayfabe-fwd` and `kayfabe-rt` have no unit tests of their own; without `--test-workspace true`, cargo-mutants ran an empty suite and reported everything missed. (This one is the exception that proves the rule — it failed *downward*, and was therefore noticed immediately.)

Lie 2 — compiler crashes vanished from the denominator. cargo-mutants rewrites one file per mutant in a copied tree that reuses one incremental dep-graph cache, and that cache does not survive the churn — rustc ICEs. cargo-mutants cannot tell a compiler crash from a type error, so it files the mutant *unviable* and **silently removes it from the denominator**. Measured: **136 of 293 mutant builds ICE'd**, turning a real 88.95% into a reported 67%. CI now fails on any ICE in a mutant log, and on a zero viable count, *before* the threshold is read.

Lies 3 and 4 — the configuration file was never read, and the gate could report 100% from a broken suite — are covered in §8.9.

★★★ Lie 5, measured 2026-08-03, and it is the sharpest of the five because the score was *exactly* 100%. A full-suite run on a rented GPU box printed:

```
mutation score 100% (killed 1936 / viable 1936), threshold 91%  ok (33374s)
```

and, *in the same phase*, fifteen `No space left on device` errors plus `Error: write outcomes.json`. It had found **8 738** mutants and scored **1 936**. **6 802 mutants — 78% — were unaccounted for, with zero `MISSED` and zero `TIMEOUT` lines. They did not survive and get counted; they vanished.**

The mechanism was already understood *in that function*. The ICE guard directly above it says a dropped mutant *"removes it from the denominator and INFLATES the score"* — and guards only the ICE trigger. ENOSPC does the same and worse: it stops the result files being written. `missed.txt` was never created, the counting helper read its absence as **0**, and `viable = killed + 0` forces the score to exactly 100%.

★★★ This is the project's own FIFTH ORACLE LIMIT reproduced inside its own harness (§3.1): *an empty*

capture is evidence of nothing, not evidence of emptiness. Absence read as zero, in a different repository, by a different mechanism, four days after the rule was written down. ⊘ **And a perfect score is the alarm, not the achievement**: 1 936/1 936 with no misses, after a run throwing ENOSPC, is the signature of a universe that collapsed under the campaign.

Four new refusals landed, each proven to fire, plus a control arm proving a genuinely clean campaign (8 738 found / 8 600 scored) still passes. ⚠ **And the harness was not wholly blind**: its summary already printed *"★ …and the disk is below the comfort bar"*. But the *phase verdict* contradicted the harness's own warning — **and the phase verdict is what a reader believes.**

The historical numbers, each true of the scope it was measured on: **99.2%** (245/247) on the pure L0 core; **92.44%** (159/172) on the L1 threaded surface. Both were measured over a population that *included the instruments*, because the exclusion file was inert.

Do not quote a mutation score from this project. There is still not a defensible one. The threshold in force is `91` (`MUTATION_THRESHOLD_PCT`), so the previous revision's "there is currently no mutation bar in force at all" is superseded — **but a bar in force is not a measurement, and the only recent campaign to complete is Lie 5's, which is void.** The project's own rule is that the bar is derived from a measured baseline under the configuration CI will actually use, and **no such run has completed**. Its number is not available to this document.

The campaign that *has* completed, labelled honestly. 2026-07-28, 2 898 mutants, 5 h 49 m: 2 305 caught, 112 missed, 3 timeout, 478 unviable. Product-only, with scaffolding removed from the denominator, that computes to **95.52%** (**95.38%** if a `TIMEOUT` is not counted as a kill, which is a real choice — cargo-mutants scores a timeout as *killed*). ⚠ **That run's tree was modified underneath it while it ran**, by a concurrently dispatched task, and the number is therefore reported as an indication and not as a measurement. It also does not describe the population CI will now test: with the configuration finally being read, `--list` yields 2 667 targets, not 2 898.

What that campaign found, which is worth more than its score. `kayfabe-gsp` is the floor at **92.71%** and holds 36 of the survivors — exactly as expected for a crate no campaign had ever covered, and the direct answer to the previous revision's "no mutation campaign has ever run over them". `kayfabe-abi`'s encode side, previously a named gap, is **closed**: 195 mutants, zero survivors. Of the 115 survivors, **8** are provably equivalent (each proven by its sibling mutant being caught at the same site), **22** are dead public API or `Debug` diagnostics with no callers anywhere, **17** are scaffolding, and the remaining **68 are real untested logic, individually named**. A debt that is enumerated to 68 named items is a different object from a percentage.

And one methodological result from closing some of them, which cuts against the usual reading of a surviving mutant. `b612a91` closed twelve `kayfabe-gsp` survivors and found that one of them — a `+` → `*` in `RegionMap::load` — was not guarding correct code. It was *masking wrong* code: the function reported `pages.len() + i` as the index of a misaligned page-table entry while `pages.push` ran in the same loop, so the reported index was $N + 2i$ and correct only for the first entry of each table page. A guest-facing diagnostic naming the wrong entry of a hostile table. **"This mutant survived" is therefore not automatically "write a test"; it is sometimes "read the function".**

Three of the targeted survivors were *deliberately not closed*: `RegionMap::is_empty` and `page_size` have zero callers repo-wide, and writing tests for them would be exactly the score-gaming the instrument exists to detect. `is_empty` cannot simply be deleted (`clippy::len_without_is_empty` requires it once `len` exists) and deleting dead public API is an owner call. That distinction — kill what a real readback naturally asserts, refuse to invent a probe for what nothing calls — is the whole difference between a mutation score and a coverage number.

7.8 The OS-free configuration, and why it exists

`KAYFABE_NO_KVM=1` forces the KVM probe to report the device absent, so the pure logic core plus mocks — no OS layer at all — runs on purpose on any box. CI runs *this* as its default test configuration.

The reason it was made explicit is the doctrine applied to itself. GitHub runners have no `/dev/kvm`, so CI was already running the OS-free configuration — **by luck of what the runner image happens to lack**. It was unreproducible on any developer box, and it would have lapsed silently the day a runner image started shipping the device. In the commit's own words: *"a guarantee that holds by luck is the same shape as a green instrument on an unexercised path."*

The owner's question that prompted it is worth repeating because it is the right question: *"I hope you didn't delete the mocks so the core can still be tested without OS layer? and that must pass as well so we don't lock in architectures/os semantics/qemu and later regret."*

7.9 What the suite does not prove

- **It does not prove anything about NVIDIA.** Every test runs against `MockArch`, whose encodings are deliberately *not* NVIDIA's. That is the right design for proving the seam, and it means the suite cannot catch a wrong belief about the real protocol. Only the C artifact and the bench can do that.
- **The with-KVM configuration has no automated gate.** CI runs *only* the KVM-absent configuration. The 36 tests that use a real `/dev/kvm` — including the entire real memory plane and the vCPU/MMIO dispatch — run on developer boxes and the bench, and nowhere else. This is the largest hole in the CI story, and the workflow says so in its own text rather than hiding it.
- **The fuzzer is compiled, not run, by CI.** One target, over the one place raw guest bytes reach the core. Its two historical findings were minimised into deterministic in-tree regressions, which is the stronger form. The corpus is git-ignored, so a fresh clone starts from empty.
- **Single-threaded logic testing of a concurrent system.** The core is tested deterministically because it is a pure state machine; the concurrency is tested separately by the threaded suites plus TSan. The argument is sound, but it depends on the core genuinely having no interior mutability — which is asserted at compile time, and is the sort of property worth re-checking if you are looking for holes.
- **The page-size axis is designed and not built.** The testing doctrine cites `[4 KiB, 16 KiB, 64 KiB]` as an application of its "both modes ship from day one" rule. The environment variable that would select it does not exist in any `.rs` file — only in forward-looking comments. The `LockMode` half of that same rule *is* built and exercised.

8 Discussion: the good, the bad, and the risky

8.1 What is genuinely good

The purity constraint is real and it is mechanised. A "pure core" claim is usually aspirational. Here it is three CI greps, a manifest-level `forbid`, a naming rule that makes `ls` the audit tool, and a ratchet on the exact number of `unsafe` relaxations. The external dependency surface is three crates, two of them dev-only.

The failure ledger is unusually honest and it is load-bearing. Two design documents carry "contact logs" — `l1_concurrency.md` §12 and `l1_os_shell.md` §14 — recording every place building the code proved the design wrong. They are long. They contain entries whose titles are things like "the criterion was wrong", "boundary-1 broken", "two sections of §3 contradict each other", "the design was wrong about its own severest claim". A project that writes these down is a project whose other claims are worth more.

The adversarial suites found real bugs before hardware existed. 15 at L0, more since at L1, including two control-plane *wedge* bugs reachable by a hostile guest process — a parked `SET_PAGE_DIRECTORY` and a parked `MAP_MEMORY_DMA`, each of which could be aimed via a dup alias to make a benign third party's allocations fail forever. Both were found by a property test, not by inspection. *The same parked-fact machinery produced this revision's worst core defect too* (§8.8), which is a fair reading of where the difficulty in this design actually lives.

The bridge was designed on paper before it was built, and the design was then reported against. `gsp_core_bridge.md` was written first, in full, including the finding that changed the roadmap; the four implementation commits then recorded roughly two dozen places where that design met the compiler and lost, *by section number*, including three of its own test obligations turning out to be unbuildable and one turning out to be wrong. A design document that is graded by the code that implements it is worth more than one that is merely followed.

Both lock modes and both fallbacks ship as first-class tested modes. The argument is sharp: the slow path is the path the system takes *when something has already gone wrong*, and a fallback exercised only by the fast path declining has test coverage proportional to how often the fast path fails — which is the quantity the optimisation exists to drive to zero.

The ABI oracle design. Pinning generated layouts against three independent readings, with a test that documents exactly how they disagree, is better than what most projects do, and it caught the C artifact's own parity test being weaker than the code it guarded.

★★ Compiling NVIDIA's own code as the oracle, five times. The strongest testing idea in the project is newer than any of the above and does not appear in its doctrine document. Where a wire encoding had to be settled — the doorbell token, the GMMU page-table format, the pushbuffer codec, the USERD channel-id decode, the VBIOS layout — the answer was not transcribed from the headers and asserted. **NVIDIA's own generator, reader and recombination logic were compiled and run as the oracle**, and our implementation differentially tested against them. A transcription cannot detect a shared misreading; a compiled oracle can, and each of these caught one. Five such families now have their own CI reached-count floor, so a box that

cannot run them says so instead of quietly counting them as passes.

★ **Corrections are struck, not deleted — and the practice survived contact with being wrong in public.** `boot_measured_2026_08_01.md` carries sections marked **REFUTED** at their heads, kept in full, with the observation preserved and only the inference withdrawn: *"it is kept, not deleted, because the observation below is accurate and only the inference was wrong — and because this is the clearest example in the file of the failure mode the whole document exists to resist: a chain of assertions read as a chain of causes."* A project that writes that down about itself is a project whose other claims are worth more.

8.2 ★★★ The execution plane was the wall — and it was crossed

This section replaced every previous revision's headline, and then was itself falsified. The previous revision's version read: *"the critical path was always the plane where a real engine has to move real bytes, and no amount of protocol work reaches it"*, with a specific, well-cited prediction of where the boot would keep dying. **It was right about the kind of problem and wrong about its finality.** The plane was crossed in eleven days, and this section is now the account of *how*, which is more useful than the wall was.

What the wall was, and why it could not be answered with a protocol

Partway through initialisation the guest driver stops asking questions and runs an experiment. `memmgrInitCeUtils_IMPL` calls `memmgrTestCeUtils` unconditionally: it writes `0xAABBCCDD` to framebuffer, issues `memmgrMemCopy` with `TRANSFER_FLAGS_PREFER_CE`, reads it back, and asserts `sysmemData == vidmemData`. **A functional end-to-end test of the copy engine with a read-back comparison. There is no control-shaped answer to it.** Its status returns through `NV_ASSERT_OK_OR_RETURN` into a `kernel_fifo.c` arm that is `NV_ASSERT(0)`, fatal for any non-`NV_OK` in this phase, so the pre-init sweep's *"`NV_ERR_NOT_SUPPORTED` = amputate and carry on"* rule does not apply.

Every escape hatch was closed by citation, and one structurally: the Blackwell-only skip takes the `else` arm on GA106; `bDisableGlobalCeUtils` requires modifying the guest, which is outside "stock, unpatched"; the `IS_VIRTUAL_WITHOUT_SRIOV` arm makes the interrupt self-test permanently unpassable; and `!IS_SILICON` **does not exist** — the three flags behind it are declared in the driver's own headers and *assigned nowhere in the tree*, so it is constant true. *This is not a lie we decline to tell; it is a lie we cannot tell.*

★★★ **That analysis was correct, and the conclusion drawn from it was wrong in an instructive way.** The reading was *"the driver demands a real copy engine, therefore the remaining problem is the copy engine"*. The actual remaining problem was that the driver demands **a real GPU behind every plane it touches** — and the way through was not to satisfy this one test but to stop treating the planes as separable. The engine executes; we do not answer for it. §8.3 is what that took.

8.3 ★★★★★ The ladder to first compute — `cup2` → `cup3` → `cup8`

Every rung below is a boot on a real RTX 3060 (GA106) with host driver **580.159.04 open**, a **stock, unpatched** guest NVIDIA driver, QEMU 10.2.4 with the QOM shim, and a named source revision embedded in the binary and checked by a stamp gate before the run is graded.

Rung	Metric	What it does and does not establish
cup2	CUP2_RC=0	A CUDA *copy-engine* round trip: cuCtxCreate, cuMemAlloc, cuMemcpyHtoD/DtoH. ⊘ **Not compute**, and the project renamed it to say so. Its 4-byte transfer does not even use a copy engine on real hardware — it is the compute class's I2M unit with the payload as a *pushbuffer literal*, so a passthrough is correct by construction and proves nothing about an engine.
cup3	CUP3_VAL=43	★★★★★ **FIRST COMPUTE.** [measured] 2026-08-14, rev 0655e6aa, traces/w297_cup3/. A JITed PTX kernel computes out = in*3 + 1 on in = 14. **43 is unforgeable in this stack**: a copy engine copies and memsets, our emulator has no arithmetic, and a forged completion carries a payload *we* chose. ⇒ the host GA106 **GR** engine executed the guest's shader. Reproduced **6/6**.
w299	CUP3A_VAL=43 CUP3B_VAL=43	★★★★★ **TWO CONCURRENT GUEST PROCESSES BOTH COMPUTED.** [measured] 2026-08-14, rev f459cffa. The device saw three isolates materialized, three live, zero refusing, with distinct proc= ids. **The C artifact's #14 shape did not reproduce** — including on a staggered arm where the second process started while the first was *demonstrably inside* cuCtxCreate, gated on the first's own print rather than a sleep.
w367	TORCH_OK ×3 8/8, 8/8, 8/8	★★★★★ **THREE SEQUENTIAL GUEST PROCESSES PASSED IN ONE BOOT.** [measured] 2026-08-20, rev cca2eb4b, 600 s step bounds. A first torch process, a second with no nvidia-smi between it and the first, and a third after nvidia-smi — all avail=True sum=4.0, all **8/8** completions observed, all ringing *identically* 104 Ce + 403 GrCompute doorbells; host dmesg 0 NVRM 0 Xid. Two boots earlier every process after the first rang exactly eight doorbells and stalled forever. ⊘ **A *fourth* process still fails**, above the ioctl boundary, cause unknown; $n = 1$ boot. §8.11.
cup8	bad=0 maxerr=0	**Byte-exact matmul at scale.** N=2048, 16 MiB per matrix, 48 MiB device, grid=(128,128) block=(16,16), traces/w308_cup8/; and again at **N=3072** (36 MiB operands, 108 MiB device) over 12 verified iterations, traces/w327_cliff/. ⊘ cup8 allocates its buffers once and keeps them, so it never performs the allocate → free → allocate-elsewhere sequence that is the real reclamation defect: *evidence the size is fine, not evidence the defect is gone.*

⊘ **Four things the ladder does not establish, each recorded by the run that produced it rather than found by a critic.**

(1) Every green is a RELAXED green, the relaxations are named, and the shipped default is not among them. Eleven arms are forced on by the bench runner. **Nine were single-ablated** and **five are measured load-bearing**: VAS_PUBLISH=drain (its alternative produces an Xid 31 FAULT_PDE), GR_ROUTE=passthrough, GUEST_RING=ring, CE_EXECUTOR=host and PT_WITNESS_EXEC=on. ★★★ KAYFABE_GR_ROUTE **still defaults to** refuse **in the shipped code**, so the configuration that computes is *not* the configuration that ships. ★ The arming was verified from the device's own emissions rather than from the environment, *because a boot happening is not an arm running*.

⊘ **And the ablation census was itself wrong and corrected within the day.** Two arms recorded INERT for cup3 were later measured to *each alone kill* a second workload (a raw copy-engine client with no libcuda), and were restored. *"Inert for* cup3*" had been read as "inert"*. The project's response was a mechanism rather than a note: an inertness claim must now clear **both** workloads before it counts.

(2) n **is small and this project has measured its own false-negative rate.** A single-boot 43 is **wrong 1 time in 5** on these boxes — 4/5 on the branch *and* 4/5 on same-hour clean master, both reds identical field-for-field. *So a single green boot is not a grade*, and the cause of the intermittent is known: a **drain-budget truncation** whose walk ascends VA order, so it always drops the top of the address space — exactly where the completion semaphore lives.

(3) It is one shape of work on one part. cup3 is a **1×1×1 launch of a six-instruction shader**: it says nothing about occupancy, multi-block dispatch, shared memory, barriers or throughput. Everything is GA106 and **no host driver other than 580.159.04 has ever been run against this code.**

(4) The bench is nested and that now matters. The rented box is itself a VM, so the Mode-2 guest is L2. The nested-virt tax is bounded at ≤ 2.2 ms/launch — which was 2.6% of an 85 ms doorbell trap and is **up to 55% of a 4 ms one**. ⇒ *Bare metal matters more now precisely because the doorbell path got fast.*

★★ How the boot got through, in the terms a reader can check

The mechanism is not one fix; it is one invariant, and the C artifact stated it in its own source before the Rust port re-derived it the hard way: *"a mapping is always backed before the engine that uses it runs."* The port's version

has **three co-equal population sources**, which is one more than either project's design documents claimed for most of the campaign:

1. **An RPC capture.** `NV2080_CTRL_CMD_GPU_PROMOTE_CTX` (`0x2080012b`) is snooped in flight and its `{gpuPhysAddr, gpuVirtAddr, size, physAttr}` entries folded into the address table. This is the source both trees' documents kept omitting.
2. **A doorbell-time sweep of the guest's GR page tables**, re-swept when a tracked page-table page is written.
3. **The observed CE page-table write**, decoded and latched at the completion-semaphore release — the commit point that replaces the TLB invalidate, *which on this path does not occur* (§6.1).

★ **And one difference between the two implementations turned out to be load-bearing, at the granularity of a single line.** The C rounds every promote-derived mapping *up to 64 KiB* before mapping it (`asize = (size + 0xffff) & 0xffff0ull`). The Rust port originally bound at the *declared* length, which produced non-page-aligned rows and a **sub-page hole** — 2 560 bytes the port's own `resolve` answered `Miss` for, inside a page the guest had mapped. **The C could not have that hole; the port had it by construction.** The C also treats `NV_ERR_NO_MEMORY` on a `FIXED` map as *success* — it means the VA is already mapped in the host VAS — a semantic the port had to match or it would refuse exactly the context buffers the host RM had already placed.

★ And a whole class of walls turned out to be one wall

Six months of this campaign chased faults one at a time: pushbuffer VAs, the semaphore page, the CE operand, its extent, the GR write. **Each is one instance of the same missing invariant**, and the C artifact went green *without servicing or forwarding a single GPU fault* — there is no fault-buffer emulation anywhere in it — because publishing the whole working set before the engine runs makes that class of fault **impossible** rather than handled.

⊘ **The honest cost of that design is stated in §8.4**: publishing the working set before every submission is what makes our doorbell handler the dominant term in a kernel launch. *The correctness argument and the performance problem are the same mechanism.*

8.4 ★★★ Performance: the binding constraint moved twice, and every number here needs a scope

⊘⊘⊘ **CORRECTED 2026-09-11: THE ARCHITECTURE THIS SECTION MEASURES HAS BEEN RESTRUCTURED, AND THE HEADLINE FINDING IS NOW A HISTORICAL ONE.** Everything below measures a device that published page tables *inside* the guest's doorbell trap, on the vCPU. It no longer does. **The doorbell defaults to deferred, the GSP command queue is serviced off the vCPU with no inline fallback, and BAR1/BAR2 default to untrapped** (§4.4). The **86.7 ms trap** and its 91.5 % page-table share remain a correct and useful [measured] decomposition of *that* design; they are not a description of the current one.

★★★ **And the successor measurement says the binding constraint moved to a place nobody predicted.** [measured] `w447`, $n = 2$: `worst_trap = 1 779 094 µs at bar0+0x110c00`, which is the **GSP queue-head register**, not a doorbell — `slow_traps(>1000µs) = 194`. **Deferring the work off the vCPU did not move it**, because the worker held the same lock the vCPU takes; two lock fixes since have not moved it either. ⊘ **And a stack sampler** (`w460`) **then found no vCPU in the MMIO write path at all across 16 s of steady-state running**, so the 1.87 s is *not* a steady-state property and where it happens is currently unknown. **No successor breakdown of that trap exists**; the histogram built to produce one has never been run, so this paper quotes no bucket figures. *The launch-floor arithmetic below should be read as pinned to the old design.*

★★★★★ **READ THIS BEFORE QUOTING ANY NUMBER IN THIS SECTION.** [measured] 2026-08-19: the guest's `submit_ms` for the *same workload, same binary, same box* was **85.32, 9.41 and 64.39 ms** on three consecutive boots — **a 9.1× spread with the middle boot fastest**. Within any one boot the seven iterations agree to ±2%, so **the value is decided at boot and then holds**. Across twelve boots that day it spanned **6.3 to 216 ms**. The same twelve boots held `sync_ms` at **10.23–10.40 ms,** ±0.8%.

⇒ **Any submit-side grade at $n = 1$ or $n = 2$ per arm is uninterpretable**, and the project says so about its own published tables. ⊘ The obvious explanation — a session ramp as host state accumulates — was **refuted in ten minutes**: the sequence is non-monotone, *and* free memory improved after the slow boot while submit stayed slow. **What varies per boot is still unknown.**

★ **The surviving half is the useful one**: a *null* on `sync_ms` is strong evidence precisely because `sync_ms` is the ±0.8% statistic, where the identical null on `submit_ms` would be worth nothing. *Two numbers printed side by side in the same*

table can have completely different evidential weight.

Where the time goes, and the two answers that were both right

★★ **The launch floor was our own doorbell handler**, and the measurement closes. [measured] 2026-08-14 (w315, `traces/w315_floor/`, $n = 2$ boots, $N = 512$): a guest `cuLaunchKernel` takes 90.9 ms, of which **86.7 ms is spent inside a single MMIO doorbell trap** with the vCPU halted. Inside that trap:

segment	ms		% of the 86.7 ms trap
`vas_publish`	48.3		55.7
`pt_decode`	22.3		25.7
`pt_sweep`	6.6		7.6
`pt_vascensus`	2.1		2.4
page-table + publication	**79.4**		**91.5**
the real host RM forward	**3.6**		**4.1**
our own logging	0.2		0.2
UNMARKED	0.009	**0.01**	⇒ *the breakdown closes*

⚠ **Two denominators, both correct, and mixing them is a documented error in this project.** 91.5% is *of the trap*; the same family is **69.9% of the 113.6 ms end-to-end launch**, and the host forward is 4.1% of the trap but 3.2% of the launch.

★ **And 95.6% of the trap is classified** `shape=work` — it costs the same on bare metal. ⊘ **That classification is [measured]; the inference from it is not**: the rung's own "what is not here" section says *no bare-metal control was run*, and the bench is nested, so the guest is at L2.

★★ The fix, and why one statistic could not grade it

Two changes were built, both default-off for weeks, and **they act on different statistics**:

arm (`mmio_doorbell total_us`)	**median**	**p90**	**max**
control (×2)	18 741 / 19 286	86 104 / 82 488	2 753 760 / 2 726 516
dirty gate	**2 197** (8.5×)	**4 431** (19.4×)	2 815 676 ⊘ *unchanged*
coalescer	30 311 ⊘ *1.6× WORSE*	76 670	**217 190** (12.7×)
both (×2)	9 016 / 11 759	10 712 / 14 807	249 118 / 394 035

⇒ **Graded on the median alone the coalescer reads as a regression; graded on the max alone the gate does nothing.** Only both statistics separate them, because they remove different work: the gate removes the per-doorbell page-table pass (the *typical* trap), the coalescer removes the guest-RAM pin drain (the *worst* trap). ⚠ $n = 1$ per arm for the two single-flag rows, so by the lottery above **those two ratios are owed a re-grade** and the project's own scoping note says so.

★ **The strongest row in that table is a cross-machine reproduction**: the gate's p90 going $86.1 \rightarrow 4.43$ ms independently reproduces an earlier campaign's $85.248 \rightarrow 4.078$ ms on different hardware, a different driver build and a different guest kernel. *A reproduced ratio is worth more than a repeated arm.*

★★★ **And the reason the handler was expensive was not the reason everyone assumed.** The hypothesis was that the per-doorbell scan is O(VAS size). **From the control arm's own log: walking 16 425 rows costs 0 ms.** The 40 ms appears only when the scan finds **eight** candidates, and it is **eight host round trips at ∼5 ms each, every one of them refused *"already joined"* and released** — repeated 87 times per boot for the same eight leaves. ⇒ *Making the check incremental would have saved 0 ms.* The fix was to arm a dirty flag at the **sink** — inside the two write sites of the table itself — rather than at an enumeration of boundary events, which makes a missed invalidation *unrepresentable* rather than merely tested for.

⊘ **And the arithmetic that pointed the wrong way looked like corroboration.** $48.3\,\text{ms} \div 16\,425\,\text{rows} \approx 2.9\,\mu\text{s/row}$ — a completely plausible per-row cost, arriving before anything was measured, for a scan that actually costs zero. **This project has now recorded four such *pre-corroborated* answers**, each a magnitude that agreed with a hypothesis

before the hypothesis was tested, and each wrong. *A quantity that agrees in advance is the weakest evidence that feels like the strongest.*

★★★ Then the binding constraint moved, and the campaign's headline number expired

An earlier rung fitted guest $= C + k \cdot$ native and got $C \approx 115$–132 **ms fixed and** $k \approx 22$–$27\times$, concluding that batching cannot help because the floor is per-*submit*, not per-*sync*. **That fit is now stale by its own tree's instruction**: the successor rung records, in writing, that *"the arithmetic must be re-derived against 4 ms, not against 115–132 ms"* — and **the re-derived multiplier is printed nowhere.** ⊘ This paper therefore does not quote one.

What replaced it is a cleaner and more uncomfortable result.

N	submit (ms)	sync (ms)	sync off-CPU	nvcsw
128	3.483	0.971	−0.003	0
512	3.699	22.734	−0.010	0
1024	4.103	59.786	−0.002	0
2048	**4.373**	**908.966**	+0.189	0

Submit is flat across a 4096× range of arithmetic while sync moves 936×, and the guest spins on-CPU for 100% of the wait, blocking zero times, at every size. ⇒ `cuCtxSynchronize` is not a floor to be optimised; **it is the kernel running**. **The submit path this whole campaign drove down is 0.5% of an** $N = 2048$ **launch**, so driving it to zero would improve that launch by half a percent.

★★ Two separable problems that bind at different kernel sizes, and conflating them is the error to avoid. At $N = 128$ submit is **78%** of the launch — the LLM shape, many small kernels. At $N = 2048$ it is **0.5%** and the compute itself is **22–81× off native**.

★★ Where the 22–81× lives: the operands are in the wrong memory

[measured] 2026-08-14, PCIe gen3 ×16 confirmed from the link itself, all three numbers taken with the same kernel on the same GPU:

placement	GB/s	
host VRAM	**313.5**	$n = 3$, ±0.04%
host pinned sysmem, 2 MiB pages	**12.33**	$n = 3$, ±0.2% — *the link, saturated*
our guest's operands, read by the kernel	**2.51**	$n = 3$ boots, ±1.5%

⇒ **125× below VRAM and 4.9× below the link.** The overall factor decomposes as **placement** $14.80\times$ × **aperture quality** $2.76\times = 40.85\times$, which is the same-hour measured ratio at $N = 2048$ — **and neither factor is the submit path.** ⚠ $n = 1$ per placement for the decomposition.

⚠⚠ **Do not conflate 2.51 GB/s with the copy path.** 2.51 GB/s is the GPU kernel reading operands during the matmul. `cuMemcpyHtoD` **runs at** ∼9–11 **MiB/s** — two independent rungs agree (≈ 9.25 and ≈ 10.8 MiB/s). *These differ by* ∼250×*; they are different planes.*

⊘⊘⊘ **And the most instructive result in the whole performance campaign is a correction to a favourable one.** A rung had reported that our guest was *faster* than natively pinned host memory at $N \geq 1024$. **The control was the anomaly.** Its one non-VRAM baseline measured **3.697 / 9.456 / 11.886 / 10.887 / 9.613 GB/s over five identical runs** — a 3.2× spread — while the three other placements reproduced to **0.2%**. Re-run against an honest baseline, "2.2× faster" became **1.37× slower** and "1.6× faster" became **2.76× slower**.

⇒ **A control is a measurement and needs its own** n **and its own scatter.** A baseline is usually reported as a single number precisely because nobody expects it to be the interesting one — which is exactly why an unstable one is invisible.

⊘ **Tokens per second: there is no number, and the ones in circulation are not measurements**

★★★ **The Rust port has never measured tokens/second.** $n = 0$**. No LLM has ever run in this guest** — the CUDA runtime now initialises, and the model still raises *"CUDA unknown error"* before any token (§8.5). The one figure that circulates, **0.04 tok/s**, is *arithmetic*: an [assumed] 250 launches/token multiplied by a per-launch floor that has since fallen by ∼20×. It is re-quoted in two design documents as though it were a fact. **Treat it as neither a measurement nor current.**

⊘ **And the C artifact's headline figure is not what it appears either.** *"20 → 60 tok/s"* is **the title of a plan**, not a result, and its 60 was borrowed from *Mode 1*, a different architecture with no doorbell trap. What the C actually measured with logs is **19.5–26.6 tok/s at** ±40% **run-to-run**. A separate bare-metal figure of **49.9 vs 47.5 tok/s host-native** exists and is credible, *but its commit is documentation-only — no log and no trace was committed*, unlike the matmul result beside it. **Treat 49.9 as plausible and uncorroborated; treat 20–38 as measured.**

★ **One external comparison was also refuted, and the refutation is the transferable part.** A sibling Mode-1 project's 0.82× eager-mode result was cited here as *"the external, measured bound on what a doorbell trap can ever cost"*. **That project has no doorbell interception anywhere in its codebase** and still loses 18%. ⇒ **exit count and per-exit work are independent axes.** The C's own slow case was root-caused to a *busy-polled register read* — one register accounting for ∼99% of all BAR0 reads, 1–3 k vmexits per token — which is a read, not a doorbell. *Our shape is the opposite*: few exits, expensive ones, and the expense is our own work.

8.5 ★★★★★ The LLM wall: a ladder of controls, a fatal PAIR, and a milestone graded on a proxy

★★★★★ ⊘⊘⊘ **CORRECTED 2026-09-11: THE WALL BROKE. THE LLM GENERATES, AND ITS OUTPUT IS CORRECT.** This whole section, and every bullet elsewhere in the paper that says *"no LLM has ever produced a token in this guest"*, is **false as of 2026-09-06 and decisively false as of 2026-09-09**. It is folded in above rather than rewritten, because the sequence of wrong answers below is the record of how it fell.

- [measured] **2026-09-06**, `w383llmgate`, real GA106, pre-registered arm: `LLM_OK=1 LLM_TOKENS=16 LLM_MS=722820.0`, `MINMM_SUM=64` intact, host `Xid` **0**. That is **45 s per token**, which is an optimisation problem rather than a wall. ⊘ **And the text was degenerate.** The milestone was graded on a *count*, and *a count cannot see a substitution* — the same species of proxy grade this paper already records twice.
- [measured] **2026-09-09**, rev `c8fe67d6`: `W392_OUTCOME=(P)`, **16 tokens whose text is byte-identical to the same-boot CPU oracle**, both `LLM_DEVICE=cuda` and `=cpu` printed with `RC=0`, host `Xid` **0**. **The content grade passes.**
- ★★★ **And the cause of the garbage text was ours, not NVIDIA's.** `reach.rs` carried `qualifies = … && !is_remap`, so a re-mapped joined row was never revoked and **the old binding kept translating: the guest was reading the previous tensor.** The guard itself had been a workaround for a release path that dropped bytes, fixed the same night — so an earlier fix is what made the later one safe.

⊘ **WHAT THIS DOES NOT ESTABLISH, and the scope is as important as the result.** **(a)** It is *not* a stable pass. The same workload fails under other arms, and the failure is now characterised as a **race rather than a missing mapping**: [measured] `w425`, the pin of the faulting range and the `FAULT_PDE ACCESS_TYPE_VIRT_WRITE` land in the **same second**. ⇒ *"we never backed it"* is false; *"we backed it too late"* is the live reading. **(b)** [measured] `w426`: the LLM boot issues **zero** `MAP_MEMORY_DMA`-family controls, so synchronisation point (2) cannot cover those rows however it is keyed — the rows are **discovered by our sweep, not announced by the guest** (§4.4). **(c)** It has **not been run under the three blocking constraints** at all: the milestone that established them says so in its own words, *"The LLM. Untouched since this landed."* **(d)** The grade remains a count in the harness; the project's own requirements document marks that **RED** and demands a deterministic content assertion at $n \geq 3$ boots. **16 tokens at 45 s/token on** $n = 1$**–2 boots is not a product.**

⚠ **Read everything below as the history of a wall that has since fallen**, and note that *this section's own prediction that it would be wrong by the time it was read has now been correct twice.*

This section was rewritten three times while this paper was being written, as five commits landed in one afternoon. That is not an apology — it is the measurement (§8.21), and each version was wrong in an instructive way. The sequence was: *"the cause is unknown and five hypotheses are refuted"* → *"the cause is one control we refuse"* → *"the wall is broken"* → **the version below, which is that the initialisation wall is genuinely broken and *the LLM still does not run*.**

⊘ **AND A FOURTH TIME, 2026-08-20 15:39 — three minutes after this document was pinned, and it is the one that changes what a reader should do with this section.** The version below ends by naming *the object graph* as the next wall. **That wall does not exist** (`129dd316`, `dbc6c953`), and the fold-in sits above it rather than beside it. ⚠ Read the whole of this section as [measured] at `3887be37`: **the LLM workload has not been re-run since**, and in particular

not across the multi-process fixes of §8.11. Its current status is [unverified], not known-failing.

The symptom as it stood, and why every conventional instrument said "healthy"

In one guest process: `libcuda` — the CUDA **driver** API — answered every call correctly (`cuInit` = 0, device count 1, name correct, 12 540 182 528 bytes, compute capability 8.6, a primary context retained, managed allocation and `cuCtxSynchronize` = 0). `libcudart` — the **runtime** API — returned **3**, `cudaErrorInitializationError`, from its first call. Host `Xid` **0**; guest `NVRM` line count identical to a fully green boot; **every** `ioctl(2)` **returned 0 on both sides.**

⊘ `strace` **is structurally blind to this entire class. RM returns its verdict *inside* the parameter struct**, not in the syscall's return value: a control that fails with `NV_ERR_NOT_SUPPORTED` is an `ioctl` that returned **0**. The standard reflex — *"strace it and find the failing syscall"* — yields a clean trace over a broken system. The project's name for this is *"*`failed=0` *is not* nothing refused*"*, and it cost several rungs.

★★ The instrument, and the four ways it was wrong before it worked

The oracle is a **host↔guest differential**: the same one-call program, recorded through the same `LD_PRELOAD` shim, on the bare-metal GA106 and inside our guest on the same box in the same hour, capturing the parameter buffer on **both sides** of every `ioctl`. Traces at `traces/cudart_ga106/`. Host **105** records, guest **104**; 98 identical; **2** status divergences; **27** body divergences at equal status.

Four instrument defects, each of which produced a confident wrong answer first, listed because they are the transferable part:

1. **The first differ compared** `(dev, nr, cmd, status)` **and threw away the bodies** — *the only place the defect could have been*. The recorder had captured them all along.
2. **A per-id refusal bisection reported all six candidate controls innocent, and was right about every one of them**, because it can only ever find *individually* sufficient causes. **It is structurally incapable of finding a fatal *set*.**
3. **A word count taken from the hex string length inspected exactly half of every body**, silently under-reporting non-zero words. Take it from `paramsSize`.
4. **Two silent truncations** — a rule array capped at 64 entries and, separately, a parse buffer capped at 1024 bytes against a 3158-byte rule set — made the substitution experiment report a confident answer derived from an arbitrary nine-rule subset. The shim now prints rules *parsed* and rules *applied* as two numbers. *One count cannot detect its own truncation; two can.*

★ **And the substitution injector was proven before use**, which is what made its negatives mean anything: corrupting one reply word of `0x20809009` from `0xd` to `0` turns the working host from **0** into **3**. So a wrong value at `status = 0` *can* kill `cudart`, and the shim *can* produce one.

★★★★★ The answer: it is a ladder of controls, and the minimal fatal set is a PAIR

Two findings, and each one invalidated the experiment that had preceded it.

(1) The wall is a ladder, not a control. Serve one, the guest advances one step, and the *next* refusal becomes the wall: `0x2080a084` → `0x2080a026` → `0x2080a001` → `0x2080200b`. ⊘ **Fixing these one per boot is the wrong shape**, and four boots were spent discovering that.

(2) The minimal fatal set is a *pair*, and no one-at-a-time experiment can find it. The full subset lattice over three of them, measured on the bare-metal host by refusing each subset in band:

refused subset	host verdict	refused subset	host verdict
∅	0	`a084`, `a026`	0
`a084`	0	`a084`, `200b`	0
`a026`	0	`a026`, `200b`	**3** ← **minimal**
`200b`	0	`a084`, `a026`, `200b`	3

⇒ **Three individually-innocent controls, one fatal pair.** That is why the per-id bisection was right six times and useless, and it is why the lattice — not the bisection — is the instrument. ★★ **It also produced the real deliverable: a zero-boot reproduction.** Refusing `{a026, 200b}` on a bare-metal host reproduces the guest's exact failure *in seconds, with no VM*.

★★★★★ **And one member of the ladder was invisible to every experiment driven by the reference capture.** `0x2080a001` appears **zero times** in an unmodified host trace — **a healthy host never needs it.** It is the *fallback* the guest takes only after the two above it are refused. So neither the refusal bisection nor the 136-word substitution sweep could ever have covered it; both were sound and both were *structurally incapable*.

⇒ A control that exists only inside the failure mode is invisible to a reference capture taken outside it. To see it you must drive the reference INTO the failure and re-capture.

★ That also dissolved a residue that had been sitting unexplained in the differential: **105 records versus 104 was never a missing call. It is a different branch.**

★★★ The initialisation wall broke — and the milestone did not

[measured] 2026-08-20, boot `w354cud1`, real GA106. The pre-registered falsifier was **met**:

`RT_cudaGetDeviceCount = 0` `RT_cudaSetDevice = 0` `RT_cudaMalloc = 0` `RT_cudaDeviceSync = 0`
`TORCH_INIT_OK` `TORCH_AVAIL = True` — *False for the whole campaign until then*

★★★ **And then the milestone test said no, and the correction is the most useful thing in this section.** [measured] the same afternoon, boot `w355llm1`, running the *actual model* rather than the probe:

`TORCH_CUDA_AVAILABLE = True` `TORCH_DEV_COUNT = 1` `nvidia-smi` prints a real header
`torch._C._cuda_init()` → `RuntimeError: CUDA unknown error`
`LLM_OK=0` `LLM_TOKENS=0` `LLM_RC=1` `LLM_OUTCOME=FAILED_rc1`

⊘ **The milestone had been graded on a PROXY.** `is_available()` calls `cudaGetDeviceCount` — the exact thing the probe measured — while the milestone is *tokens*. **They disagreed, and the proxy was the optimistic one.** The project has paid for this lesson before, under a different name: an earlier rung declared "first compute" on a copy-engine round trip that was not compute.

What is true, stated at the width the evidence supports: the `cudart` *initialisation* wall is fixed, now confirmed $n = 2$ **across two different workloads**; **the failure moved one rung**, from `cudaErrorInitializationError` to *"CUDA unknown error"* inside torch's full CUDA init, which walks the primary-context path; and the control-serving plane is genuinely clear — the refusal counter on that path fired **zero** times.

⊘⊘⊘ REFUTED: "the next wall is the object graph" — there is no object-graph wall

⊘⊘⊘ **EVERYTHING BELOW THIS BOX IS REFUTED, AND IT WAS REFUTED THREE MINUTES AFTER THIS DOCUMENT WAS PINNED.** It is kept, not deleted, because the *way* it was wrong is the section's most useful content. [measured] `129dd316`, 2026-08-20: the same refusal census was run against the boot that **passes** — `w354cud1` (`cudart` green) versus `w355llm1` (LLM failing), **same revision** `3887be37`**, same binary**:

refusal	passes	fails
`PromoteFault::UnknownContextObject`	4	3
`RmGraphError::FreeUnknown`	14	10
`BridgeRefusal::UnmappedAllocClass`	6	4
`wrong-kind(Device)` census lines	2	2

★★★★★ **Every element of the "wall" is MORE NUMEROUS in the boot that works.** It is baseline. **There is no object-graph wall**, and the failing run was never the experiment — the *difference* is.

⊘ **And the sharpest line was the most wrong.** `tsg = mid-miss(h0xa, wrong-kind(Device))` is **not a defect** (`dbc6c953`): a channel's parent is legitimately either a TSG or a Device — grouped channels have the first, bare channels the second — and the route that does not apply says so by name. The same line appears *byte-identical* in a boot from 2026-08-09 with the same count. **"The sharpest line on the board" was an eleven-day-old baseline line.** Two of the three refusals were separately already ruled: `PromoteFault` is the guest kernel RM's *RC watchdog* channel, refused deliberately since 2026-08-09; `FreeUnknown` is faithful RM behaviour, decided 2026-07-28.

★ **The process failure is the transferable part, and it is this project's own written rule.** `memory/a_refusal_needs_a_negative_control.md` says *"the failing run is not the experiment; the DIFFERENCE is"* — and the census below was read as a wall **without running the control, which was two log files already on the bench and one** `grep` **away**. ⇒ **Before naming any refusal a wall, run the control. Mechanically, first, always.** ⚠ A second-order cost: the refuted claim lived in a *commit message*, and a research agent later returned it as its top-ranked candidate *with a hash attached to lend it weight*. Speculation in a commit message becomes somebody's premise.

⇒ **What replaces it: nothing yet.** What fails inside `_cuda_init()`'s ∼479-ioctl primary-context graph is **unidentified**. Eliminated by measurement so far: the reply-body plane, the C artifact's CE-caps precedent (both of its controls are never issued here), `pdb=N` (by design since 2026-08-09), `wrong-kind` (a route that legitimately does not apply), and now the entire refusal census. The recorded method that has worked twice is to trace host versus guest at the failing layer and **diff the bodies** — not to read a census.

The refuted text follows, verbatim.

`Xid` is **0** and guest `NVRM` sits at the green baseline, so this is a *refusal*, not a fault. The boot names it:

refusal	count
`PromoteFault::UnknownContextObject`	3
`RmGraphError::FreeUnknown`	10
`BridgeRefusal::UnmappedAllocClass (0x70, 0x208f)`	4

★ **And one line is sharper than the counts.** The census for the first refusal reads `tsg = mid-miss(h0xa, wrong-kind(Device))` **— a handle that resolves, but to the wrong kind of object.** That is a named, specific defect in the context-promotion path, and it is *not* the class of thing the previous six rungs were about. *The campaign has moved from "which control does the driver want" to "is our object model right", which is a different kind of problem and squarely inside our own code.*

End of the refuted text. ⊘ **The caveats below are about the** `cudart` **control ladder — the fix that *did* land — and they stand.**

⊘ **Four caveats on the fix, none of them small, all from the rung that landed it.** (**1**) $n = 1$ on the `cudart` green, against this project's own measured rule that a single-boot green is wrong **1 time in 5**. (**2**) The fix is **capture-derived and therefore expiring**: it is pinned to GA106 and 580.159.04, and two of its rows were captured from **a host driven into the failure state**, because a healthy host never asks those controls. (**3**) Two members of the ladder are *still refused* — correct per the lattice, since breaking either member of the fatal pair suffices, but narrower than *"serve what* `cudart` *asks"*. (**4**) One row had to be **spliced rather than substituted**: the guest's request carries real content, so returning a constant body would have clobbered the guest's own words — the under-filled-body defect pointing the other way. The implementation keeps the guest's buffer and overwrites exactly the nine words a real GA106 overwrites.

8.6 ★★ A trace of a boot that SUCCEEDS — and the ladder, enumerated

⚠ **Scope note, first, because it is a claim about where code lives.** The recorder and the demand lists described here are on branch `replay-conformance`; they are **not on** `master`, which is where this document lives. Everything below is real, committed and dated, and a reader checking out `master` will not find the paths. This paper would rather say that than either omit the work or cite a path that resolves to nothing.

Until now every trace this project owned was of a boot that **fails** — the C's `cap1` ends where the C's emulator stopped, and our own boots end where ours does. A trace of a failing boot has an ambiguity that cannot be argued away: **every absent control is indistinguishable between "the driver never wants it" and "the driver died before asking."**

A recorder now exists inside CPU-RM itself. Two capture points in NVIDIA's own `message_queue_cpu.c` — one per direction — recording the *complete* message-queue element, header and body, into a `vmalloc` ring drained through `/proc`. **Two call sites. That is the whole instrumentation surface.** Its one invariant is stated as such: *a record has its bytes, or it does not exist* — fill-and-stop, never overwrite, and a non-zero drop count makes the trace unreplayable and the decoder rejects it.

What it captured, 2026-08-03, on a real GA106: 1 076 records (535 out, 541 in), 1.17 MiB of a 64 MiB ring, **zero dropped**, 14 distinct RPC functions, 620 control elements resolving to 310 request/reply pairs and **104**

distinct controls — and, the line that matters most, **replies declaring params with no bytes present: 0**. Two independent GSP bring-ups in the trace (sequence numbers restart), with *zero* retransmits inside either, which makes it a repeatability check as well as a capture. **The boot succeeds**: `nvidia-smi` reports the card, and the capture script fails if it does not.

★ **Six of the C oracle's empty rows now have bodies**, including the one that cost four rungs: `0x20802a08` answers 20 480. *A genuinely zero-length reply is now distinguishable from an unmeasured one* — which is the whole of §3.1's fifth limit, repaired at the source rather than worked around.

The ladder, enumerated rather than discovered one boot at a time. A successful boot demands **104** distinct controls. The failing `cap1` capture reaches **53**. **53 are demanded by the successful boot and never asked for by** `cap1`, and two go the other way. ⚠ *The two 53s are a coincidence of this capture and the source doc goes out of its way to deny a relationship between them*; this paper repeats the denial. And the ladder's *serve* obligation is at most **93**, not 104, for the reason in the next paragraph.

Each control is bucketed **DATA** (the reply is the whole answer; servable from a table), **ACT** (params all `[in]`; the reply only acknowledges something that *happened*), **MIXED**, or **UNKNOWN** — statically, out of the driver source, with a citation to both the params struct and the code that decides direction. **81 of 106 (76.4%) are bucketed with a citation**; 79 are DATA or ACT; **25 are UNKNOWN**. ⊘ *The classification is a reading, not a measurement*, and the docs keep the two in different sentences.

★ **The 25 unknowns are not a tail — they are one wall.** Three are a binary-API class with no export, no flags and no struct on the CPU side. The other 22 are GSS-legacy commands that RM forwards verbatim to GSP with a comment saying, in NVIDIA's words, that it no longer supports them and is forwarding them so GSP can decide. **None of the 22 appears in either vendored driver tree**; the versions do not disagree, they are both silent. *The ladder is not 53 hard problems; it is 31 easy ones and one closed-firmware wall.*

> ★★★ **And the finding that bears hardest on our own design: a real GSP refuses 13 controls on the boot that works.** Eleven come back `NV_ERR_NOT_SUPPORTED` unconditionally; two are conditional. `NV_ERR_NOT_SUPPORTED` **is** `0x56` **— the same status this port emits when nobody claims a command.**
>
> **And CPU-RM anticipates 7 of the 7 it can name.** Six are on a list RM keeps of commands whose unsupported status it declines even to *log*; the seventh has an explicit tolerance arm at its call site that converts the refusal into a normal branch. The four it does not anticipate are exactly the four GSS-legacy ones, **which the source cannot name**. ⊘ Scope, honestly: the list RM keeps is written about a vGPU guest, so it evidences that RM has a named class of legitimately-unsupported controls containing these ids — not by itself a statement about the bare-metal path. **What makes the pairing strong is that the refusal was [measured] on a real GA106 while the anticipation was [inferred] from source. Two instruments, one answer, neither doing the other's job.**
>
> ⇒ **Refusal is ordinary protocol behaviour, not merely our posture.** That is a real strengthening of this project's refusal-first design: for eleven controls, refusing is demonstrably what the real device does on a boot that then works.

⊘ **But refusal is not a free pass, and the correction here is worth more than the rule.** An earlier document concluded from the two conditional refusals that *"a static policy cannot express it."* **That was retracted as too strong.** A policy keyed on the *command id* cannot; a policy keyed on the *params* expresses one of them exactly — the status is a deterministic function of the argument, re-measured identically across both independent bring-ups in the trace — and is the only shape that could ever express the other. **The surviving distinction is cmd-keyed versus params-keyed, not static versus dynamic.** One row cannot be served in any form: two identical requests return different live PCIe byte counters.

And two facts a replay needs are independent. A control can be **DATA and still be refused by hardware**: two controls bucketed DATA off a clean reading of the driver get `NV_ERR_NOT_SUPPORTED` from a real GA106 on a boot that then works. *The bucket says what the reply means; it never said the reply exists.*

What the recorder cannot see, stated by its own author. It answers *what a well-behaved boot demands*, and nothing else. It is a strong *positive* oracle and a weak negative one — it says nothing about what the driver does when refused, when something arrives out of order, or late, so our refusal-first cases must be *authored*, not recorded. It covers one part, one kernel, one driver, and only the open driver — the closed one cannot be instrumented and the correspondence is [assumed]. Nothing in `crates/` reads the format. And ⊘ **observational neutrality is not proven**: the hooks add a `memcpy` of up to 64 KiB under a spinlock on the RPC path, the 88/88 cross-validation is against a *differently instrumented* build of the same driver, and **no measurement in that task establishes what an uninstrumented driver issues**. ⚠ For the same reason this paper does not

call it a stock-driver capture: it is real GA106 silicon and real GSP firmware, with the CPU-side driver rebuilt from stock *source* plus two hooks.

8.7 ★ The bridge, and the two NVIDIA findings that survived being right

This section has been the paper's headline twice and been falsified twice. Revision two: *"the critical path is the bridge, not the crate."* Revision four: *"the bridge exists and it does* not *unlock compute."* **Compute is unlocked** (§8.3), so the second is retired with the first.

★ **What is worth keeping is the part that was a claim about NVIDIA rather than about our schedule**, and both such claims held up. They are stated below unchanged, because *they are the reason the address plane had to be built out of three unusual sources instead of the obvious one* — and a reader who wants to know why this system's address table looks the way it does will find the answer here rather than in the architecture section. If you read one supporting document, read §2.7 of `docs/design/gsp_core_bridge.md`.

What was built

`translate(&DriverAbiTable, &RpcCommand)` is a stateless free function; `GraphPolicy` is the `CommandPolicy` the boot FSM calls, doing translate → `Gpu::apply` → reply; `kayfabe-abi` gained the per-class alloc table and four params decoders; and the control arm (function 76) decodes what it can model and **refuses the rest by name**. The refusal set is fifteen variants wide and every one of them carries the number that caused it, because the census is the stage's real output: it says which decoder is missing, not merely that one is.

★★ `MAP_MEMORY_DMA` is not on the wire, so the bridge cannot deliver compute

This is the finding to carry away, and it is a roadmap fact rather than a code fact.

> On a GSP-client GA106, `rpcMapMemoryDma` (function 14) and `rpcUnmapMemoryDma` (function 15) are `_STUB` **through the entire HAL chain** — `_GA106` → `_GA102` → `_GA100` — and the stub returns `RPC_UNKNOWN_FUNCTION`. The real implementation is installed only by the *vGPU* IP-version table. **So** `RmEvent::MapMemoryDma` **has no producer on this path, and no stage of this bridge builds an** `NVOS46` **decoder.**

Three independent oracles agree, which is why this is stated flatly: the open kernel modules' own generated HAL tables; the C artifact, which booted a real GA106 guest to CUDA and has **no function-14 hook at all** in its complete snoop set (47/76/21/103/10/65/70/72/73); and two C-era design documents that say it in words — *"there is no per-map GSP-RPC carrying VA↔phys"*.

★★ **The consequence, and it is the one prediction in this paper that was both correct and acted on.** If no RPC carries VA↔phys, then the address table cannot be populated from the object model at all, and *finishing the bridge would leave the address plane empty*. That is what the previous revision said, and it was right: the compute path was never one bridge stage away, it was a different plane. **So the different plane was built**, out of the three sources §8.3 enumerates — the `GPU_PROMOTE_CTX` snoop, the doorbell-time GR page-table sweep, and the CE page-table write latched at the release semaphore. None of them is an `RmEvent`; `GPU_PROMOTE_CTX` in particular declares a *set* of `(gpuPhysAddr, gpuVirtAddr, size, bufferId)` entries, which is an address-table population rather than an object-model edge, and it was deliberately kept out of this seam for that reason. *The seam discipline held: a fact that is not object-model shaped did not get forced through the object-model door.*

★ `SET_PAGE_DIRECTORY` is not how an ordinary address space declares its root

B4 was built to decode the control that carries a page-directory base. It settled the question *against* the design that asked it.

On a bare-metal GSP client, `SET_PAGE_DIRECTORY` reaches the wire **only** for an address space marked `SHARED_MANAGEMENT` or `IS_EXTERNALLY_OWNED` — `gvaspaceExternalRootDirCommit_IMPL` asserts on exactly that, and its only callers are the UVM / gpu-ops path. Every *ordinary* RM-managed VAS declares its root through `NV90F1_CTRL_CMD_VASPACE_COPY_SERVER_RESERVED_PDES` (`0x90f10106`) at construct time, as `levels[0].physAddress`, on a path that is on by default for any GSP client. There is a third: `NV2080_CTRL_CMD_INTERNAL_GMMU_COPY_RESERVED_SPLIT_GVASPACE_PDES_TO_SERVER`.

The design's arm was therefore **necessary and not sufficient**. All three commands are refused as a *distinct* variant, `PageDirControlNotModelled{cmd}`, rather than folded into the generic unknown-control refusal — and the reason is the sharpest small argument in the bridge: **a dropped page-directory declaration is invisible**. The VAS never routes, every channel defers at its first doorbell forever, and nothing anywhere says why. A separate refusal variant converts a silent hang into a counted, named record of exactly which decoder the address plane is waiting on.

Two smaller results from the same arm are worth keeping because they are the kind of thing a version table does not catch. `hVASpace == 0` is **not** "unspecified": NVIDIA's own header says verbatim, in both vendored trees, that zero means *the implicit VA space associated with the client/device pair* — a real object the RPC does not name and the graph has no node for, so passing it through would park a PDB on a node the guest never declared. It is refused. And note the deliberate asymmetry: on the *alloc* arm, a zero handle means "nothing declared" and maps to `None`. Same zero, opposite meaning, documented differently by NVIDIA — so neither may be inferred from the other.

Where the design met the compiler

Roughly two dozen items are recorded across the four commit messages, and the pattern in them is more useful than any single one: the design's *taxonomy* claims survived and its *buildability* claims did not. "Known-and-inert versus unknown, there is no third state" was contradicted by the first stage's own row — there are at least four states. Three separate non-vacuity arms the test plan specified turned out to be unbuildable at the stage that specified them, one of them because it was also *wrong*: refusing a second, different PDB for one VASpace requires remembering the first, which a stateless bridge cannot do and which `RmGraph` had already decided the other way, with an argument (re-binding is protocol-legal; refusing would hang a conforming guest).

One item is a finding the design does not have and could not have had: **functions 72 and 73 never reach the policy at all.** The FSM returns on a no-reply disposition *before* calling `respond`, so those arms are unreachable through this adapter. Harmless today, because neither carries object-model content — and load-bearing to know, because a future no-reply function that *did* carry a fact would be dropped silently.

8.8 ★★ A live-memory-free defect — and the independent oracle had the identical bug

This is the most serious defect the project has found in its own core, and the part that matters is not the defect.

The defect, one line wide. `RmGraph::apply` drained its three parked-fact tables from the `Alloc` arm only, on the reading that a parked fact waits for its target's *allocation*. It waits for its target's **handle** — and `Alloc` and `Dup` are the two events that mint one. So a dup-of-a-dup whose middle handle was minted by a `Dup` stayed parked *forever* while being fully resolvable.

The measured consequence. `project()` follows parked dups through `origin_of`, so the process grouping saw the alias; the refcount never did. The undercount takes references to zero, the resource is destroyed, the component vanishes, the process is vacated — and **the reaper emits `Free` for memory a live kernel alias still references.** That is the project's own "never free host memory RM says is live" rule, reached through a door nobody was watching. It was three defects, not one: the same missing drain also left `Resource::pdb` unset for a parked page-directory declaration and never installed a parked mapping, giving a fault on a *legal* guest.

> ★★ **And the independent oracle carried the identical bug.** The property suite's `RefTracker` — the deliberately separate reference-count model whose entire purpose is to disagree with the implementation — also promoted parked facts only in its `Alloc` arm. Model and implementation were wrong *the same way*, so the property stayed green over a three-defect bug for as long as both existed. The correction is proven to be a strengthening rather than an assertion: with the model fixed and the implementation's drain removed again, the property fails on its own.

Why this is the most instructive entry in the paper. An independent oracle is this project's strongest single testing idea — it is what `kayfabe-abi`'s three transcriptions are, what `gspworld` is, and what the bridge's hand-hex fixtures are. This is the failure mode of that idea, in its purest form: *independently written* is not *independently reasoned*. The same author, holding the same wrong sentence about what a parked fact waits for, wrote it into both. Nothing about the shape of the test caught it; what caught it was a different invariant, derived from the other side of the graph, disagreeing.

8.9 ★★ The quality instruments were wrong — three of them, in one day

This is the real theme of the work since the last pin. The code got better; what got *much* better is the confidence one is entitled to have in the instruments that say so. Each of the three below was a live, silent, one-directional error in the machinery this document quotes.

1. The mutation configuration was never read. `mutants.toml` sat at the workspace root. cargo-mutants reads `.cargo/mutants.toml`. **The file was never opened, and every exclusion argued inside it was inert** — including the exclusion of `kayfabe-mocks` and `tests/src`, which is the one this paper has repeatedly relied on when saying "excluding scaffolding".

It was proven rather than inferred, which is the right standard for a claim of this shape: appending a bogus key to the root file changed nothing and produced no parse error; appending the same key to `.cargo/mutants.toml` fails loudly. `cargo mutants --list` goes from **3 760** targets to **2 667**, and scaffolding mutants listed goes from **361** to **0**.

The consequence for every number this document has ever quoted. Every mutation figure recorded before 2026-07-28 was measured over a population that *included the instruments*, and the "≈94.9% excluding scaffolding" figure — which this paper printed at the previous revision — was a **hand computation**, not something the tool ever did. That is exactly why it read as a parenthetical rather than as the gate: it was never the gate.

2. The gate could report 100% from a broken suite. cargo-mutants baselines with `--package=` naming only the mutated packages, while the mutants themselves run `--workspace`. Measured: the baseline runs about 200 tests across 39 binaries and **omits** `kayfabe-tests` **entirely** — the conformance suite that catches most mutants. A red conformance suite would therefore pass the baseline and then make *every* mutant look caught. Fixed with `needs: stable`, which makes the real suite a precondition of the gate rather than an assumption of it.

The same job's `--timeout 240` sat at or below honest completion (the workspace suite alone is 201 s on a 25-core box, and a 4-vCPU runner at `-j 2` is slower). **cargo-mutants scores a** `TIMEOUT` **as *killed***, so mass timeouts would have reported near-100% and the gate would never have fired. Raised to an absolute 1 800 s — and deliberately *not* to a `--timeout-multiplier`, because the multiplier is computed off the same baseline that omits the suite, and would reintroduce the identical inflation through a different door.

3. Thirteen test park-points hung forever instead of failing, producing zero output. The shape is a `thread::scope` whose cleanup is reached only by statements *after* the assertions: any failed assert unwinds past them, and the scope blocks forever joining threads that were never told to stop.

> **It had already happened, for twenty-three hours.** Three wedged test binaries were found alive on the box, three threads each, all parked in `futex_do_wait`, at ∼83 000 s. And the failure mode is worse than "buried": under CI-realistic output capture **the panic message is never emitted at all** — the log stops at `running 1 test` and stays empty forever. One site hung *spinning*, burning a full core, and outlived the `cargo` process that spawned it.

Every one of the thirteen was **proven vulnerable before and proven fixed after by inducing a real failure at each site**, not by reading the code — which is the same bite-check discipline the project applies to its assertions, applied to its harness. The fixes are `Drop` guards armed as the first statement inside each scope closure, plus a barrier type that panics instead of blocking when its rendezvous breaks. Each guard carries an in-comment R1 argument naming its callee, because *a* `Drop` *is a call site* and R1 does not guard `Drop`.

And a fourth, smaller, which is the same species. The test count itself was wrong: 551 came from **stale prebuilt binaries** in `target/`. Two test files had 16 and 32 `#[test]`s at HEAD while the inherited cache ran 13 and 24. A related trap bit twice in one day inside the bite harnesses: `cp -a` and `shutil.move` both preserve mtimes, so a restored file keeps the previous mutation's compiled artifact and a whole bite table comes back *inflated*. Any harness that restores files must `touch` them.

> **What these four have in common, and why they belong in a whitepaper rather than a changelog.** Every one of them made the project look *better* than it was, and every one was invisible from the output. An inert config, an omitted baseline, a timeout scored as a kill, a hang that prints nothing, and a stale artifact are all the same failure: **the instrument reported a number that no longer described a measurement**. This is the project's own founding doctrine — *"a green instrument on an unexercised path is worse than no instrument"* — turned around and pointed at

the instruments themselves, and it is the strongest available evidence that the doctrine is being applied rather than quoted.

8.10 `kayfabe-gsp`: boot is modelled, reboot is modelled and never exercised, and no teardown has ever been run

⊘⊘⊘ **SUPERSEDED IN ITS TITLE, 2026-08-21 — read §8.12 first.** This subsection is called *"reboot is modelled and never exercised"*. **It has now been exercised** — eight consecutive device opens in one boot, driven by nothing more exotic than `nvidia-smi` — **and it broke on the fifth**, permanently, from a message-queue sequence carried across a driver life. The prediction below that this was the untested seam was *correct*; what it could not say is that the first workload to touch it would find a defect in the first five seconds. ★ The sharper form of the warning stands and is now evidenced: **"modelled" and "exercised" are different words, and the gap between them was hiding a bug that makes the product unusable after five invocations of its simplest command.**

Two revisions ago this section said "34 lines, and it is the critical path". One revision ago it said "3 550 lines, and nothing calls it". Both are now superseded: the crate is **5 497** lines across 8 modules, it has *six* production consumers, a stock guest driver boots against it to a working `nvidia-smi` and then computes, and the one invariant this paper flagged as prose-only — GSP-S1 — has been paid off in code.

What has changed below, and it is the subsection title. Three of the plan's nine stages are still unbuilt and all three need a GPU. **But "reboot is not modelled" was too strong** — the boot FSM carries suspend and reload states and transitions. ★ **What is true is sharper and worse**: they have **never been exercised against a real guest driver reload**, and the core has **no reset entry point of any kind**, so on a guest `modprobe -r` the graph, every live process with its sandboxes, arenas and host handles, both routing maps and the guest-physical window all survive into the next driver's life with nothing reclaimed. ⊘ The project's own words: this limitation is *"inherited here by omission rather than by decision, which is worse, because nothing names it."* **The item that used to occupy this slot — §11-O7a — is resolved** (below). And the plan the crate was built from was written against the wrong driver version, a fact that keeps producing new consequences rather than being absorbed once.

What is built, and what "built" is worth here

The five seams the crate claims — GPU architecture, driver version, hypervisor, GPU lifetime, CPU architecture — each have a test that pushes on them, which is the difference between a seam and a decoration. Two are worth singling out because they are cheap to fake and were not faked. The lifetime seam is exercised by running *repeated* driver lives in one process: the C artifact's measured second-life failure (`msgqRxLink` timeout, `-7`, 71 064 retries) has exactly one cause in the driver's own source, `rx.size != size`, and the FSM is shaped so that cause cannot arise. The architecture seam is exercised by driving the same FSM through *two* deliberately-different register models, which is the same argument as `MockArch` one layer down.

The oracle situation has changed completely since the previous revision, and it is the one unambiguously good piece of news in this section. That revision said: *"Those traces do not exist: the capture patch is ∼60 lines against a C file this work does not own, and it has not landed."* **It landed.** Five captures exist, two are committed into this repository decompressed, and `kayfabe-crec` replays them against the real FSM in the default suite. ⊘ **And no replay has been closed** — see §3.1: `cap1` stopped on a missing observation at transaction 978, `cap1b` carries it to 1028, and what stops it there is a plane the C's recorder cannot witness at all. *"Byte-equivalent with the C" is therefore established up to a stated transaction and not beyond it*, which is a far better position than untested and is not the same as tested.

Boot is modelled. Reboot and resume are not.

The plan has nine stages. S0–S5 — the six that need no GPU — are the boot and steady-state protocol, and they are built. **S6, S7 and S8 are the three that need a GPU, and none is built.** The bench-offline blocker that also held S6 is gone (§8.20), so what remains is work rather than an obstacle — but it has not been done, and the two open questions behind it (**O3**, whether sequence numbers reset across a true `rmmod`/`insmod`; **O5**, which boot mode a post-teardown re-acquire uses) are still answerable *only* by a bench run. This matters more than the stage count suggests, because the owner's standing requirement is that a GPU restart — idle, released, re-acquired — must work *with no bolt-on*, and restart is exactly the region S6–S8 covers. **And a boot that dies in `RmInitAdapter` never reaches a teardown**, so nothing on the bench exercises this region even incidentally. Boot is the part that could be modelled offline, so boot is the part that got modelled.

⊘ RESOLVED: the "sharpest open item" was closed the day after this paper last called it unresolvable

★★★ This subsection is kept, struck, because the correction is worth more than the item. The previous revision printed §11-O7a as *"the one item in the whole component with no proposed resolution, because the evidence that would settle it is not in any open-source tree"*, and listed it in its risk register as the one thing that *"is not yet a known quantity."* **It was resolved on 2026-08-06** — before three of the four subsequent whitepaper revisions were written — and this paper carried it forward as open for two more.

The resolution, on both driver versions. CPU-assist events are `emit-never`, and the version split is *not* "RPC versus local": on **both** 580 and 610 every route to `CORE_RESUME` begins with a *GSP→CPU* event and the CPU driver never starts one itself. Only which event carries it differs. The opcode family exists so the GSP can borrow the CPU **while real silicon is down**, and an emulated GSP has no down-silicon phase; nothing on the driver side waits for one, because the boot wait is `rpcRecvPoll(GSP_INIT_DONE)`. [measured]: the C artifact contains *zero* occurrences of the sequencer and a stock 580.159.04 driver nonetheless completed cold boot → `cuCtxCreate` → matmul → teardown → GSP reload.

★★ And the premise underneath it — "the evidence is in closed firmware" — is dead too. [measured] 2026-08-06 from a committed trace of a real GA106: a real GSP sends **exactly one** `RUN_CPU_SEQUENCER` per bring-up, `rpc_len = 6328`, roughly 200 ms *before* `GSP_INIT_DONE`. It is ordinary cold boot and the buffer is byte-decodable. ⊘ Replay is still refused, on a different and better argument: it would be *a too-capable double*, obliging us to emulate falcon halt bits, SEC2 start, a scratch-register handoff, a mailbox and ∼104 register polls each with its own timeout — binding the port to one chip's boot script to serve a round trip whose purpose (reviving real silicon) has no referent here.

⚠ **What actually remains open is smaller and differently shaped**, and it is the residual risk `emit-never` names for itself: *a handler side effect we skip that something later reads*. Two candidates are named rather than assumed away — an `appVersion` write and the mailbox stores the GSP reads back after `CORE_RESUME`. Both *should* be vacuous because we own producer and consumer, **so the stage that lands must assert it rather than assume it.** That is a test obligation, not an unanswerable question.

The original item, struck, for the reader who wants to see the shape of the mistake. It read: this is the one item with **no proposed resolution**, because the evidence that would settle it is not in any open-source tree.

The mechanism is present at both driver versions and reads identically: a core resume resets into RISC-V, reprograms the LibOS boot-args address, starts SEC2 and waits on `NV_PGC6_BSI_SECURE_SCRATCH_14._BOOT_STAGE_3_HANDOFF`. What differs is **who initiates it**:

	580.159.04 (the bench)	**610.43.02 (the vendored spec)**
where the wait lives	inside the CPU sequencer executor, as opcode `CORE_RESUME`	promoted to a first-class HAL, `kgspExecuteCoreResume_TU102`
how it is reached	**only** through `_kgspRpcRunCpuSequencer` — i.e. only if *we send* a `GSP_RUN_CPU_SEQUENCER` event	locally, from two call sites in the driver. **Nothing is required of us.**

Three things follow, and none of them is comfortable.

It is not a layout difference, so a version table does not absorb it. Every other 580-vs-610 divergence the project has found is an offset, a field count or a constant — one row in a version-keyed table. This one is a difference in *who initiates*, which means a faked GSP that must emit sequencer buffers at 580 and must not at 610 has version-conditional *behaviour*. `docs/design/four_axes_of_variation.md` §5 rule 1 forbids exactly that: *"No `if guest_version == …` in a logic crate. Transitions fire on observed protocol facts, never on driver identity."* The two honest options named in the correction brief are a version-keyed *capability* on the FSM or a loud refusal, and the brief declines to choose between them on the grounds that choosing requires an answer it does not have.

The answer cannot be read out of the open source. What the open tree shows is that the path exists and how it is triggered. What it cannot show is whether the firmware ever triggers it on a path we must support, because the *contents* of a sequencer buffer come from GSP-RM firmware, which is not open. Specifying an emitter now would be guessing, and the plan says so rather than shipping a guess.

⊘ **And the conclusion that was drawn from it — struck.** It read: *"the requirement is genuinely at risk … 'GPU restart must work with no bolt-on' is not currently satisfiable by the design as written."* **The next step named was correct** — *"a reading task, not a hardware task"* — and doing that reading is what dissolved the item. ★ The general form is the one this paper keeps re-learning: **an item recorded as unresolvable because "the evidence is in closed firmware" is worth one more read before it is believed**, because the boundary between open and closed source is itself a claim, and this one was wrong.

One further sharpening worth carrying, because it cuts against the project's own instinct: the relayed finding that started this said "580 has a resume handoff 610 never reads". **That half was wrong** — 610 has the

identical routine reading the identical register. The correction log reports it as wrong rather than quietly fixing it, which is the behaviour a reviewer should be checking for.

★ The plan was written against the wrong driver, and one item was RESOLVED backwards

This is the most instructive thing that happened to this component, and it is not flattering.

The port plan took its NVIDIA citations from the vendored open kernel modules at **610.43.02**. **The bench runs 580.159.04.** The two do not merely differ in offsets; they differ behaviourally, and 3 550 lines were built against the un-caveated plan before anyone vendored the matching tree. Eight claims changed when `ogkm-580.159.04` was finally vendored (`7af23cb`): the element count is *read from a field* at 580 and *derived from a length* at 610; MCTP/NVDM transport headers do not exist at all on the 580 GSP path; the layout break interval narrowed from `(570, 610]` to `(595.84, 610.43.02]`; the init RPCs are queued *before* bootstrap at 580 rather than inside it; the suspend sentinel is an exact equality at 580 and a mask at 610; and the queue geometry is compile-time at 580 rather than negotiated, which removes the "the guest declares its own geometry" argument the plan had presented as its highest-leverage design choice.

> **★★ And at this revision the same seam bit inside a range the project claims to support, which is a strictly worse shape than "the plan cited the wrong tree".** `NV_CHANNEL_ALLOC_PARAMS` **diverges at offset +32**: 610 inserts `hHandleVASpace` there, where 580 — *the bench driver* — has `hUserdMemory[0]`. A generated 610 mirror would mis-read **every field from +32 onward** for the guest we actually run, `engineType` included, and would do so *plausibly* rather than loudly.
>
> The response is the one the four-axes rule implies and it costs something: **the struct is deliberately not generated at all**, only the 32-byte agreed prefix is decoded, and a test named `the_channel_alloc_prefix_stops_where_the_two_trees_stop_agreeing` asserts the *absence* of the generated struct so that nobody helpfully adds it later. What that costs is concrete: `engineType` lives past the prefix, and **a CUDA process's GR and CE channels are the same** `hClass` — so `Arch::classify` cannot answer which engine a channel is for, and a CE channel only becomes one later, via its engine object and a refinement pass. A version-keyed *table* does not absorb this; only a decoder that declines to decode does.

The worst instance is §11-O7, which had been marked RESOLVED with the answer that is backwards for our bench. It said `GSP_RUN_CPU_SEQUENCER` "is not implemented, do not emit it" — true at 610, where the executor was deleted; at 580 it is fully implemented, dispatched, and first on the bootup-window allowlist, so an emitted sequencer buffer would execute register writes and falcon resets inside the guest kernel rather than being harmlessly ignored. The RESOLVED status has been withdrawn.

The class, which is the part worth keeping. A *versioned* specification was cited as if it were *the* specification. The fix is mechanical and now normative: every citation carries its tag (`ogkm-580:` / `ogkm-610:`), an untagged claim is unverified by definition, and line numbers *and paths* never cross tags — the whole `msgq` library moved directories between the two.

★ That fix is being applied to new code and the backlog is getting worse, which is the honest and unflattering measurement. At the previous pin the crates carried **121** bare, untagged `ogkm:` citations and **zero** tagged ones. At this revision there are **56 tagged** citations — the rule is real and new work follows it — and the bare count has risen to **159**, because four crates' worth of new code was written faster than the backlog was converted. `kayfabe-gsp`'s own share fell slightly, 96 to 92. **The CI grep that would stop the class recurring is specified and still not written**, and it is now the only thing standing between a normative rule and a growing pile of counter-examples. The task is filed (correction-brief item B11) and was explicitly not reached.

The self-critical note in the correction log is the one to read: one claim in the same document was marked `[inferred]` and turned out to be right, while O7 was marked RESOLVED and turned out to be backwards — and the difference was not luck. *The inferred claim knew it was second-hand and the resolved one did not.*

What this does and does not change about the critical path

For three revisions this subsection said the gap had moved up one layer and had not closed. It has closed, and the answer was not what any of those revisions predicted. The L2 adapter exists, `GuestRam` has a production implementor, `GraphPolicy` is constructed outside `tests/`, and `DriverAbiTable::gsp_element` has a caller: the shipped register plane selects a wire layout to answer a real guest through. The version-dispatch machinery finally has something production-side asking it a question.

And the critical path turned out never to have been any of the layers this paper kept nominating. It was not the crate, then the bridge, then the L2 adapter. **It is the execution plane** (§8.2) — a plane no amount of protocol modelling reaches, because the guest driver's own acceptance test for it is a functional copy with a read-back comparison. *Read §8.7 with that in hand*: even a complete bridge fed by a complete adapter produces a correct object model and an address table that only one of its two sources fully populates.

★ **Worth noticing about this document rather than about the code.** Three consecutive revisions each identified "the critical path", each was invalidated within days by the work it named, and *each was invalidated in the same direction* — the named obstacle got built and was not the obstacle. That is a pattern in the paper's own reasoning, not in the project's execution: **a layer that is missing is easy to see, and a plane that cannot be faked is not a layer.** It is recorded here because the fourth answer above should be read with exactly that much suspicion.

8.11 ★★★★★ Sequential multi-process: the wall fell, three processes pass, and the fourth does not

⊘⊘ **FRAMING SUPERSEDED 2026-08-21 by §8.12: the ordinal is not over CUDA processes, it is over DEVICE OPENS.** A bare `nvidia-smi` — no CUDA, no context — both consumes the resource and dies of it, at #5 of 8. Every measurement below is still correct as measured; only the word *"process"* in the reasoning is wrong, and it kept the search inside CUDA for an evening. The framebuffer-join defect it names is real and was really fixed; it was simply not the whole of what this workload was hitting.

★★★★★ **This subsection was added after the rest of the paper was written, and it falsifies the paper's own headline blocker.** Everything in it is dated `w361`–`w367` and landed in the five hours *after* this document's pin (`5c367a38`, 15:36) and before its first reading (`56dc01f3`, 20:32). The claim it replaces — *"a second CUDA context sequentially in one boot still hangs"* — is left standing wherever it appeared, corrected in place rather than deleted, per this paper's own house style (§8.21).

What it was. [measured] boots `w361`, `w362`, `w363` (revs `3887be37` and `ecb67fae`), real GA106, internally controlled inside each boot: the *first* guest CUDA process rang 403 `GrCompute` + 104 `Ce` doorbells and got **8/8** completions; *every later one* rang exactly **8 + 8** and got **0/8**, then spun forever (`R` state, 100% CPU, unbounded at a 600 s step limit). ★ The effect was **ordinal, not predecessor-kind**: an `A2` arm — a second torch process with no `nvidia-smi` between — failed identically, which retracted this project's own earlier reading that a predecessor *poisoned* its successor.

The cause, named by our own refusal text. Three GR context frames — `0x400000` (`SET_VALID_SPAN_OVERFLOW_AREA`), `0x600000` (`SET_TEX_HEADER_POOL`), `0x800000` (`SET_TEX_SAMPLER_POOL`) — were refused 48 times each with *"that framebuffer range is already joined; installing a second backing over it would give one leaf two memories"*. The first process joined all three; the later ones got *zero* joins, so their GR context had no host backing, so nothing completed and `libcuda` waited forever. Everything upstream was identical between the working and failing processes and everything downstream was proportional to doorbell count; ⊘ **the vocabulary diff was empty both ways** — only the *count* diff could see this.

⊘ **And the takeover mechanism that already existed could not fix it *by construction*.** `supersede_joined_fb_leaf` routes by the *caller's own* `(gpu, pdb)` and searches only that address space, while the framebuffer store is per-*device* and keyed by physical address — so a join left by a different, exited process lives under another page-directory base and is unreachable to it. Arming it harder does nothing: 32 successful supersedes in the same boot and **zero** on these frames.

The two fixes.

1. **Orphan reclaim** (`4d88723c`, `w366`). [measured] `w365`: all 48 refusals of each of the three frames, arriving from *two independent* later processes, censused `live=0 retired=0`. **The framebuffer store held a join that no address table anywhere named** — refusing it refused on behalf of nobody. The fix gives the store's join back at the `SUPERSEDE NO-ROW` branch when neither a live nor a retired row names the frame, recomputed at the decision point rather than read from the diagnostic's cache. ★ **The cross-process boundary is kept, and it is measured**: `live > 0` stays refused *by name* — **132** times in the passing boot. A live peer's backing is never taken.
2. **Re-key the takeover cap** (`cca2eb4b`, `w367`). `SUPERSEDE_CAP_PER_FRAME = 4` was keyed by *frame* and reset by nothing, so a device-lifetime budget spent by early actors left later leaves *fabricated* — with no host backing behind them. **Our own bound then raised a host** `Xid`: `w366` recorded `Xid 31 FAULT_PDE` on `CE2` at `0x7e59_c6000000`, traced to `fb_phys=0x1e00000`, which appears `CAPPED` nine times in our own log. Re-keyed by `(phys, taking va)`, a ping-pong between one pair of aliases is still bounded — at 2× CAP for that pair — while a genuinely new owner arriving later gets its own budget.

The result. [measured] `w367`, rev `cca2eb4b`, real GA106, 600 s step bounds, one boot:

arm	what it is	at `w363`	at `w367`
`A`	first torch process	`TORCH_OK`	`TORCH_OK`
`A2`	second, *no* `nvidia-smi` between	`TORCH_HUNG` at 600 s	`TORCH_OK`
`C`	torch after `nvidia-smi` ran and exited	`TORCH_HUNG` at 600 s	`TORCH_OK`
`D`	a *fourth* process	`TORCH_FAIL`	`TORCH_FAIL`

`avail=True sum=4.0` on all three passing arms; completions **8/8 OBSERVED** for `proc=2`, `proc=4` and `proc=7`; doorbells **104** `Ce` **+ 403** `GrCompute`**, identically for all three**; the three frames at `refused=0 joined=3` each; reclaim `ORPHAN-RECLAIMED=192 NOT-AN-ORPHAN=132 NO-OP=0`; host `dmesg` **0 lines, 0** `NVRM`**, 0** `Xid` — both of `w366`'s faults gone.

★ **The identical doorbell counts are the strongest single line**: the second and third processes now do exactly the same amount of work as the first, where two boots earlier they rang eight doorbells and stalled forever.

⊘ **What this is NOT, stated at the width the evidence supports.**

(1) Three is not unbounded. `D`, the *fourth* process, still fails — `avail=False`, *"CUDA unknown error"*. [measured] `w367`: only *three* processes ever existed for four workloads, so `D` never reached the device at all, and the guest kernel logged nothing while it ran — guest `dmesg`'s last line is at 28.99 s uptime. **It fails above the** `ioctl` **boundary and the cause is unknown.** ⚠ The `w367` run's logs are not committed to this tree; this row rests on the boot's commit message (`56dc01f3`) and on its operator's reading of the log, not on an artifact a reader can open.

(2) The LLM workload has not been re-run at this revision, so nothing here says it passes, and §8.5 should be read as pinned to `3887be37`. ⚠ `D`'s error string is *verbatim* the LLM workload's, which makes the two plausibly one defect that bites after ≥ 2 processes — **deliberately not welded**: the `w355`/`w360` LLM boots died *earlier*, with `COMPLETION-WATCH = 0`.

(3) A host object is leaked per reclaimed orphan frame — deliberately, and the log says so. With no table row there is no host virtual address and no memory to hand to the revoke path; *the missing row is exactly what would have carried them*. Bounded at one object per orphaned frame, and a frame can be orphaned only once. The follow-up is named: plumb the backing that `FbStore::release_join` already returns out through `RegPlane::release_fb_join`, which currently discards it.

(4) $n = 1$ **boot**, against this project's own measured rule that a single-boot green is wrong **1 time in 5**. [unverified] at any larger n.

★★ **And the mechanism prediction was wrong in sign, which is the transferable part.** The rung pre-registered *"*`SUPERSEDE CAPPED` *falls sharply"* as its mechanism clause. It went **202 → 5 291**, and the frame that raised the `Xid` still reports `refused=482` — while `joined` went to 21 and the `CE Xid` disappeared. Re-keying creates far *more* tracked pairs, so *more* `CAPPED` lines is the expected consequence of the fix, not evidence against it. ⇒ **The outcome clauses were met and the mechanism clause was measuring the wrong quantity.** *Pre-register outcome and mechanism separately, and say in advance which one grades the rung.*

8.12 ★★★★★ The fifth device open wedges the GPU — and the code predicted it, then filed it as a trade-off

★★★★★ **Added 2026-08-21, and it supersedes §8.11's framing and §8.10's subsection title at the same time.** §8.11 calls the defect *"sequential multi-process"* and reasons about CUDA processes throughout. [measured] `w370`: running `nvidia-smi` **eight times** in one boot — no CUDA anywhere, no context ever created — **#1–4 print the GPU name and #5–8 print "No devices were found"**, permanently. A bare `nvidia-smi` is both a consumer of the resource and a victim of its exhaustion. **The ordinal was never over CUDA processes. It is over device opens.**

And §8.10 is titled *"reboot is modelled and never exercised"*. It has now been exercised, eight times in a row, by the cheapest possible workload — **and it broke.**

Why nothing had ever seen it

A census of every probe log on the bench: **no boot has ever run more than five GPU-touching processes**, and every boot before `w361` ran *one or two*. The pattern was always *boot → one CUDA program → grade → tear down*. The defect lives at $N = 5$; every instrument stopped at $N = 2$. ⊘ **This is therefore not a regression** — it is a limit that was always there and was never probed, and the reason it was never probed is that **the project's own workflow documentation told everyone to reboot between runs.** The line *"the emulated GSP's WPR2*

state only resets on a full QEMU restart, so each clean run needs a fresh boot" has been in this tree's contributor guide for months, written as a *convention*. It was a defect report the whole time.

The guest kernel's own words, and a decode error worth recording

No boot in this series had ever captured guest `dmesg` *during* a workload: the harness took it at boot time, so all twelve boots wrote a **5 379-byte, byte-identical** file whose last line is at 28.99 s uptime. That absence had already been reported upward as *"the guest kernel logged nothing"* — a measured-sounding claim that had never been measured. With the capture moved after the workload, the guest says:

```
NVRM: GspMsgQueueReceiveStatus: Bad sequence number. Expected 0 got 161.
NVRM: …NV_ERR_INVALID_DATA (0x25) from rpcRecvPoll(…, GSP_INIT_DONE, 0)
NVRM: _kgspBootGspRm: unexpected WPR2 already up, cannot proceed with booting GSP
NVRM: RmInitAdapter failed! (0x62:0x40:2028)
```

⊘ **The triple is `initStatus:rmStatus:line`, and reading it as a status cost an hour.** `0x62` decodes as `NV_ERR_RESET_REQUIRED` if you assume it is an `NV_STATUS` — plausible, thematically perfect, and wrong: it is an `RM_INIT_*` enum and the actual status is the *second* field, `0x40`. ★ The same class as this paper's `_IOC_SIZE` finding: **a field parsed as the wrong type still yields a believable name**. And the `0x40` is not even the first domino — it is the *retry's* failure at the WPR2 gate. The poll returned `0x25`.

PC-D7: the comment that described this exactly, and filed it

The root cause is that we post `GSP_INIT_DONE` carrying a message-queue sequence carried out of the *previous* driver life — 161 — into a queue the new life has just constructed at `rxSeqNum = 0`. The guest recovers only from `seqNum < rxSeqNum`; for > there is **no branch**, so the stream never resynchronises and every later open dies instantly at the WPR2 gate.

★★★ **The code predicted this failure verbatim, symptom for symptom, and marked it *"a named trade-off, not a defect to be fixed"*.** The instance identity was the shared region's *guest-physical address*; the comment observes that the allocator recycles pages, that a recycled page therefore reads as "same instance", and that the guest then sees `seqNum > rxSeqNum` *"permanently"*. It judged the collision rare because it modelled the guest's queue as living from module load to module unload. **At 580 that queue lives and dies with the *adapter***: every `nvidia-smi` destroys and reconstructs it, so *every* open is a coin flip on allocator reuse. That nondeterminism is why the failing cycle wandered between 3 and 5 across boots and read like a counter — and why a day went into looking for a resource with four units.

> **The lesson is not "write the hazard down".** It was written down, in the right place, in detail, with the exact symptom. **The load-bearing claim was the *frequency estimate* attached to it** — and that estimate, not the mechanism, was what made it a trade-off instead of a bug. A hazard note without a measured rate is a hypothesis wearing a decision's clothes.

The fix, and the two rejected fixes recorded beside it

[measured] `w374`, real GA106, `147694ff`. The discriminator is now the guest's own command sequence rather than an address: the guest stamps its private `txSeqNum` into every command element it submits and zeroes that counter only at queue construction, so the command stream carries the instance's identity **in-band, in memory we already read**. The address survives as a logged cross-check, and a publish where the two disagree is exactly this bug happening — now a counted line. ★ It fired on hardware:

```
GSP-PUBLISH gpa=0x23200000 gpa_same=true seq_last=Some(1) cmd_seq=162 ⇒ same_instance=false
★ PC-D7 COLLISION: address and sequence disagree --- the address was lying
```

`Bad sequence number` went **3 → 0**; cycles 1–4 are now completely clean. ⊘ **And the device still wedges at cycle 5**, of a *different* cause — a scrub-channel event-notification failure — which an independent analysis had pre-registered as a separate bug that this fix would not touch. **The wall moved; it did not fall.**

> ⊘ **UPDATED 2026-09-11, on two points.** **(1) The ordinal is not five.** [measured] `w423`, in the guest, one fixed configuration repeated so the ordinal is the only variable: opens 1–3 pass and **open 4 fails**

with `EIO` **on** `openat("/dev/nvidia0")`, permanently, with no CUDA, no map and no engine anywhere in it (`R34_OPEN_ORDINAL_FIRST_FAIL = 4`). The earlier 5 is not refuted — a different boot with different preceding opens — but **"five" is a sample, not a constant**, and this paper's section title reads as a constant. **(2) The guest driver's own reason is now captured**, and it confirms the pre-registration rather than a new guess: RM's internal CeUtils **scrubber** channel fails, `RmInitAdapter failed!`, with left-over IOVAS mappings reported on teardown. ★ **And it is not the LLM's blocker**: skipping the diagnostic that consumes opens leaves the LLM failing identically. ⇒ *the wedge and the LLM race are two bugs, and the cheap probe now runs first so no later sweep is read through it.*

★★ **Two rejected fixes are recorded at the source site so they are not retried.** The obvious one — releasing the instance identity at the driver-life boundary — fails an existing test that encodes the C artifact's own expensive lesson, because that boundary serves both a real teardown and an idle release, which need opposite answers. Its "safe" repair, scoping to the Booter-Unload transition, is **worse**: measurement of the 580 teardown path shows that transition **never fires** in this configuration, because our own emulation takes WPR2 down first and the guest's own gate then skips it. That variant would have been **inert on the bench with every test green** — the exact shape of a change that reads as done.

What this says about the amplifier

★ **The "WPR2 already up" wedge is faithful to real hardware and is not the defect.** Any single failed adapter init leaves the device unusable until the guest reboots; a real GPU behaves the same way. So this was never one bug — it is N triggers behind one amplifier, and each trigger has to be killed separately. **The amplifier is also why the symptom is so much larger than any individual cause**, and why "the GPU stops working after the fifth `nvidia-smi`" is a fair description of a defect whose actual mechanism is a sequence-number comparison.

8.13 What contact with hardware has and has not bought

Stated once more, in its corrected form, because it is the thing most likely to be half-remembered between here and the end of the paper.

What is no longer true, and every one of these was printed by a previous revision of this paper as a settled fact. *There is no binary* — false. `RmBackend` *and* `IsolateFactory` *have only mocks* — false. *No isolate process has ever been spawned* — false. *No guest driver has ever posted to the faked GSP* — false. `Arch` *has zero production implementations* — false, there are three. *The boot does not complete* — false; the guest reaches a working `nvidia-smi`. *No compute exists in Mode 2* — false; `cup3` returns 43 and `cup8` is byte-exact at 2048^2 and 3072^2. *Every boot reports* `doorbells: 0 arrived` — false; the `cup3` boot reports `480 arrived, 480 served, 0 REFUSED`. *The control plane and the execution plane have not met* — false. ★ **And one added 2026-08-20, hours after this list was written:** *"a second CUDA context sequentially in one boot still hangs"* — **false**; three sequential processes pass (`w367`, §8.11). *"The next wall is our own object graph"* — **false**; every refusal in it is more numerous in the boot that passes (§8.5).

What is still true, and it is now a different list rather than a shorter one.

- ⊘⊘⊘ **FALSE AS OF 2026-09-06, AND THIS BULLET IS KEPT ONLY TO SHOW WHICH WAY IT WAS WRONG: THE LLM GENERATES.** [measured] 2026-09-09, rev `c8fe67d6`: `W392_OUTCOME=(P)`, 16 tokens, **text byte-identical to the same-boot CPU oracle**, host `Xid` 0 (§8.5). ⊘ **What is still true is a much narrower sentence**: it is **45 s per token**, the harness grade is still a *count* and the project's own requirements mark that RED, n is 1–2 boots, the same workload still fails under other arms on a [measured] **race** (the pin and the `FAULT_PDE` land in the same second), and it **has never been run under the three blocking constraints** (§4.4). *Nobody would run this yet; "it has never produced a token" is no longer the reason.* The superseded original follows, kept rather than deleted.
  "No LLM has ever produced a token in this guest. The `cudart` *initialisation* wall is broken (§8.5); torch's full CUDA init raises *"CUDA unknown error"*. ⚠ `Xid` 0 — *a refusal we issue, not a fault.* ⊘ **This bullet used to name the cause** — *"the failure moved one rung, into our own object graph:* `PromoteFault::UnknownContextObject` *and a handle resolving to the wrong kind on the context-promotion path"* **— and that was refuted three minutes after this document's pin** (`129dd316`): every one of those refusals is *more* numerous in the boot that passes. **The cause is unidentified.** ⚠ And the workload has not been re-run since `3887be37`, so its status across the §8.11 fixes is [unverified].
- **Multi-process is demonstrated for *concurrent* and for *sequential* processes, and bounded at three.** ⊘ **This bullet has moved out of the "still true" list it was written for.** It read: *"a second context after the first's teardown in the same boot still hangs at* `cuCtxCreate` *(w311) … the bench's 'fresh boot per clean run' discipline is still standing in for a fix."* **That is false as of** `56dc01f3`: three sequential processes pass in one boot, 8/8 com-

pletions each, identical doorbell counts (`w367`, §8.11). ⚠ **What is still true is narrower and is a different sentence**: a *fourth* process fails, above the `ioctl` boundary, on $n = 1$ boot, cause unknown — and the C artifact's own *"exactly one CUDA process works per QEMU lifetime"* was **not** the same defect after all.

- **There is no tenant axis.** §8.18. No two-VM run has been attempted by anyone.
- **Every green is a relaxed green** with named relaxations still armed (§8.3), and no rung has yet removed them one at a time to see which the 43 survives.
- **Performance is not close.** §8.4: 22–81× off native on large kernels and operands in host system memory at **2.51 GB/s** against VRAM's 313.5. ⊘ **The clause "our own doorbell handler dominating small ones" is now historical**: the doorbell defaults to deferred and the publication runs on a worker (§4.4). **No successor breakdown exists**, and the one number that replaced it points at a different register: `worst_trap = 1.78 s at bar0+0x110c00`, the GSP queue head, which two lock fixes did not move and which a stack sampler then showed does *not* occur during steady-state running. *The floor moved and has not been re-measured.*
- **Reclamation is modelled on an event that does not occur.** [measured] 2026-08-19: CUDA's suballocator *does not unmap on* `cuMemFree`, so the shipped release trigger fires **zero times** on this workload and is behaviourally identical to `off`. ⊘ **Refined 2026-09-11, and the refinement is a rule rather than a new trigger**: a source audit of the 580 driver found that an unmap which *sparsifies instead of unmapping* is a genuine **revocation** and *is* signalled, while the bulk sparsify at construction that looked like a missed event is not an event at all. ⇒ *what must be observed is not "a page-table entry changed" but "real backing appeared or disappeared"* (§4.4).
- **The trace vocabulary still does not emit.** A grep for the emit call over `crates/*/src` hits only the crate that defines it.
- **One part, one driver, one guest.** GA106, 580.159.04 open. The second architecture (`Ad10xArch`) has never been booted, and its `mmu()` still delegates to `MockArch`'s *invented* GMMU format.

★★★ **What this means for the claims in this document, and the shape of it changed.** Everything about *structure* — the DAG, the purity, the type-level isolation, the lock discipline — is verifiable from the tree and this paper verified it. Everything about *behaviour against NVIDIA* was inherited from the C artifact and re-expressed, and the re-expression is now checked at the strongest point available: **a stock guest driver ran a CUDA program whose numerical answer only a real GR engine could have produced.** That retires the whole class of doubt about whether the protocol model is *right*.

⊘ **It retires nothing about whether it is *complete***, and the remaining walls are all of that kind. ⊘ **CORRECTED 2026-08-20 (`56dc01f3`), and both halves of this sentence had moved by the time it was read.** It named *"`cudart` fails on a control we refuse"* — already superseded by §8.5 at this very pin, which reports that wall broken — and *"the multi-process property fails or holds on publication paths no boot has exercised"*, which `w361`–`w367` then exercised, found a defect in, and fixed (§8.11). **What stands is the shape rather than the instances**: *the project has stopped being blocked on "does this work at all" and started being blocked on "what else does the driver want" — which is a better problem and a much longer tail.*

★ **And the sharpest single lesson of getting this far is about testing, not about NVIDIA.** An R1 invariant was violated on the production path for an unknown number of commits while **1 975 tests were green** (§8.17). The assert existed, was correct, and was old. It was simply unreachable from any path CI can execute, because the suite spawns isolates through a mock that cannot block. *Where the mock and the real implementation differ in whether they can violate a rule, the assert belongs at the seam they share* — and until it does, the suite's green is a statement about the mock.

8.14 The quadratic control plane — an open, measured DoS

The invariant catalogue's I4 says hostile input is *contained*: no wedge, no OOM, no bystander corruption, every hostile event earning only its own refusal. All of that is property-proven. It says nothing about *cost*, and the project measured the cost and recorded the result against itself:

The control plane is O(live objects) per event, so N events cost $O(N^2)$. Measured on a debug build: **1 000 events 0.85 s; 8 000 events 54.8 s**. The capacity cap is 2^{18} live handles, so a guest can drive that curve deliberately — and **PyTorch startup reaches it benignly**. Deleting the per-event graph clone saves ~24% and leaves the curve quadratic, so the clone is neither the cause nor the fix.

The stated statement in `ARCHITECTURE.md` — "no $O(n^2)$ path on hostile floods" — is marked **false** in the same document. Two candidate fixes are named and neither is small: incremental derivation, which is a redesign of the "derived state is a pure function of the graph, never accreted" decision that the entire determinism differential rests on; or an undo journal in the most safety-critical file in the repository. A sibling of the

same species *was* fixed (a condemned-list carry-forward, $O(n^2 \log n)$, 55 s → 3.8 s, by union-find), which is evidence the class is tractable and also evidence that it recurs.

8.15 Reclamation is the least finished plane

Figure 5 marks two triggers red. Restating the sharper form:

"Retire eager, reap deferred" is currently half a law. The deferral has no bound and nothing arms it. In a run that never quiesces, reap-deferred and reap-never are indistinguishable.

The one genuine cross-process reference has a disposition that no ledger will like. On a kernel/UVM dup of a user address space, refcount zero and per-object `Free` are not the same event: the last reference retires and reaps the owner *without issuing a single* `Free` *verb*, and the disposition of record is the isolate process's death. GPA is conserved independently and asserted. But any future reclamation ledger, quota or accounting hook will be wrong on that path first — which the design says, in advance, in its own words.

★★ And a third disposition arrived on 2026-08-20, unplanned, as the fix for the sequential multi-process wall (§8.11). A framebuffer join can outlive *every* table row that named it: the store is per-device and keyed by physical address, the tables are per-address-space, and a process's rows are unbound in the same tick as the reap — so by the time anything looks, the corpse's table is empty while the store still holds the join. [measured] `w365`: three GR context frames, 48 refusals each, from two independent later processes, all censused `live=0 retired=0`. **An orphan is provably unowned, so reclaiming it needs no cross-process trust argument at all** — and `live > 0` stays refused *by name*, 132 times in the passing boot. ⊘ **The disposition is a deliberate, logged leak**: with no table row there is no host virtual address and no memory to hand to the revoke path, so the host object is dropped rather than freed. Bounded at one per orphaned frame, and a frame can be orphaned only once. **The follow-up is named and not done**: plumb the backing that `FbStore::release_join` already returns out through `RegPlane::release_fb_join`, which discards it. ⇒ *This is the second reclamation path whose disposition of record is "the owner died", and any future ledger or quota will be wrong on it first — for the same reason as the one above.*

And the most leak-prone moment has no vocabulary at all until L1-M2 lands. A worker that dies *mid-chain* cannot run its own unwind, so everything allocated before the failure is in no `Orphans` and in no core state. It is unrecoverable and no current test can see it. The design's answer — escalate to killing the isolate, because killing the process is how you reclaim state you cannot enumerate — is correct and is not yet built.

8.16 arm64 is measured for the core and unmeasured for the shell

The full suite *ran* on a GB10 Grace-Blackwell aarch64 host: **372 passed, 0 failed**, identical to x86_64. That is a real upgrade over a cross-compile check, and it covers the concurrency suites, whose memory-ordering behaviour is exactly what a `cargo check` cannot see.

What it does not establish, stated by the project so nobody over-reads it: `getconf PAGESIZE` was 4096 on that host, so the 16/64 KiB page-size rule — the one real pressure point — is still unexercised. The box was a container with no `/dev/kvm` and no `/dev/userfaultfd`, so **no L1/L2 OS-shell behaviour was exercised on ARM at all**. And there are zero VM-capable arm64 machines on the bench provider, so settling the remaining question by experiment requires hardware from somewhere else.

The honest summary is the project's own: *the pure core is arm64-clean by measurement; the OS shell on arm64 is entirely unmeasured*.

8.17 ★★★ R1 was broken on the one path the suite could not see

Invariant **R1** is the one the whole concurrency design is built on: **no blocking call under any lock, ever**. Blocking calls take *zero* ranked locks. It is asserted, not documented — a thread-local lock-depth counter checked at every blocking-verb entry.

It was violated on the production path, and only a real GPU could say so. With the real isolate plane enabled, QEMU **aborted** on the guest's first register write that reached an object allocation: *"R1 no-blocking-under-lock violation: spawning a sandboxed child process while holding rank(s) [0]"*, in a non-unwinding panic, with `kayfabe_shim_regs_write` ← `nvkvm_trap_write` below it — an ordinary guest register write.

The defect in one sentence. Materialising an isolate is a `clone` into six namespaces, an `execveat` of a sealed memfd, a blocking handshake and a chain of host RM ioctls — and it had been placed on the `Ok` arm of the core's apply, which runs under the device write lock. **R1 admits no exception for that, and the design already said so.**

★★★ **Why the suite was blind, which is the more important half.** The assert that fired is real, correct and old — it lives at the syscall, in the raw-syscall crate. **It is reachable only on the host plane.** The whole suite spawns through `MockIsolateFactory`, whose spawn blocks on nothing and asserts nothing, so on every path CI can execute the violation was invisible: **1 975 green tests over a live R1 breach.**

An invariant asserted only in the production adapter is an invariant the test suite does not hold. The repair was not only to move the spawn — it was to move *the assert*, to `IsolateBox::new`, which is the one way an isolate enters core state whichever plane made it. **That turns the entire mock-driven suite into a live guard for it.**

⊘ **And when it broke is [unverified].** Earlier increments' live real-isolate measurements no longer reproduce, no bisection was run, and no claim is made about which commit did it.

The fix is a phase-shape change, not an edit, and it is the same latch-and-discharge two-step the codebase already used twice elsewhere: *decide* which isolates are needed under rank 0, *spawn* with zero locks held, *install* under rank 0 with R5 re-validation. A new deferred state, `IsolatePending`, resolves in the device loop. ★ **Six doors needed the discrimination, not two**, and *four of the six were found by the test and by nothing else*.

The acceptance, and it is a properly-controlled one. Three boots on an RTX 3060: two with the real plane, both exit 0, **zero** R1 violations, one isolate child each at `t+42 s` and `t+43 s` against a device open at `t+37 s` — so the spawn still *follows* the guest's action, re-measured rather than assumed. ★ And the pre-fix control was taken **on a second physical machine the fix's author never touched**, at a master revision, where the abort reproduces — so the pair is a genuine control rather than a comparison across two variables at once.

⊘ **And the honest headline is that the wall did not move.** The guest dmesg is byte-identical across all three arms — 26 lines, 22 `NVRM`, 3 adapter lines. *Why unchanged is the expected answer and not a disappointment*: the isolate plane serves *verbs*, and the boot dies long before a channel is scheduled, so there is no guest submission for it to serve. What the run buys is that **the plane can be switched on at all**, which is the prerequisite for everything after it.

★ **Five bite-checks, four bit, and the fifth found the test.** A `#[should_panic(expected = "R1 no-blocking-under-lock")]` was *vacuous*: the assert under test shares its panic message with the isolate's own `Drop`, so the expectation was being satisfied by the wrong door. The general form, worth carrying: **a `#[should_panic]` whose expected string is produced by more than one site in the code under test is not a test of either of them.**

8.18 Multi-tenancy: two shapes contained, one that bites, and no tenant axis at all

Multi-tenancy is the thesis (§3), so it is the claim most worth attacking. Three denial shapes were measured on a real GA106, and they did not come back the same way.

Shape	Verdict	What was measured
Infinite compute kernel	**contained**	229 376 threads against a ∼43 008-thread resident capacity, so the device is genuinely oversubscribed. A second process: **12/12** runs over 60 s, `bad=0` every time, **zero Xid**, no reset. It costs its neighbour roughly 2× throughput — *a fairness problem, which is the honest name for it*. ⊘ The *mechanism* is [inferred]; the run observed only that the victim survived.
Guest-reachable MMU fault	**contained**	A genuine `Xid 31 FAULT_PDE`. The attacker's context dies sticky; ★ a victim **holding a live context across the fault** completed **2 682 576 iterations, 2 682 576 correct, 0 wrong**. **Stronger than the row above, because the escalation path actually ran** rather than merely not being needed.
VRAM exhaustion	★ **NOT contained**	An *unprivileged* hog took **11 776 of 12 288 MiB (95.8%)**. A second tenant **cannot even create a context** — out-of-memory in **0.29 s**.

★ **Why only the third bites, and it is a one-line argument.** The first two rest on *the GPU's scheduler*. The third rests on nothing — it is ordinary allocation. **Denial is total**: not "the victim's allocation fails" but "the victim cannot get onto the device at all", which is strictly worse, because a tenant that cannot create a context cannot even begin to degrade gracefully. ★ **But it is honest denial** — immediate, named, and fully reversible, with no Xid and no reset. *That is a resource problem, not a recovery problem*, and the mitigation is ordinary and buildable: a per-tenant VRAM quota, enforced where we already broker allocation. It is the one multi-tenant exposure so far that is squarely inside our reach.

★★ **And the instrument was wrong first, in the way this paper keeps recording.** The first version of the wedge probe span **1 block of 32 threads on a 28-SM GPU**. The victim survived *because 27 SMs were free*. It reported total containment and **could not have shown otherwise** — and the one number that looked like a saturation check, GPU utilisation, reads 100% in both cases, so it could not discriminate either.

★★★ **The confidentiality answer is NO, and the reason is structural rather than a missing control.** Asked whether the security model is sufficient for aggressive multi-tenancy, the assessment answers: **no — because there is no tenant axis in this system at all.** Every isolation property this tree proves is *intra-VM, between guest processes*. There is no `VmId`, `TenantId` or `GuestId` anywhere in `crates/`. Cross-tenant separation is the incidental consequence of two VMs being two host processes.

And at the one place two tenants genuinely meet — the host NVIDIA driver — the driver's own cross-client check is defeated by construction. RM compares client tokens with an **OR**: a matching euid alone passes. Every isolate presents the same euid, so two tenants' isolates pass RM's cross-client check against each other. ★ And the handle is *not* guessable-in-theory-only: it is a fixed base OR'd with a monotonically increasing index, so concurrent isolates receive adjacent values. **There is nothing to guess.**

★ **Two narrowings that are genuinely reassuring, and are not the same as a fix.** RM's default cross-client *dup* policy is PID-scoped, not euid-scoped, so the dup path across tenants *is* refused even at matching euid — the exposure is the direct-naming path only. And the expected break — residual tenant data on GPU memory reuse — **does not exist on the paths we drive**: the host driver scrubs VRAM on free and makes the frame un-allocatable until the scrub retires.

⊘ **Every claim in this box is [inferred] from driver source. No GPU was switched on for it, both source documents say so in their own banners, and the single experiment that would settle it — two VMs on one box — has never been attempted.** The highest-value single change is named and is one line of deployment policy: **run each isolate at a distinct non-zero uid**, which moves the boundary from our code to the kernel's.

So where does that leave "red-teamed at the logic layer only"? That sentence, which this paper printed at the previous revision, is now **half discharged and half still true**, and the split is worth stating precisely. **Discharged at the host-GPU boundary**: a system-level property now exists — *every effect a hostile guest can produce on the host GPU is an effect a local unprivileged process could already produce* — with a fourteen-finding audit of the actual host-facing surface behind it, the three denial shapes above measured on hardware, and a **measured host-filesystem escape found and closed** (the isolate's own `/dev` descriptor opened `../etc/shadow` on a real host). **Still open at the cross-tenant boundary**: no tenant axis, **no** `seccomp` **anywhere in the tree** (absent, and named as absent rather than stubbed), no two-VM run, and the security review of the new FFI surface is the declared exit gate of the L2 work and **has not been done**.

8.19 The QEMU $\geq$ 10.2 floor buys less than the decision that took it assumed

★ **This section is no longer a forecast.** At the previous pin the L2 adapter had not been started and this section priced it. It is now built through stage Q5, compiled into *two* real hypervisor binaries (9.2.0 and 10.2.4, the same overlay unchanged), and booted against repeatedly on a real GA106 — and on 9.2.0 the identical binary path is **refused by name** at the runtime floor, which was watched rather than assumed. **Every cost itemised below was paid, and none of the estimates below turned out to be wrong.** The exit gate, Q7 — the security review of the FFI surface and the written R1/R3/R5 re-audit — is **not done**.

The design requires `memory_region_enable_lockless_io()`, which lands in QEMU 10.2. That buys a genuinely important thing — MMIO dispatch without the big QEMU lock, which is what makes the doorbell path viable — and upstream's helper also sets the reentrancy guard in the same body, so a discipline the design planned to maintain is no longer ours.

The floor decision's second claim was economic: requiring a version rather than patching one deletes *"the whole 'which QEMU are we built into' ambiguity"*. **The L2-Q adapter design** (`b31487a`) **checked that claim against QEMU's source and it does not hold.**

QEMU v10.2.0 has no supported out-of-tree device mechanism, so our device must be compiled inside a QEMU source tree. Two mechanisms, both closed: `module_load_qom()` resolves a QOM type name against `module_info`, a table *generated at QEMU build time*, so a `.so` that is not in that table can never be discovered by type name — and `QEMU_MODULE_DIR` changes only the directory searched, not the table. `module_load_dso()` additionally requires the module to export a per-build `CONFIG_STAMP` symbol; upstream's own failure hint is verbatim *"Only modules from the same build can be loaded."* [v10.2.0 util/module.c, include/qemu/module.h]

State it exactly, without overstating it. The floor *does* delete a patch we would otherwise have had to carry: the cancelled 9.2 backport modified upstream semantics inside `system/physmem.c`, had to be re-derived every release, and a mis-rebase would silently change how *every* device in the machine dispatched. What replaces it is an additive overlay — a directory dropped into a stock 10.2.x checkout plus two hunks (one `meson.build` line, one `Kconfig` stanza) — whose only conflict surface is a build failure, never a behaviour change. That is a real improvement. **What it does not buy is "install your distro's QEMU and run".** The user builds QEMU once. Three of the four costs the floor decision itemised against the backport — a rebase every release, a build script that must reproduce it, and a supply-chain claim we make to every user — still apply.

There is exactly one path to a genuinely stock QEMU, it is named, and it is not taken for v1. `vfio-user` would make our device a separate process speaking a documented socket protocol to an unmodified distro-packaged QEMU. It is present at v10.2.0 (`hw/vfio-user/`, two documentation files, a vendored `libvfio-user`) and `MAINTAINERS` lists it `S: Supported`. It would delete the QEMU tree, the overlay, and the second `unsafe` crate the in-tree route needs. It is rejected for v1 on one source read: nothing in `hw/vfio-user/` calls `memory_region_enable_lockless_io`, so a trapped BAR access becomes a socket round trip *taken with the BQL held* — reintroducing and amplifying the exact serialisation the floor was taken to remove. The design records the small experiment that would change the verdict (measure the round trip both ways, and ask whether marking the client's regions lockless is a patch upstream would accept) rather than closing the question.

The facility inventory that priced the floor also **reversed two of its own design assumptions**, which is worth knowing before trusting the rest of it: the read-only-memslot region-lock fallback, priced in the design at 1.49 µs as "a flags-only flip", is *unreachable through QEMU* — QEMU's flatten pass turns a readonly change into a delete-plus-add. And checkpoint-restart (`cpr-transfer`) is a lifecycle event the design had never considered.

One more cost the adapter design surfaced, did not argue away, and then paid. A QEMU adapter needs `extern "C"` entry points, raw `MemoryRegion *` handles and adoption of a host pointer QEMU owns — all `unsafe`. The workspace's CI gate exists to keep the audit surface at *one* crate readable in one sitting, and its own failure text says adding a second is "a design decision, not a manifest edit". **That decision was taken and the price was paid in full**: `kayfabe-qemu-raw` is the second audited `unsafe` crate at **49** relaxations, the ratchet is now a named two-element list, and the acceptance criterion the gate protects *is* "read two crates". ★ Its bar started at **0** deliberately, so the first relaxation had to be a reviewed commit rather than a manifest edit — and the crate's first act as an FFI consumer was to prove the ratchet's own counting pattern wrong (§5).

And the packaging verdict is unchanged and now testable. What the floor buys is an additive overlay — a directory dropped into a stock 10.2.x checkout plus two build-file hunks — and that is exactly what shipped: `qemu/hw/misc/nvkvm/`, 2 676 lines of C, built by `scripts/build_qom_shim.sh`. **What it does not buy is still "install your distro's QEMU and run".** The user builds QEMU once. The `vfio-user` route that would delete that cost remains named, priced and not taken for v1.

8.20 The bench — rented, serialised, and now more than one box

All hardware work happens on rented GPU boxes, strictly serially: concurrent or mid-ioctl activity wedges the GPU. Two cargo builds on one box corrupted a mutation run through the shared `~/.cargo`, so the rule is one cargo per box.

★ **The previous revision said "there is no second machine, no second GPU generation, and no continuous hardware testing." Two thirds of that is now false and the correction matters, because $n = 1$ was this paper's standing caveat on every hardware number.** Work since has run on **four machines and three distinct physical GPUs**: a GPU-less four-core development box (where, notably, *every* guest boot in the boot log ran); a rented 38-core box with an **RTX 3090 (GA102)**; and **two different RTX 3060s (GA106)** on separate rented instances. Three cross-machine reproductions are load-bearing rather than decorative:

- A wall reproduced **line for line, 36 of 36 dmesg lines**, between the GPU-less box and a real-GPU box — which is what establishes it is a property of the port and not of a machine.
- The bring-up ladder run on **two different physical GA106s** differs by **one line of thirty-three**, and that line is a handle. A committed transcript is therefore a real cross-machine regression test.
- The R1 abort (§8.17) was reproduced at a master revision on **a second machine the fix's author never touched**, which is what makes its before/after a control rather than a comparison across two variables.

⊘ **What has *not* changed, and the reader should hold onto it.** Every *boot* result is GA106, one driver family (580.159.04 open), one guest OS, and one guest at a time. The GA102 appears only in ladder and suite runs, never in a boot. There is still **no continuous hardware testing** — GitHub CI has no GPU and never will. And the project's own standing caveat is repeated verbatim in every boot section for a reason: **one Mode-2**

symptom was 1/3 one day and 9/9 the next on a bit-identical binary, so a single green boot is weak evidence and the documents treat it that way.

★★ And the bench has lied about its own provenance, more than once, which is why every claim here carries a revision. A binary served for weeks was built from a commit nobody had noticed. A later one **could not name its own revision at all**: it contained exactly one 40-hex string, which was a build-id and not a commit in either repository, and its only provenance was a worktree **19 commits behind master**. The instrument that was supposed to discriminate — `strings` for feature markers — scored **0 for every marker in both binaries**, because a release build strips enum names and inlines constants, and that was only discovered by checking it against a known-good symbol. ⇒ `strings` **cannot answer "what revision is this"; embed the SHA** — which the archive now does.

8.21 The recurring failure mode: claims that were true *once*

This is the pattern the project keeps hitting, it has a name in the repository, and it is the single most useful thing for a reviewer to internalise.

> Across three independent passes, roughly **twenty** instances were found of documentation asserting more than the code did — **several stating the literal opposite of their own code**. **Every one of them was true when written.** That is the point: this is not carelessness, it is entropy. [docs/design/testing_doctrine.md §6]

★★★★★ The 2026-08-21 instance is the most expensive one yet, and it is a variant this list did not previously contain. Every entry below is a claim that *became* false. The `PC-D7` comment (§8.12) was **never true**: it described a real hazard in correct detail, predicted the exact failure symptom, and attached a *frequency estimate* — "rare" — that was wrong by orders of magnitude because it modelled the guest's message queue as living for the module's lifetime rather than the adapter's. On that estimate the item was filed as *"a named trade-off, not a defect to be fixed"*, and it then made the GPU unusable after five invocations of `nvidia-smi`. ⇒ **A documented hazard carries two claims — the mechanism and its rate — and only the first is usually checked. The second is what decides whether it gets fixed.**

The instances are specific and they are not small:

- A design document named `memory_region_clear_global_locking()` as the mechanism for a decision. **That QEMU function had been deleted five years earlier.** The reference document written to price the floor now carries a `file:line` on every row for exactly this reason — and found two *more* live instances of the same decay while being written.
- A refactor plan cited a userfaultfd proof-of-concept file **that does not exist**. The mechanism was planned four times and never built; its demand-fault path is dead code that calls `stub_exit(139)` and never returns, so the fault address has been permanently zero since a commit early in the project's life. Nobody noticed because "planned" and "presumably done" read the same in a plan.
- **Two sections of one design document contradicted each other**, found only when the code between them was written.
- Two claims about the C's behaviour, cited from the C's own source comments, turned out to be **false when measured on the bench**. The rule extracted: citing a comment is citing a belief.
- A rustdoc described a default-to-GPU-0 guess *inside the one resolver whose entire discipline is "never guess"*. Another claimed "MISS=FAULT is preserved — never a silent skip" while its own body correctly skipped on both arms; the body was right and the sentence was wrong about the most-cited exception in the codebase.

The project's response is the right one: prefer a mechanism to a sentence. A rule stated only in prose has no reader. `HostHandle` carrying its `IsolateId` is the same rule with a compiler behind it. The house style for corrections is *strike visibly* — keep the original, struck, with the correction and its evidence — so a reader who remembers the old claim learns it was wrong rather than doubting their memory.

★★★★★ A FURTHER INSTANCE, ADDED 2026-08-20 AT `56dc01f3`, AND IT IS THE SHORTEST INTERVAL YET MEASURED: FOUR HOURS AND FIFTY-FIVE MINUTES. This paragraph is folded in above the tally below because the tally is what it corrects.

This revision was pinned at `5c367a38` and committed at **15:36**. Its **headline blocker** — *"a second CUDA context sequentially in one boot still hangs"* — was **fixed at 18:31** (`ecb67fae` onwards) and **measured passing at 20:20** (`cca2eb4b`, three sequential processes, §8.11). A *second* headline — *"the next wall is a different subsystem: the object graph"* — was refuted at **15:39**, **three minutes** after the pin, by a negative control that had been sitting on the bench as two log files the whole time (§8.5).

⇒ **That makes four consecutive revisions in which the falsified claim was the one the paper led with**, and the interval has gone from *days* to *minutes* — this time without a new revision being written at all. ⊘ **And the two instances are different species, which is the useful part.** The multi-process claim *went* stale — true when written, overtaken by work done afterwards, and no discipline available to a writer would have caught it. The object-graph claim was **false when written**, and the control that refutes it *already existed*: it was a census of a failing run read as a wall, by an author who had written down the rule *"the failing run is not the experiment; the DIFFERENCE is"* eleven messages earlier. *Only one of those two is a drift problem. The other is a method problem wearing a drift problem's clothes.*

And the pattern is live right now, in this document's subject, and in this document. The first revision found fourteen instances (§10 rows 1–14). The second found six more, two of them in the first revision's own text. The third found six more again, *three* in this paper including its headline. **This revision found eight more, and *five* of them are in this paper** — including, again, the sentence it led with, which has now been the falsified claim in **three consecutive revisions — four** counting the post-pin correction pass recorded in the box above.

★★ **The direction of the drift inverted at this revision, and that is a new thing to say about it.** Every previous instance was a document claiming *more* than the code did. Five of this revision's eight are the opposite: the paper claimed *less* than the code did — no binary, no isolate, no `Arch` implementation, no walk function, one adapter. **An understatement decays exactly as fast as an overstatement and is harder to notice**, because nobody audits a document for being too modest, and a reader who acts on it makes a *different* wrong decision rather than an obviously wrong one. ⊘ It is the same defect: a description that used to be true of a thing.

Every claim that *does* have a gate is mechanically true today — the boundary greps, the generation-name grep, the two-crate unsafe ratchet, the seven reached-count floors, the "exactly one GPA call site" assertion, the E0639 compile-fail row that makes the ring gate structural, the bridge-exclusivity grep *which fired on its first run*, the ogkm-version-tag gate, and the claim ledger. Four revisions of this paper have now found roughly thirty-four ungated claims to be stale and **zero** gated ones. That is either a devastating footnote or the strongest possible evidence for the project's own thesis, and it is the latter.

★ **With one qualification this revision earned the hard way, and it cuts against the sentence above.** A gated claim is mechanically true *of what the gate quantifies over*, and three separate instruments were found this cycle to be quantifying over less than anyone thought: the mutation gate scored 100% over a universe that a full disk had eaten (§7.7); the unsafe ratchet's own regex could not match the dominant form in the crate it was newly guarding, and **counted 23 of 31 while calling itself a complete audit** (§5); and a citation gate was satisfied by a source that said nothing (§3.1). *"Zero gated claims were stale" is a true statement about a set whose size is itself a measurement* — and all three of those measurements were wrong upward.

★ **The sharpest form of it at this revision is not in the prose at all — it is in the instruments** (§8.9). A configuration file that was never read, a baseline that omitted the suite, a timeout scored as a kill and a hang that printed nothing are all the same species as a stale sentence: a description that used to be true of a thing. The difference is that those four described *the evidence*, which makes them the more expensive kind.

The strongest single instance is in §8.10 and it is worth restating here as a pattern rather than a fact about NVIDIA. A specification was cited as if it were the specification, when it was one version of a specification; 3 550 lines were built against it; and one question had been closed with an answer that is exactly backwards for the machine the code will first run on. The claim that survived that audit was the one marked `[inferred]` — because it knew it was second-hand. The claim that did not survive was marked **RESOLVED**.

8.22 Risks this document adds

- ⊘⊘⊘ **CORRECTED 2026-09-11 — "Nothing anyone would actually run works" is no longer the right sentence, and the right one is worse in a different way.** The model runs: [measured] 2026-09-09, 16 tokens, **output byte-identical to the same-boot CPU oracle**, 0 `Xid` (§8.5). ⊘ **At 45 s per token, on a grade the project's own requirements mark RED because it is a count rather than a content assertion, at $n = 1$–2 boots, on a workload that has never been run under the three blocking constraints, and that still fails under other arms on a measured race.** ⇒ *The risk is no longer "it does not work"; it is "it worked once, slowly, and nothing holds it there."* The superseded original follows. *"Nothing anyone would actually run works.* §8.5. The `cudart` initialisation wall is broken ($n = 2$ workloads) and **the model still produces zero tokens**. ⊘ **This bullet used to continue** *"the failure moved one rung into our object graph"* **— refuted at** `129dd316`, three minutes after this paper was committed (§10 row 44): **the cause is unidentified**, which is worse news than a named wall, not better. ⚠ **And the workload has not been re-run since** `3887be37`, so "zero tokens" is [measured] at that revision and [unverified] at HEAD. ⊘ **And the milestone before it was graded on**

a proxy — `is_available()` went `True`, was reported as the wall breaking, and the actual model disagreed within the hour. *This project has now made that exact mistake twice*, the first time declaring "first compute" on a copy-engine round trip.

- ★★★ **Every green result is a relaxed green, and the shipping default is not among the relaxations.** §8.3. Eleven arms are forced on by the bench runner and five are measured load-bearing; `KAYFABE_GR_ROUTE` — one of the five — still defaults to `refuse` in code. **The configuration that computes is not the configuration that ships**, and no rung has yet removed the relaxations one at a time to see which the `43` survives.
- ★★ **Performance is not close, and the two problems bind at different sizes.** §8.4. Operands sit in host system memory read at **2.51 GB/s** against VRAM's 313.5; compute is **22–81**× off native at large kernel sizes, while at small ones our own doorbell handler is the launch. ⊘ **CORRECTED 2026-09-11 on both halves.** The doorbell no longer runs the publication on the vCPU (§4.4), so the second clause describes a design that has been replaced and **no successor breakdown has been measured**. And *"no tokens/second figure exists at all* $(n = 0)$*"* is now false in the narrowest possible way: [measured] 2026-09-06, `LLM_MS=722820.0` for 16 tokens is **45 s/token**. ⚠ **That is a number, not a parity figure** — there is still no native baseline on the same box, and the project records that anything computed from it is *"for the legacy arming"* and must say so.
- **The mocks are still the only implementation of `Device`, and a mock is where an invariant hid for an unknown number of commits.** §8.17 is the measured demonstration that this is not a theoretical concern: 1 975 green tests over a live R1 breach, because the double could not violate the rule. **If a belief about RM semantics is wrong, the suite will confirm it consistently** — and if a rule is only assertable in production, the suite does not hold it at all.
- **The project moves faster than its own documentation, and it has not slowed. 974 commits since the previous pin**, and the whitepaper's own headline claim was falsified *twice while this revision was being written* — once by a commit that landed mid-session (§8.5). Note also that the committed trace README describing that work is, at this pin, **one rung stale relative to the commit beside it**: it still names a suspect that the next commit retired. `crate_maturity_map.md` — a document whose entire job is to say what is built — is now wholly stale, and this paper stopped citing it rather than laundering numbers through it. **No gate covers prose**, which is why §10 keeps growing.
- ★ **Multi-tenancy has no tenant axis, and that is the thesis.** §8.18. Two denial shapes are contained and one is not; the confidentiality answer is *no*; the host driver's own cross-client check is defeated by a shared euid; and **the single experiment that would settle any of it — two VMs on one box — has never been attempted**. The highest-value fix is one line of deployment policy, which is encouraging and is not the same as having done it.
- ★ **Multi-process holds for three processes, sequential or concurrent, and fails at the fourth.** ⊘ **This bullet was the paper's headline blocker and it was fixed at 18:31 on the day the paper was pinned.** It read: *"fails for sequential ones … a second context after the first's teardown in the same boot still hangs at* `cuCtxCreate`*, which is the C artifact's own limitation inherited … the bench's 'fresh boot per clean run' discipline is still standing in for a fix."* **False at** `56dc01f3` (§8.11) — and the C's limitation was not inherited: the port's cause was its own orphaned framebuffer join. **What remains a risk is smaller and sharper**: a *fourth* process still fails above the `ioctl` boundary with the cause unknown, the result is $n = 1$ boot, and **a host object is leaked per reclaimed orphan frame** — bounded at one per frame, logged, with the plumbing fix named and not done. The concurrent run still scopes itself as before: two identical short workloads with emulated copy work, bounded by the hypervisor's big lock — *a bound that weakens exactly when the forwarded work gets longer*.
- ★★★ **Reclamation is built on an event that does not occur, and the fix is capped by a constant nobody has justified.** CUDA's suballocator does not unmap on `cuMemFree`, so the nominated release trigger fires **zero times**; the working mechanism is a takeover, capped because without a cap the settlement oscillates forever. ⊘ **The cap's key changed on 2026-08-20 and the old key was itself a defect**: keyed by *frame* with no reset, a device-lifetime budget spent by early actors left later leaves unbacked, and **our own bound raised a host `Xid` 31 `FAULT_PDE`** (`cca2eb4b`, §8.11). It is now keyed by `(phys, taking va)`. **Which of two live aliases should win is still unproven**, and failure in this area is a function of allocation *history* rather than size — which is why it took a campaign to find and why `cup8` cannot close it (its buffers never move).
- ★★ **Every mutation number this project has published has been wrong upward, five times, by five different mechanisms** (§7.7). The most recent scored **exactly 100%** while a full disk deleted 78% of its universe and the harness reported `ok`. **The bar has since been lowered from 91 to 80**, and no clean campaign has completed under either, so there is still no defensible score to quote.
- **The C oracle is positively wrong about a whole class of rows, and a third class is unmeasured** (§3.1). 16 truncated rows — more than the 11 empty ones — have never been checked, and a truncated row passes

the test built to catch the empty ones.

- ★ **The security review of the new FFI surface is the declared exit gate of the L2 work and has not been done**, while the surface itself is now 49 audited `unsafe` relaxations in a crate a hypervisor calls directly. There is no `seccomp` anywhere in the tree.
- ⊘ **The item this list used to end with is RESOLVED, and it had been resolved for two revisions before this paper stopped printing it.** §11-O7a was closed on 2026-08-06 (§8.10). What replaces it in this slot is a *measurement* rather than a question: **no teardown / re-acquire boot has ever been run.** The 2026-08-05 recommendation was to spend the first boot that survives `RmInitAdapter` on one, *"because nothing will surface this incidentally."* `RmInitAdapter` has since been survived many dozens of times, and a search of the log, the committed bench evidence and every design document finds no such boot. Combined with the core having **no reset entry point of any kind**, a guest `modprobe -r nvidia; modprobe nvidia` is an untested path against a core that cannot reclaim.

9 How to read the code

9.1 Entry points

If you want to understand...	Read
the whole system, fastest	`README.md`, then `ARCHITECTURE.md` — but read §10 of this paper first, because both are stale in specific ways
what a "process" is and why	`crates/kayfabe-core/src/project.rs` — the pure derivation
the source of truth	`crates/kayfabe-core/src/rmgraph.rs` — the refcounted graph and the parked-fact machinery
the transactional apply	`crates/kayfabe-core/src/gpu.rs`, `Gpu::apply` → `Spine::apply`
the forwarding entry points	`crates/kayfabe-fwd/src/lib.rs` — start at `route_doorbell`, then `plan_doorbell`; the ring gate itself now lives one crate over, at `kayfabe-isolate` `VerbPlan::gated_doorbell`, with its compile-fail pin in `crates/kayfabe-isolate/tests/ui/`
the faked GSP	`crates/kayfabe-gsp/src/lib.rs` (the module map and the five seams), then `boot.rs` — `BootPhase`×`QueueState` is the whole boot protocol; the test-side guest is `tests/src/gspworld.rs`
why the project is not finished	`traces/cudart_ga106/README.md` and the commit messages after it (§8.5), then §11 of this paper. ⚠ The README is one rung stale at this pin and the commits are not
what is in flight and in what order	`docs/design/RESUME_HERE_2026_08_15.md` — including its own supersession banner and its §6, which lists three corrections that invalidate phrasings across the whole tree
what the hypervisor actually runs	`crates/kayfabe-qemu-raw/src/shim.rs` and `qemu/hw/misc/nvkvm/nvkvm.c` — the Rust `staticlib` and the C QOM shim that links it
the host side	`crates/kayfabe-isolate-host/src/rm.rs` — the real RM verbs, and `CeEvidence::copied()`, the predicate the whole execution plane is accepted against
the GSP → core bridge	`docs/design/gsp_core_bridge.md` §2.7 *first* (it is the finding that changed the roadmap), then `crates/kayfabe-rmrpc/src/lib.rs` `translate` and `BridgeRefusal` — the refusal enum is the honest map of what is not modelled
the concurrency design	`crates/kayfabe-rt/src/lock.rs` (ranks) then `device.rs` `verb_op` — the whole plan/execute/commit shape is one function
the `unsafe` surface	`ls crates/kayfabe-linux-raw/src/*_unsafe.rs` — that is the complete list, by construction
what a test is supposed to look like	`tests/tests/l1_mean.rs` (the arbiter) and `tests/tests/miss_taxonomy.rs` (the sharpest single property)

9.2 Conventions

- `*_unsafe.rs` — any file using the `unsafe` keyword is named this way, and the gate greps whole words so writing *about* the rule (`unsafe_code`, `_unsafe.rs`) never trips it.
- `plan_* / commit_*` — the two halves of the verb seam. `plan_` runs under locks and issues nothing; `commit_` runs after re-acquisition and re-validates.
- `route_* / exec_*` — the route/act split (structural rule R4): resolve identity against device-wide state, then

act on one process's state.

- Test names carry their provenance: `t12_`, `t13_`, `t14_` are C-bug regressions; `cb*` are the regression matrix rows; `b1_`–`b6_` are security boundary cases; `i1`–`i4` are the isolation properties; the marker *Mutation-gate kill* in a doc comment names the mutant a test exists to kill.

9.3 Which design documents to read, in order

1. `docs/design/testing_doctrine.md` (321 lines) — read this *first*, even before the architecture. It is the shortest path to understanding what the project believes evidence is.
2. `docs/design/core_state_and_consolidation.md` — the reviewed per-crate state and the L1 hand-off contract.
3. `docs/design/core_security_threat_model.md` — the attacker model and the I1–I4 properties.
4. `docs/design/l1_concurrency.md` §12 and `docs/design/l1_os_shell.md` §14 — the contact logs (the two documents run to 10 265 lines between them). **Read these rather than the summaries**; they are where the design was found to be wrong, which is the part a summary cannot carry.
5. `docs/design/RESUME_HERE_2026_08_15.md` — **read this before any other status document.** It is the ordering rather than the record, it opens with its own supersession banner naming what a later rung invalidated in it, and its §6 (*"how to read anything written before today"*) lists three campaign-wide corrections that invalidate earlier phrasings across the whole tree — including that **four different things in this project are called "drain"**, a misattribution that propagated through three rungs.
6. `traces/cudart_ga106/README.md` — the live blocker's evidence. ⚠ **And it is one rung stale at this pin** (§10 row 41): read the commit messages after it, not it alone. ⊘ **As of `56dc01f3` it is several rungs stale**: the wall it documents was refuted the same afternoon (row 44). The commit messages `dbc6c953` → `56dc01f3` are the current record and they carry their evidence verbatim.
7. `docs/design/execution_plane_increments.md` and `docs/design/boot_measured_2026_08_01.md` — the dated, revision-stamped boot logs. ⚠ **Both predate first compute** (last touched 2026-08-11 and 2026-08-01) and describe walls that have since been crossed. They remain the best record of *how* a wall gets characterised and of at least three sections marked **REFUTED** that were kept rather than deleted — *read them for method, not for status.*
8. `docs/design/guest_blast_radius.md` and `docs/design/multi_tenant_isolation_assessment.md` — the security position on the thesis. Read the banners first: neither switched on a GPU for its reasoning.
9. `docs/reference/` — measured ground truth, kept out of the design documents so a wrong fact is corrected in one place. Note the version caveat in `rm_semantics_measured.md` §0 before quoting any number from it.
10. ⊘ **Do not read** `docs/design/crate_maturity_map.md` — §10 row 29.

★ **ADDED 2026-09-11 — the list above is three weeks old, and four documents now sit ahead of most of it.** Read these first, in this order:

1. `docs/design/PLAN_barrier_driven_refresh_w434.md` — **the current architectural plan**, with the owner's rulings verbatim. ⚠ **Read its own** ⊘⊘⊘ `CORRECTED` **block first**: the plan's original unification hypothesis was wrong, and the correction is what makes the count three rather than one.
2. `docs/design/uncovered_page_table_writes_w452.md` — the source audit of what the three points do *not* cover, and the one-bit predicate that generates the covered set mechanically. Read to the end: the correction there withdraws one of its own rows.
3. `docs/MILESTONE_raw_client_under_constraints.md` — the current measured state, and it is the rare status document that **lists what it does not claim.** ⚠ Its `inline_exceptions = 0` column is a grep artefact (§10 row 51).
4. `docs/design/what_i_think_is_rotten_w433.md` — an honest assessment of the tree's failure mode, written because the owner asked for pushback. It is the shortest route to understanding why a line count in §5 is not a capability claim.

⊘ **And** `RESUME_HERE_2026_08_15.md`**, item 5 above, has three successors** ending at `RESUME_HERE_w417.md`, which is itself 40-odd commits stale: its open question was answered two rungs later and its framing superseded a rung after that. **There is no current resume document.** The commit ledger plus the four files above is the handoff.

★★ **One rule that applies to every document in that list, and the project states it about itself.** Design documents here open with a dated `STATUS` block — and the convention's own limit is written down: **a `STATUS: LIVE` header certifies that the document has not been *retired*, never that its body is *current*.** Several headline documents carry `LIVE` and have not been touched since 2026-07-31, while the campaign that produced every result in §8.3 through §8.5 ran from 2026-08-11 onward. ⇒ *Treat "open" in an untouched document as* ***open-and-unrevisited****, not open-and-rechecked* — this paper found one item that had been resolved for two weeks and was still printed as unresolvable in three places (§10 row 36).

10 Claims corrected while writing this paper

Each of these is a claim that this paper checked against the tree and found stale or wrong. They are listed because the list is more useful than any individual correction: it is a measurement of the drift rate described in §8.21. None is a defect in the code; all are defects in descriptions of the code.

Rows 1–14 are from the first revision, at `0b78102`; rows 15–20 from the second, at `6ce8599`; rows 21–26 from the third, at `b612a91`; rows 27–34 from the fourth, at `f33e1c8`; **rows 35–42 (and 41b) are new at** `5c367a38`; **rows 43–45 were added after this document was pinned, at** `56dc01f3`**, without a new revision being written** — they correct claims in the text above rather than in a predecessor, and rows 43 and 44 are this paper's *headline* claims. **Rows 46–52 were added on 2026-09-11 at** `5f323dd5`, three weeks after the pin and again without a new revision; five of the seven are claims that went stale, and **two (rows 47 and 51) are reasoning defects rather than drift**. **Eleven of the first fourteen were fixed or superseded within four days**, and each row says which — that is the loop working, and it is the single most encouraging measurement in this paper. Row 9 is the instructive exception: it was fixed *in one document*, which then recorded four further documents still citing the retracted number and declared them out of scope.

★★★★★ This revision's rows are a different species, and the difference is the most important thing in this section. Every previous row was a claim that *went* stale — true when written, overtaken later. **Rows 35, 36 and 37 were false at the moment the previous revision was published**, by five days, eight days and zero days respectively. The device enumerated on a stock guest on 2026-08-08; that revision is dated 2026-08-03 and its wall analysis described a boot that had already been passed by work in the same repository.

⇒ **The failure was not update latency. It was scope.** The previous revision's own method section states the rule that would have caught it — *"every claim carried over from the previous revision was re-measured rather than trusted"* — and applied it to line counts, crate inventories and two figures supplied in its brief, **but not to the sentence the document was built around.** The headline is exactly the claim nobody re-derives, because it is the one everybody already believes. *This revision's headline should be read with that in mind, and so should this sentence.*

Six of this revision's seven rows are defects in this paper's own previous revision, and one of them is — for the *fourth* consecutive time — the sentence that revision was built around. That is the point rather than an embarrassment to be minimised: a whitepaper is a description of code and decays at the same rate as any other description of code. **The claims that decay fastest are the ones stated most confidently**, because confidence is what stops the next reader re-checking them.

	Claim, and where	What the tree says
1	*"*`handle_doorbell` *is the ONLY caller of* `RmBackend::ring_doorbell`*"* — `ARCHITECTURE.md:112`, `core_completeness_gate.md:119`, `core_state_and_consolidation.md:174`, `fwd/src/lib.rs:15`	**False as stated.** `ring_doorbell` has exactly one call site and it is inside `Worker::execute`, two hops away. `plan_doorbell` had *two* callers: `handle_doorbell` and `SharedDevice::doorbell` — and the L1 shell path, the one a real guest MMIO write takes, goes through the second and **never enters** `handle_doorbell`. The safety property survived; the cardinality claim did not. **Superseded at** `634b253`: the gate moved into `VerbPlan::gated_doorbell`, the sole constructor of a `#[non_exhaustive]` variant, so the property no longer rests on a cardinality claim at all — it rests on E0639. *A correction that the code then made unnecessary is the best outcome on this list.*
2	`kayfabe-abi` is a *"stub (shape only)"* — `README.md`, `ARCHITECTURE.md`, and its own `Cargo.toml` description	**False**, and **FIXED SINCE** (`e6ba19e`). 7 455 lines of source, a working offline generator, 11 generated structs, a 13-method version-dispatch decode surface, 2 272 lines of oracle tests. All three sites now carry struck-through corrections. *Listed here because the correction is the evidence the house style works.*
3	*"the implementor is the L1 shell* (`kayfabe_rt::SharedDevice`)*"* — `ARCHITECTURE.md:38--43`, `kayfabe-vmm/src/lib.rs:816`, `core/src/gpu.rs:1012`	**No** `impl Device for SharedDevice` **exists.** The only two implementors are test types that wrap an `Arc<SharedDevice>` and forward. **Still true at** `6ce8599`, but **the descriptions are FIXED**: both `ARCHITECTURE.md` and `kayfabe-vmm/src/lib.rs` now state the absence in bold rather than implying the shell already does it.
4	*"283 tests"* — `README.md`, `ARCHITECTURE.md`	**515** measured on 2026-07-27; **547** at this revision. **FIXED SINCE** (`e6ba19e`) — and fixed in the right way: both files now say "in the **500s** as of 2026-07-27" rather than carrying a fresh exact number that would rot again in a week. *Deleting a number is a legitimate repair.*

	Claim, and where	What the tree says
5	*"four standing CI gates"* — `README.md`	**Six jobs; twelve steps in the `stable` job alone.** The list of four omitted the VMM-vocabulary gate, the GPA-accessor gate, three unsafe-containment asserts, the KVM reached-count floor, the fuzz-compile job, the slow job and the mutation job — and, since it was written, the generation-name gate. **FIXED SINCE**: the phrase no longer appears in `README.md`.
6	*"TSan (28 threaded tests, 0 races)"* — `README.md`	28 was the first campaign. The four TSan targets contain **68** `#[test]` functions at this revision (65 at the previous one). The "0 races" half is not disputed here — it was not re-run. **FIXED SINCE**: `README.md` now names 28 as "the first campaign" and says it has not been re-run.
7	*"the two measured-slow tests"* — `README.md`	Four to six, depending on how you count; the `g10_*` capped pair joined later. **FIXED SINCE**: the README now says membership "has grown since it was measured" and points at the `skip_slow!` call sites as the authority instead of quoting a count.
8	*"the conformance suite (23 files)"* — `ARCHITECTURE.md`	**29** files in `tests/tests/` at this revision, plus **6** under `crates/*/tests/` (28 + 5 at the previous one). **FIXED SINCE**: the count was removed rather than updated.
9	`core_mutation_gate.md`'s 91% threshold and its *Reproduce* block	Both described the pre-2026-07-27 four-path scope. **FIXED SINCE** (`e6ba19e`): the document now opens with a banner saying so. *But the correction propagated only one hop* — the same document records that the 92.44% and the 91% floor are "still cited as settled" by `README.md`, `ARCHITECTURE.md`, `l1_architecture_summary.md` and `core_state_and_consolidation.md`, and reports them as out of its own edit scope. A correction that names its own unpropagated instances is the honest form; it is still four live instances.
10	`kayfabe-mmu::walker`'s own doc: *"Also here (skeleton this milestone): the GMMU walk algorithm"* — `mmu/src/lib.rs:30`	There is no algorithm: one trait, one enum, 41 lines, zero constructors, zero implementors — **unchanged at** `6ce8599`. **The doc was FIXED** first: the module comment carried a struck-through correction naming what it actually contained. *The code did not move; the claim did.* ★ **And then the code moved too** — see row 30: the module is now 863 lines with six public functions. *This is the only row on this list that was resolved from both ends.*
11	`kayfabe-gsp`'s doc: RPC decode *"against `kayfabe-abi`'s generated message layouts"*	**Was false; is now true.** At the previous pin `kayfabe-gsp` did not depend on `kayfabe-abi` and nothing did. At `6ce8599` the manifest lists it and five of the eight modules use it. *A doc claim that was aspirational and became correct is still a drift instance — nothing gated it either way.*
12	*"`Isolate/RmBackend/Vmm` trait-only (mock-implemented)"* — `ARCHITECTURE.md:31,56`	True for `Isolate/RmBackend`. **False for** `Vmm`: `crates/kayfabe-vmm-kvm` is a real KVM adapter with a real memory plane. **FIXED SINCE** — `ARCHITECTURE.md:31` now carries the struck-through original.
13	*"~14 months of quirk capture"* in the C's architecture and ABI documents	Git: 2026-05-24 to 2026-08-01. **Ten weeks, 753 commits** (it was "two months, 739" when this row was first written; the C is still maintained as the oracle, so the number moves).
14	The framing "7B LLM at 63 tok/s" as a headline C result	63.4 is a Mode-1 A/B endpoint. The audited parity figure is **0.97×** (**66.0 guest vs 67.8 host**). The underlying milestone note still records the pre-fix **20 tok/s** and was never updated. All three numbers are real; they are different builds.
15	`kayfabe-gsp` is a **stub** — `README.md:161`, `ARCHITECTURE.md:68`, and its own `Cargo.toml` description, which still opens *"SKELETON — the faked GSP…"*	**False.** 3 550 lines, 8 modules, 24 dedicated tests, an independent 1 133-line guest model, and a composed run inside the arbiter suite. The two markdown rows at least carry an `[unverified]` hedge telling the reader to read `lib.rs` instead; the `Cargo.toml` description carries none. *This is the same shape as row 2, one crate later, four days after row 2 was written.*
16	`mode2_gsp_port_plan.md` §11-O7 marked **RESOLVED**: *"`GSP_RUN_CPU_SEQUENCER` is not implemented, do not emit it"*	**Backwards for our bench, and the status has been withdrawn** (`7af23cb`). That is 610's answer. At 580 it is fully implemented, dispatched, and first on the bootup-window allowlist. The whole plan's citations were taken from 610.43.02 while the bench runs 580.159.04; eight claims changed when the matching tree was vendored. See §8.10.

	Claim, and where	What the tree says
17	The plan's §1.3: *"the guest declares its own geometry"*, presented as the highest-leverage design choice	**610 only.** At 580 `MESSAGE_QUEUE_INIT_ARGUMENTS` has four fields, not nine, and the queue geometry is compile-time. On the bench nothing is negotiated, so the argument the design leaned on does not apply to the machine it will first run on.
18	**121** bare `ogkm:` citations in the crates' doc comments, 96 of them in `kayfabe-gsp`	**Every one is a 610 path with a 610 line number**, and several name files that do not exist at 580 — the whole `msgq` library moved directories between the tags. The version-tagging rule that fixes the class is normative in the plan and **not applied to the code**; the CI grep that would prevent the recurrence is specified and not written.
19	**This paper's own previous revision:** *"*`kayfabe-abi` *and* `kayfabe-gsp` *are disconnected islands"*	**Half false at** `6ce8599`**.** `abi` has a consumer now (`gsp`); `gsp` still has none outside `tests/`. Re-checked against the manifests, not inherited from the caption.
20	**This paper's own previous revision:** Figure 2, *"all 65* `[dependencies]` *edges are shown"*	**The manifests had 66.** The edge `tests` → `rt` was omitted, in a figure whose stated method is "derived from the manifests". A drawn matrix is exactly as ungated as a sentence.
21	**★★ This paper's own previous revision, its headline:** *"*`RpcCommand` *has zero references anywhere in the tree outside the crate that defines it … the critical path is the bridge, not the crate"*	**False at** `b612a91`**.** `kayfabe-rmrpc` (`76a0077–3a1704f`) names it in three files and is a production crate depending on both `gsp` and `core`; a CI grep now enforces that it is the *only* crate permitted to. *The claim was true when written and was invalidated by the work it identified as necessary* — which is the best possible outcome for a criticism, and still a stale sentence four days later. The successor claim, stated so it can be checked the same way: **nothing but** `tests/Cargo.toml` **names** `kayfabe-rmrpc`.
22	**This paper's own previous revision:** *"Checked-in proptest failures:* ***0*** *—* `find` *for* `proptest-regressions`*: nothing exists"*	**Wrong when printed.** Four `*.proptest-regressions` files existed at `6ce8599` holding **18** seeds; there are 20 now. The row's stated method was a `find` that would have returned them. *This is the only correction on this list that is a measurement error rather than decay*, and it is the one that most deserves to be here: the method section claimed a command was run, and the number it reports is not the number that command produces.
23	**This paper's own previous revision:** the GSP-S1 row, *"Nothing. It is a design-doc invariant with no code behind it …* `grep` *finds the variant nowhere"*, and the risk-list item calling it "the highest-stakes sentence it has"	**Fixed since** (`250b78c`). `GspFault::ElementCountOutOfRange` exists at the write site, unconditionally, plus a receive-side range check and a derivation cross-check. Noted here rather than silently updated, because this paper named the gap and the gap closed — that is the loop working on the paper's own output.
24	`gsp_core_bridge.md` §4.3: *"known-and-inert versus unknown — there is no third state"*; §5.2's B2/B4 non-vacuity arms; §6's Memory-alloc row; §7.1's assumption that `SET_PAGE_DIRECTORY` carries the PDB	**All corrected by the build, in the commit that hit them.** There are at least four dispositions, not two. Three non-vacuity arms were unbuildable at the stage that specified them and one was *wrong* (refusing a second PDB requires state a stateless bridge cannot have, and `RmGraph` had already decided the other way with an argument). The Memory row is unbuildable twice over — the address fields are `[OUT]` in the guest→GSP direction, and its only consumer is driven by an event with no producer. §7.1 is settled against the design: see §8.7.
25	`mutants.toml`'s own header, and every "excluding scaffolding" figure this project has published, including one in this paper's previous revision	**★★ The file was never read** (`b64a3f5`). It lived at the workspace root; cargo-mutants reads `.cargo/mutants.toml`. Every exclusion was inert and every scaffolding-excluded score was a hand subtraction. Proven, not inferred: `--list` targets 3 760 → 2 667, scaffolding mutants 361 → 0. *An inert configuration file is the same failure as a stale sentence, with worse consequences.*
26	*"121 bare* `ogkm:` *citations, and the version-tagging rule is normative"* — §10 row 18, at the previous revision	**The rule is being followed by new code and the backlog grew.** 56 tagged citations now exist where there were none; bare untagged ones went **121 → 159**. The CI grep that would stop it is still specified and not written, and it is now the difference between a rule and a wish. **★ FIXED SINCE**: the grep exists — an *ogkm-version-tag* gate, ratcheted at 161 untagged and exact in both directions. *This paper named the gap twice and the gap closed.*

	Claim, and where	What the tree says
27	★★★ **This paper's own previous revision, its headline and its warnbox:** *"No part of the Rust codebase has ever driven a real GPU … there is no binary in the workspace … no isolate process has ever been spawned … no guest driver has ever posted to any of it"*	**Every clause is now false**, and this is the third consecutive revision whose headline was the falsified claim. A stock 580.159.04 module binds to the emulated device and 92–95 commands decode per boot; three binaries exist plus a `staticlib` in a real QEMU; a sandboxed isolate is spawned at `t+42 s` in response to the guest's action; and a real host copy engine has moved 4 096 bytes for a request this core decoded. ⊘ **And the replacement is deliberately a conjunction, not a sentence** (§8.13): the boot still fails, there is still no `cuInit`, and the two hardware results have not met.
28	**This paper's own previous revision:** *"Defect #57 … trips roughly 1 in 6 runs under race. This is why the QEMU adapter has not been started"* — and two more sites	**False twice over.** #57 was closed at `020790c`, and *the shape of its resolution is the part worth keeping*: the violation was not the lock split or a syscall in a critical section but `Arc` ownership — **a `Drop` is a call site**. And the QEMU adapter is built through stage Q5, compiled into two real hypervisor binaries and booted against. ⚠ **Do not conflate #57 with §8.17**: different crates, different defects, different lessons.
29	`docs/design/crate_maturity_map.md`, cited by this paper as the cross-check for its own maturity column	**Wholly stale, and this paper stopped citing it.** Pinned at `d65eb75` ("496 tests green"); lists 16 crates against 24; calls `kayfabe-gsp` a **34-line skeleton** against 5 056 lines; records #57 open five days after it closed; and names neither QEMU crate. *A document whose entire job is to say what is built is the worst possible one to be stale*, and it is the sharpest instance of §8.21 in the tree.
30	**This paper's own previous revision:** *"the* `walker` *module is 41 lines containing one trait and one enum, with no walk function of any kind"*; and *"still zero production implementations … that remains the standing proof that a real one will need no core edits"* (`kayfabe-arch`)	**Both closed, and both by this paper naming them.** `walker` is 863 lines with six public functions and `FbRead` has two implementors. `Arch` has three production implementations — and the "standing proof" was settled *partly against itself*: Ada needed no seam, Hopper needed one, and the seam it needed was in `kayfabe-arch`. *A standing proof that is never executed is a hypothesis.*
31	★ **The brief for this revision:** *"84 of 106 classified DATA/ACT with citations"*	**The number 84 does not appear in any of the source documents**, and re-derived from `gsp_control_classification.tsv` directly the figures are **79** DATA-or-ACT and **81 of 106 (76.4%)** bucketed with a citation, cross-checked two ways (bucket column and confidence column, HIGH 78 + MEDIUM 3). **25 are UNKNOWN.** The measured numbers are what this paper prints.
32	★ **The brief for this revision:** *"every* `dlen=0` *row checked against a real GA106 is contradicted"*	**Too strong, and the exceptions are the interesting part.** All eleven were checked and **nine** are contradicted. `0x20800a70` has `psize = 0` so nothing could have failed to be captured; `0x20800a4c` genuinely answers zero — **and nothing about the row says so**, it being byte-identical to the nine that are wrong. *Believing the empty rows would have been right once and wrong nine times for the same reason*, which is a strictly better argument than the blanket one. §3.1.
33	**This paper's own previous revision:** *"there is no second machine, no second GPU generation, and no continuous hardware testing"*	**Two thirds false.** Work has run on four machines and three distinct physical GPUs, including *two different* GA106s and a GA102, and three cross-machine reproductions are load-bearing rather than decorative (§8.20). **The third clause is still true.** ⊘ And every *boot* result is still one GPU model, one driver family, one guest.
34	**This paper's own preamble**, and it was introduced by this revision	★ **The glyph ⊘ was being dropped silently from the PDF.** The preamble carries an explicit fallback for ★ and ⚠ with a comment saying a dropped marker is *"a lost signal, not a cosmetic defect"* — and the marker added to the vocabulary since then hit the identical trap, visible only as a `Missing character:` line in a log nobody reads. ⊘ is this project's marker for *"this does NOT establish X"*, **so a dropped one inverts the sentence it qualifies**. Found by reading the build log rather than the output. *The mechanism generalises past this document: a fallback table is a hand-list, and a hand-list is a smaller true statement.*

	Claim, and where	What the tree says
35	**This paper's own previous revision, three claims at once:** *"the boot does not complete"*, *"there is no* `cuInit`*, no* `cuCtxCreate` *and no compute in Mode 2 anywhere in this codebase"*, *"on every boot to date the device reports* `doorbells: 0 arrived`*"*	★★★★★ **All three false, and the first two were *already* false when the previous revision was published.** `nvidia-smi` enumerated the device on a stock guest with the probe empty on **2026-08-08** — *five days before* that pin's date. Compute landed 2026-08-14 (`CUP3_VAL=43`) and 2026-08-14 (`cup8`, `bad=0 maxerr=0`); doorbells were served from 2026-08-08. ⊘ **Note what this means about the failure mode**: the paper was not slow to update, it was *wrong at the moment of writing*, because the wall it described had been passed by work the writer did not check for. *The revision's own method rule — "the brief is a source like any other and is re-derived, not obeyed" — was applied to two numbers and not to the headline.*
36	**This paper's own previous revision:** *"§11-O7a … is the one item in the whole component with no proposed resolution, because the evidence that would settle it is not in any open-source tree"*	**RESOLVED 2026-08-06 — before the pin this paper carried it at.** CPU-assist events are `emit-never` on *both* driver versions, and the premise was wrong twice over: the version split is not "RPC versus local" (both begin at a GSP→CPU event), and the evidence is *not* in closed firmware — a real GA106 sends exactly one such event per bring-up, `rpc_len = 6328`, decodable from a committed trace. ★ **The general form is the costly one**: an item marked unresolvable *because the evidence is closed* deserves one more read before it is believed, since the open/closed boundary is itself a claim.
37	**This paper's own previous revision:** *"the multi-process property — the founding requirement of the whole rewrite — is unverified on hardware … #14 cannot yet be shown fixed on a GPU"*	**False, and it is the single most important structural result the project has.** Two *concurrent* guest CUDA processes both computed on 2026-08-14, including on a staggered arm where the second started while the first was demonstrably inside `cuCtxCreate`. ⊘ And the correction has a correction: the property that *does* still fail is the **sequential** one — a second context after the first's teardown in the same boot hangs — which is the opposite of what everyone predicted, and is the C artifact's own limitation inherited rather than fixed. ⊘⊘ **AND THE CORRECTION'S CORRECTION HAS ONE, five hours later: see row 43.** The sequential property **passes** for three processes as of `56dc01f3`, and the C's limitation was *not* inherited — the port's cause was its own.
38	**The repository's own claim-ledger gate**, which the previous revision reported green with all three bars holding	★★★ **RED at this pin, and by a wide margin**: **1 166 unattributed against a bar of 381, 90 conflated against 66, 36 bare-hardware against 17**, exit 1 ([measured] here, §7.3). The bars have not been touched since 2026-07-31 while the tree tripled and forty directories of committed bench artifacts entered a universe derived from `git ls-files`. ⊘ **Which half is regression and which is denominator, this paper did not measure** — and that ambiguity is itself the finding: *a ratchet whose denominator moves is not a ratchet.*
39	**The CI job table this paper printed at the previous revision**, listing `clippy -D warnings` and `fmt --check` among the things CI asserts	**Both are RED on** `master` and have been for at least a week. `fmt` fails on **40 files** ([measured] here); `clippy` fails with 5 errors at recorded sites. ★ **And the** `fmt` **red is not attributable**: `rust-toolchain.toml` pins `channel = "stable"`, a *moving* channel, and the project's own bench notes record this toolchain reformatting 62 files of untouched `master`. **A gate that can go red with no commit is a gate whose red carries no information.**
40	**The dependency figure this paper re-derives every revision**	**Its row ordering had silently stopped being topological.** `qemu-raw` acquired a dependency on `shell`, which sat *below* it, so drawn in the previous order the matrix would carry one dot above the diagonal — reading as a cycle where there is none. Edges moved 126 → 134; members unchanged at 24. ⊘ **The instructive part is the failure mode**: the figure's correctness is a *visual* property, so the defect would have been one dot out of place in a 24×24 grid with nothing in the build objecting. *Re-derive by script; never edit an edge list.*

	Claim, and where	What the tree says
41	**This paper's own §8.5, three times, while it was being written**, and the committed trace README beside it	★★★★★ **The live blocker moved five commits in one afternoon**, and this paper described it wrongly at three of them: *"the cause is unknown, five hypotheses refuted"* → *"the cause is one control we refuse, fix landed, unbooted"* → *"the wall is broken"* — and the true statement is that **the initialisation wall broke and the model still produces zero tokens.** ⊘ The trace README committed alongside is, at this pin, *one rung stale*: it names a prime suspect the very next commit retired by measurement. **Listed as this paper's cleanest live specimen of §8.21** — the document was correct when written, is wrong now, says nothing about it, and is the artifact an outside reader is most likely to be pointed at. ★ *And the third version was wrong in the direction nobody audits for: it was too optimistic because it graded a milestone on a proxy.*
41b	**The trace README, restated as the general rule**	**One rung stale relative to the commit beside it.** It names a suspect (*"the prime suspect is now the one thing I excluded"*) that the very next commit **retired by measurement**, having found the real chain. ⊘ Listed not as a criticism but because it is this paper's cleanest live specimen of §8.21: **the document was correct when written, is wrong now, and says nothing about it** — and it is the artifact an outside reader is most likely to be pointed at.
42	**This revision's own build**, and it is listed because the paper's own doctrine demands it	★★★ **The build exited 0, wrote a 66-page PDF, and contained six LaTeX errors and seven undefined macros.** A bare `^` inside a `\code{}` span — copied verbatim from a bench log's anchored metric — put TEX into *math mode*, which then swallowed a list item, dropped an em dash, and typeset three bullets of the executive summary in the monospace font. Separately, `\text{}` is `amsmath`'s and only `amssymb` was loaded, so seven arguments typeset while their macro vanished. ⊘ **Under** `-interaction=nonstopmode` **every one of those is a line in a log and nothing else**: the exit status is 0, the PDF exists, and it looks finished. *This is the identical shape as everything in §8.21 and §7.7 — an instrument whose failure mode is to look like success — arriving in the artifact that describes them.* The build is now checked for `^!`, `Undefined control sequence` and `Missing character` in its log, all three at **zero**, and that check is the only reason this row exists.
43	★★★★★ **This paper's own headline blocker, in six places** (the front-matter pin, the executive summary, §8.13 twice, §8.22 and row 37 above), and in Figure 4: *"a second context sequentially in one boot still hangs at* `cuCtxCreate`*"*	**False, 4 h 55 min after this document was committed.** [measured] `w367`, rev `cca2eb4b`, 2026-08-20, real GA106: three sequential guest CUDA processes — first, second with no `nvidia-smi` between, third after `nvidia-smi` — all `TORCH_OK avail=True sum=4.0`, all **8/8** completions observed, all ringing *identically* 104 `Ce` + 403 `GrCompute` doorbells; host `dmesg` 0 `NVRM` 0 `Xid` (§8.11). Two fixes: an **orphaned framebuffer join** — held by the per-device store, named by no address table, live or dead — is now given back, with `live > 0` still refused *by name* 132 times; and the takeover cap was **re-keyed** from *frame* to `(phys, taking va)` after the old key left our own leaves unbacked and **raised a host** `Xid` **31**. ⊘ **The claim that survived is the narrow one**: a *fourth* process still fails, above the `ioctl` boundary, cause unknown, $n = 1$ boot. ★ *And the inherited-from-the-C framing in row 37 was wrong: the port's defect was the port's own.*
44	★★★ **This paper's second headline, §8.5 and §11 Q1:** *"the next wall is a different subsystem: the object graph"*, and *"the sharpest single line is* `tsg = mid-miss(h0xa, wrong-kind(Device))` *— a named, specific defect in the context-promotion path"*	★★★★★ **Refuted *three minutes* after this document was committed** (`129dd316`, 2026-08-20 15:39), by a negative control that was two log files already on the bench. Same revision, same binary, passing boot vs failing boot: `PromoteFault::UnknownContextObject` **4 vs 3**; `RmGraphError::FreeUnknown` **14 vs 10**; unmapped alloc classes **6 vs 4**; `wrong-kind` census lines **2 vs 2**. **Every element of the wall is more numerous in the boot that works.** `wrong-kind(Device)` is not a defect at all — a channel's parent is legitimately a TSG or a Device — and the line appears byte-identical in a boot eleven days older (`dbc6c953`). ⊘ **This row is a method defect, not drift**: the rule that would have caught it was written down by the same author, in the same session, eleven messages earlier. ⇒ *Before naming any refusal a wall, run the control — mechanically, first, always. A census of a failing run is not an experiment.*

	Claim, and where	What the tree says
45	**This paper's own closing note and §9.3**, at the moment it was committed	**Internally inconsistent with its own §8.5 at the same pin.** The closing note describes the open item as *"the CUDA runtime does not initialise, and the fix is unbooted"* while §8.5 of the same document reports that wall broken and booted ($n = 2$ workloads); and the reading list points at a trace `README` the paper itself flags as one rung stale. ⊘ **Listed because it is the cheapest species on this list and the only one no external event caused**: two passages of one document, written hours apart, disagreeing — exactly the failure §8.21 attributes to *other* people's design docs. *Found while correcting rows 43 and 44; not found by any gate, because no gate covers prose.*
46	**§8.13 and §8.22:** *"No LLM has ever produced a token in this guest"*, and *"Nothing anyone would actually run works"*	**False from 2026-09-06 and decisively false from 2026-09-09.** `LLM_TOKENS=16` with degenerate text on the first date; on the second, rev `c8fe67d6`, `W392_OUTCOME=(P)` with the **output byte-identical to the same-boot CPU oracle** and host `Xid` 0. ★ **The cause of the garbage was ours**: a `!is_remap` guard meant a re-mapped joined row was never revoked, so the stale binding kept translating and the guest read the *previous* tensor. ⊘ **The replacement sentence is not a victory lap**: 45 s per token, graded on a count the project's own requirements mark RED, $n = 1$–2 boots, never run under the three blocking constraints (§4.4).
47	**§6.1:** *"round 6 of the #14 investigation measured zero occurrences of both invalidate transports … a design that had encoded 'read at the invalidate' as the Layer-2 rule would have had no synchronisation point at all on the path that matters most"*	★★★ **The most consequential row on this list, because a normative rule was built on it.** Both zeros are real and both are artefacts: `INVALIDATE_TLB fn=200` cannot fire (its caller needs `IS_VIRTUAL`), and `MEM_OP` is *UVM's* transport, not RM's. **The census had no row for the transport RM uses**, a raw BAR0 store. ⇒ *A census over transports is only as complete as its list* — the paper's own recorded lesson, applied to the paper. There are **three** synchronisation points and the architecture now uses all three (§4.4).
48	**§8.4, §8.13, §8.22:** the doorbell trap as the launch floor, and the device's blocking model generally	**Restructured since 2026-08-21.** The doorbell defaults to *deferred* with no inline fallback; the GSP command queue is serviced off the vCPU (**561** RPCs, `w447`); BAR1/BAR2 default to *untrapped* (BAR1 traps **88 193** → **91**). The 86.7 ms breakdown is a correct measurement of a design that has been replaced, and **no successor breakdown exists.** ⊘ **And the wait did not follow the work**: `worst_trap = 1.78 s` at the GSP queue head, unmoved by two lock fixes.
49	**§5:** `kayfabe-mmu` listed as **BUILT** with `gpga.rs` named beside the walker	`gpga.rs` **has zero production callers**, and so does `refresh.rs`, whose own doc calls itself *"the model of that rule … a pure fixpoint over mocked page tables."* Both are **DESIGN-ONLY**. ⊘ **This is the paper committing the defect it diagnoses**: a line count cannot see reachability, and the tree's own assessment of itself names *"mechanism without consumers"* as the single shape of its rot. `promotion.rs` (332 lines, zero referrers) has since been deleted; `dbtable.rs` (316) survives, unwired.
50	**§11 Q2:** *"where does a slow host RM verb run"*, with three facts offered as closing the obvious escapes	**Answered, and two of the three facts were wrong.** The reply need not be synchronous (the guest *polls*); a second thread posting is not an inversion once the queue leaves the state lock. What survives is the third: a host RM verb is one call with one latency and no budget splits it, which is why `verb_op`'s execute phase is still inline and **counted** (`inline_exceptions = 24`) rather than moved.
51	**The milestone document §4.4 reads, and this pass's own first draft:** *"declared inline vCPU blocking: 166 → 0"*	**The 0 is a grep artefact and the real value is 24.** The ledger's `grep -o 'inline_exceptions=[0-9]*'` piped to `tail -1` was matching the harness's own goal string, `(target: inline_exceptions=0)`. ⚠ **It was reported to the owner twice and still stands in the milestone document's table.** ⊘ **Second defect in the same metric, and the more interesting one**: `inline_exceptions` fell from 166 while the wait it proxied for did not move at all. *A metric can stop measuring its purpose and keep reporting.*
52	**§8.12's title:** *"the fifth device open wedges the GPU"*	**The ordinal is a sample, not a constant.** [measured] `w423` in the guest, one configuration repeated so the ordinal is the only variable: opens 1–3 pass and **open 4** fails with `EIO` on `openat`, permanently. The driver's own reason is now captured (an internal CeUtils scrubber channel failing adapter init), and the wedge is *not* the LLM's blocker.

Claim, and where	What the tree says

10.1 What this paper could not verify

★ **ADDED 2026-09-11, for the correction pass at** `5f323dd5`**.** What that pass *did* execute, on this box: the line, crate and commit counts re-derived at `5f323dd5`; a read of `qemu/hw/misc/nvkvm/nvkvm.c`, `shim.rs`, `trapwitness.rs`, `kayfabe-rt/src/device.rs` and `crates/kayfabe-mmu/src/` for every structural claim in §4.4; a caller census for `gpga.rs` and `refresh.rs`; and the PDF build. **Nothing else was run** — no `cargo test`, no boot, no GPU. **Every hardware number added in that pass** (`w383`, `w392`, `w393`, `w423–w426`, `w447–w460`) **is read out of a committed design document or commit message and was not re-measured.** ⚠ And two of them are *explicitly* unmeasured: the `w457` lock split is landed and unmeasured, and the `w459` slow-trap histogram **has never run**, so this paper quotes no bucket figures for it.

- ★ **What changed at this revision: some things *were* run, and the list of which is short.** [measured] here, on this box, at `5c367a38`: all line and crate counts; the `#[test]` census; the `unsafe` relaxation counts for both audited crates; the entire dependency edge list (re-derived from the manifests by script, which is how row 40 was found); `scripts/claim_ledger.py --gate` (**exit 1**); `cargo fmt --all --check` (**red, 40 files**); and the PDF build itself. **Nothing else was executed** — no `cargo test`, no `cargo clippy`, no mutation campaign, no boot.
- **Every hardware result in this paper.** §8.3, §8.4, §8.5, §8.17 and §8.18 rest on runs this paper did not perform and could not perform — the box has no GPU. What was checked here is that each claim names a run, a revision, a machine and a committed artifact. **The artifacts were sampled, not audited line by line**; the `cup3` doorbell census and the `cup8` result lines were read from the committed logs directly, the rest were not.
- **The current test pass count, and whether the standing red set is still 7.** The 2 926 figure is a recorded measurement at `9871d315` (2026-08-14), six days before this pin, and the red-set count is one that *"moved and was mis-stated twice"* by the project's own account. [unverified] here.
- ★★★ **Anything in §8.5, which moved five commits in one afternoon and twice *after* this paper had described it.** The `cudart` green is $n = 1$ against a measured 1-in-5 false-negative rate; the fix is capture-derived and pinned to one part and one driver; and the object-graph refusals that replaced it have had exactly one boot. *This is the section most likely to be wrong by the time it is read*, and it is the reason the closing note says what it says.
  ★★★★★ **THIS PREDICTION WAS CORRECT WITHIN THREE MINUTES.** The object-graph refusals were refuted at `129dd316` (§10 row 44) — *by the one-boot control this bullet asked for*, which found them *more* numerous in the boot that passes. ⊘ **Kept as written, because a correct pre-registered doubt that nobody acted on is worth more on this list than a corrected sentence.**
- ★ **Whether the QEMU/KVM backend differential passes.** §8.19 reports that `crates/kayfabe-vmm-qemu/tests/kvm_differential.rs` exists and is the experiment the previous revision said only a second backend could run. **It was not executed here**, so whether the "zero trait changes" half of the agnosticism claim held is [unverified].
- ★ **The current mutation score, deliberately and by the project's own rule.** The most recent campaign to complete is void (§7.7, Lie 5) and the one before it ran against a tree modified underneath it. A bar of 91 is in force; **no clean campaign has completed under it**. There is consequently no defensible score to state, and this document states none.
- **Anything about NVIDIA's 580 driver that this paper did not read itself.** The §8.10 claims about 580-vs-610 are taken from `docs/design/gsp_580_correction_brief.md` and `mode2_gsp_port_plan.md` §14, which state their own limit: only 580.159.04 and 610.43.02 are vendored and were read directly. Seven further driver tags used to narrow the element-layout break interval are **relayed from another agent's probe and were read by nobody in this chain** [unverified]. The predicate the project derived (`major >= 610`) is safe under either reading because the 610 boundary is the directly-verified one; the sentence "580, 590 and 595 are all on the 48-byte side" is not.
- ★ **Whether any teardown / re-acquire boot has ever been run.** No commit, log or design document records one, and this paper searched. **That is absence of evidence and is reported as such**, not as evidence it was never done. (The *other* question this bullet used to hold — the 580 resume handoff — is resolved; see §8.10.)
- **"TSan, 0 races."** Not re-run; only the target list and the test count in those targets were checked.
- **The relationship between the C's 2026-06-16 bare-metal result and the 2026-07-25 one-process-per-lifetime finding.** The memory note flags this itself: the finding was not cross-checked against the bare-metal box, and warns against assuming the C regressed.

- **Whether Mode 2 ever produced any display output.** No positive evidence was found, only the plan. Absence of evidence.
- ★ **Anything about the demand ladder or the CPU-RM recorder beyond its own documents** (§8.6). The classification is a static reading of the driver source, stated as one by its authors; the two numbers this paper prints were re-derived from the committed TSV, but the *arguments* behind each row were not re-checked. And ⊘ **the recorder's observational neutrality is not proven by anybody** — its author says so.
- ★★ **The entire multi-tenancy confidentiality analysis** (§8.18). Every claim in it is [inferred] from driver source; both source documents open by saying no GPU was switched on for them; and the experiment that would settle the central finding — two VMs on one box — has never been attempted by anyone.
- **Whether `memmgrTestCeUtils` is really the next wall.** It is [inferred] from source and **no boot has ever reached it**. The escape-hatch analysis that makes it un-fakeable is a reading, and this paper checked that the readings are cited, not that they are right.

11 Open questions — posed for a reader with no access to this tree

Who this section is for. Someone who knows Linux, kernel internals, NVIDIA's RM object model and GSP firmware architecture, and systems engineering — and who has never seen this project. Each question below is written to be **answerable without our source**: it states what was measured, on what hardware, what has been *ruled out* and by which experiment, and what observation would settle it. Where the decisive evidence is a file in this repository, the question says so and says what is in it.

What would be most useful. Not "have you tried X" — most of the obvious X's are in the ruled-out rows, which exist for that reason. The valuable answers are of the form *"that mechanism does not work the way you have assumed, and here is the source that says so"*, or *"there is a fourth possibility your experiment cannot distinguish"*. **This project's single most expensive recurring failure is an instrument that cannot see the thing it is pointed at**, and every question below is stated with its instrument's limits attached for that reason.

Q1 — Why does `torch._C._cuda_init()` fail with "CUDA unknown error" when every RM control on the path is served?

Where this stands, because it moved three times in one afternoon. The CUDA *runtime* would not initialise in our guest at all: `libcuda` answered every call correctly while `libcudart` returned `3` from its first. **That is fixed** (§8.5) — `cudaGetDeviceCount`, `cudaSetDevice`, `cudaMalloc` and `cudaDeviceSynchronize` all return 0, and `torch.cuda.is_available()` is `True`. **And the model still does not run**: `torch._C._cuda_init()` raises `RuntimeError: CUDA unknown error`, `LLM_TOKENS = 0`.

What the failing boot reports. Host `Xid` **0**; guest `NVRM` line count at the green baseline — **so this is a refusal we issue, not a hardware fault.** The control-serving path refused **nothing**.

⊘⊘⊘ **CORRECTED 2026-08-20** (`129dd316`, `dbc6c953`)**, and the correction removes this question's most attractive answer.** This paragraph used to continue: *"what did refuse, and it is a different subsystem entirely, is our object graph* — `PromoteFault::UnknownContextObject` ×3, `RmGraphError::FreeUnknown` ×10, *an unmapped allocation class* ×4 (`0x70`, `0x208f`) — *and the sharpest single line is* `tsg = mid-miss(h0xa, wrong-kind(Device))`." **Every one of those is baseline.** Run against the boot that *passes*, at the same revision and the same binary, they count **4, 14, 6 and 2** respectively — all *higher* than in the failing boot. `wrong-kind(Device)` is not a defect at all: a channel's parent is legitimately a TSG *or* a Device, and that exact line appears byte-identical in a boot eleven days earlier. **⇒ Do not answer sub-question 3 below about `0x208f`: it is measured innocent by negative control.** What `_cuda_init()` fails on is **unidentified**, and that is the honest state of this question.

★ **And one datum arrived after this question was written that may or may not belong to it.** [measured] `w367`, 2026-08-20: a *fourth* sequential CUDA process in one boot fails with the **verbatim same string** — `avail=False`, *"CUDA unknown error"* — while the first three now pass completely (§8.11). ⊘ **Deliberately not welded to the LLM failure**: the LLM boots died *earlier*, with no completions watched at all, so the two may share only an error string. *If they are the same defect, this question is about process ordinality and not about torch at all* — which is exactly the kind of "there is a fourth possibility your experiment cannot distinguish" answer this section asks for.

The questions.

1. **What is *"CUDA unknown error"* actually reporting?** It is CUDA's fallback for a driver-API status it has no runtime mapping for. *Which* driver-API failures reach the caller as that string, and is there a way to

recover the underlying `CUresult` from outside the process?

2. **What does torch's full CUDA init do that** `is_available()` **does not?** It walks the primary-context path. **Which RM objects and controls does primary-context creation require beyond what** `cuDevicePrimaryCtxRetain` **alone touches** — given that plain `cuDevicePrimaryCtxRetain` *succeeds* in the same guest?
3. **Is** `0x208f` **what we think it is?** It is `NV20_SUBDEVICE_DIAG`. ⚠ It was mis-identified once here as the shared-user-data class (`0xDE`) by a substring match across two adjacent log lines — *a substring match in a log is not an identity.* Does `cudart` or torch require a diagnostic subdevice, and if refused, does that surface as an init failure?
4. **Is** `cudart` **the whole interface?** Our differential covers `/dev/nvidia*` `ioctl` traffic and nothing else. `mmap` **results,** `/proc`**,** `/sys` **and all UVM behaviour are outside it** — the largest unexamined surface we know of.

★★★ And the two transferable findings from getting this far, which we would most like checked, because if they generalise they apply to any differential oracle built against a reference system.

(1) The minimal fatal set was a PAIR, and no one-at-a-time experiment could find it. Three controls, each measured *innocent alone* on a working host — and refusing two of them together reproduces the failure exactly. A per-id bisection reported all six candidates innocent and **was right about every one**. Only the full subset lattice found it. *Is "individually necessary, jointly sufficient" a recognised shape here, and is there a cheaper search than the lattice?*

(2) One member of the ladder is invisible to any experiment driven by a healthy reference. `0x2080a001` occurs **zero times** in an unmodified host trace — it is the *fallback* the guest reaches only after two earlier controls are refused. So the reference capture, which is the whole instrument, cannot contain it. **The countermeasure we found is to drive the reference INTO the failure and re-capture**, which then also produced a zero-boot reproduction (refuse the fatal pair on bare metal, get the guest's exact failure in seconds). *Is that a named failure mode of differential debugging with a better countermeasure?*

Q2 — Where does a slow host RM verb run, when every entry point is a vCPU trap under the big lock?

★★★★★ ⊘⊘⊘ ANSWERED 2026-09-11, AND THE ANSWER IS NOT THE ONE THE QUESTION EXPECTED. The question is kept in full below because the *reasoning* in it was wrong in an instructive way, and because the part of it that is still open is now much sharper.

What was built. The GSP command queue is **serviced entirely off the vCPU**. A guest write to `NV_PGSP_QUEUE_HEAD` records that a write happened, offers a job, and returns; a worker services the RPCs and raises the interrupt. **There is no inline fallback, deliberately** — a full lane counts itself rather than running on a vCPU. [measured] `w447`, $n = 2$: **561** RPCs serviced off the vCPU, declared inline blocking **166** → **24** (§4.4).

⊘ **Both "escapes" the question ruled out were wrong. (a)** *"The reply must be produced synchronously by type"* — it must not. The guest **polls** the message queue, so a reply posted on a later poll is a reply the guest's own protocol already tolerates; the deferred-reply objection priced the reply as a push when it is a pull. **(b)** *"A second thread posting a reply is a lock-order inversion by construction"* — it is not, once the shared queue stops being a field of the state lock and gets its own.

★★★ And the honest half: the work moved and THE WAIT DID NOT. [measured] `w447`: `worst_trap = 1 779 094 µs at bar0+0x110c00` — *the very register that had just been deferred*. The worker took the *same* lock the vCPU takes and held it across the whole drain, so the wait moved from the call into the lock. **That is the owner's second rule** (§4.4): no blocking work under a lock a vCPU takes, and transitive blocking counts. Two fixes since have not moved the number; a stack sampler then found no vCPU in the MMIO path at all during steady-state running, so **where the 1.8 s actually happens is open.** ★ A per-site census has since named a *second* offender and explained it: the TLB invalidate trigger at `bar0+0xb830b0`, **48 slow traps, worst 54 ms**, caused by this very deferral reading a pending-count through the lock the worker holds (§4.4). **The queue head's own 1.8 s is still unexplained.**

⇒ The question that replaces Q2, and it is a better one. *Not* "where does the slow verb run" — it runs on a worker — but: **for a device model whose guest-visible protocol is request/reply over a memory ring, what is the correct lock discipline such that a worker draining the ring never serialises against the vCPUs it exists to stay out of?** And the specific sub-question we cannot answer from inside: **is a 1.8 s hold at the GSP queue head during bring-up harmless because no guest timer is running yet, or is it a soft-lockup waiting for a slower box?**

⊘ **One escape the question ruled out is still correctly ruled out**: a host RM verb is one call with one latency and no budget can split it. **That is why** `verb_op`**'s execute phase is still inline and counted rather than moved** — the remaining open item, and the one the text below describes accurately.

The constraint, which we believe is structural rather than a wiring gap. Every guest MMIO write into our emulated device arrives on a vCPU thread holding QEMU's Big QEMU Lock. Blocking there freezes *every vCPU and QEMU's main loop* — so the binding constraint is not any NVIDIA timeout, it is the guest kernel's

own soft-lockup and RCU-stall detectors, which see the whole VM stop. Three further facts close the obvious escapes:

- The reply must be produced **synchronously by type**: the policy hook returns `Option<Reply>` to the ring FSM, with no deferred-reply channel. A deferred queue was considered and rejected in source, on the argument that *a deferred reply is state that can be lost, reordered or double-sent*.
- Posting into the GSP ring needs `&mut` access to state living inside the rank-0 lock, so a second thread posting a reply is a lock-order inversion *by construction*.
- Our own invariant R1 forbids issuing a blocking call while any ranked lock is held, and it is asserted, always-on, with a panic.

This is not hypothetical. [measured] 2026-08-20, on the first boot that wired a host forward end to end for a real control: the process aborted with *"R1 no-blocking-under-lock violation: issuing a host RM verb while holding rank*(s) *[0]"*. **The wiring was correct; the architecture refused it.**

Ruled out. Budgeting the work — which *did* fix an analogous case, an unbounded teardown loop, by splitting it into 40 ms slices — **does not generalise**: that work was *ours and divisible*, whereas a forwarded host RM verb is one call with one latency and no budget can split it. Using the one existing off-vCPU thread is also out: it is handed a **read-only closure by type** and structurally cannot issue a verb.

The question. For a device model whose guest-visible protocol is request/reply over a memory ring, and whose replies are generated inside a BQL-held MMIO trap, **what is the correct architecture for a bounded-but-slow host operation?** Specific sub-questions we cannot answer from inside the problem: does QEMU's lockless-MMIO path (`memory_region_enable_lockless_io()`, 10.2) actually change this, or only remove the BQL for the *dispatch* while any state we touch re-serialises anyway? Is there prior art for **returning "retry" to a guest driver that is polling a ring**, without desynchronising its sequence numbers? And is the guest driver's own RPC receive path tolerant of a reply arriving on a later poll than the one that would first have observed it?

Q3 — Does host RM fence the engine before `NV01_FREE` of a channel returns?

Why it matters. We refuse the guest's channel-stop verbs (Q4 context: they name host hardware state we cannot produce). UVM's own source says the stop exists to *"prevent spurious MMU faults during teardown"* — and then discards the status and unmaps anyway. So the question is whether the free path itself provides the fence.

What is established from open source. CPU-RM's channel destructor was read in full and contains **no stop, no preempt, no idle-wait and no channel-halt** — it issues a synchronous free RPC to the GSP and returns. **GSP's half is closed firmware.**

The experiment, fully pre-registered, needs no VM and no part of this project. On bare metal: run a channel writing a monotonically increasing counter into a mapped buffer for ≥ 2 s; issue `NV01_FREE` on the channel with *no* prior stop; sample the counter at t_0, $+50$ ms, $+500$ ms and $+2$ s. **Three outcomes, not two**: frozen at t_0 (RM fences); still advancing at $+2$ s (it does not, and the exposure is real); or it stops somewhere in between (the question becomes *what the bound is*).

Two controls the experiment must carry. A positive control with an explicit stop first. And an **authorship** control proving the GPU, not the CPU, wrote the counter — via a report timestamp tracking the GPU's own clock, **not** a hardware watchpoint, because *a DMA write is invisible to x86 debug registers*. (We learned that the expensive way: a watchpoint is a negative control only.)

If you know the answer from documentation or experience, it saves a bench day and it is severity-informative for every Mode-2-style project, not only this one.

Q4 — Two unprivileged host processes at the same euid, each holding `/dev/nvidiactl`: what actually separates them?

The setting. Our design gives every guest process its own sandboxed, unprivileged host worker holding its own RM client. Two *different guests* therefore become two host processes — **at the same euid**, because nothing in the system currently assigns them different ones.

What we read from the open driver. RM's cross-client validation compares `{euid, pid}` with an **OR**, so a matching euid alone passes. And client handles are not guessing-hard: the encoding is a fixed base OR'd with a monotonically increasing index, so concurrently-created clients receive *adjacent* values.

Two narrowings we found, which are reassuring and are not a fix. RM's default cross-client *dup* policy is PID-scoped, not euid-scoped, so the dup path across tenants is refused even at matching euid — leaving

the direct-naming path. And residual-data-on-VRAM-reuse does not exist on the paths we drive: the driver scrubs on free and a frame with a pending scrub is not allocatable.

The questions.

1. **Is the same-euid direct-naming path exploitable end to end?** We have never run the experiment — two unprivileged processes at one uid, one naming the other's client handle. If you have, we would like the result more than almost anything else in this section.
2. **Does one-uid-per-isolate actually close it?** Our proposed mitigation is a deployment policy — run each worker at a distinct non-zero uid — which moves the boundary from our code into the kernel's. **What else in RM keys on euid** such that distinct uids would break something legitimate, or such that it would *not* close the hole?
3. **What is the right threat model for an unprivileged process holding** `/dev/nvidiactl`**?** We assert that every effect a hostile guest can produce is an effect a local unprivileged process could already produce. Is that assertion defensible?

⊘ **Stated plainly so nobody over-reads the rest of this paper**: every claim in this area is [inferred] from driver source. **No GPU was switched on for any of it, and the single experiment that would settle it — two VMs on one box — has never been attempted by anyone.** There is also no `seccomp` anywhere in the tree; it is absent and named as absent.

Q5 — What is the observable event that says a GPU virtual address range's backing may be released?

The design assumed the guest's own unmap. It does not occur. [measured] 2026-08-15 and again 2026-08-19: **CUDA's suballocator does not unmap on** `cuMemFree`. A workload that allocates and frees repeatedly ends with the guest VAS holding *one* 140 MiB run covering both buffers; the release trigger the design nominated fires **zero times** on every workload tested, and the shipped default was therefore *behaviourally identical to disabling it*.

The consequence, and it is the sharpest failure this project has diagnosed. A freed allocation's framebuffer frames stay bound forever. The guest recycles them; our machinery correctly refuses a second binding for a range already bound; the mapping goes stale; the next write into it kills the channel — **with no** `Xid` **and no driver log line, because nothing ever reaches hardware**. Failure is therefore a function of **allocation history, not allocation size**: the same 64 MiB allocation succeeds alone and fails when a 28 MiB allocation preceded it. *We spent a campaign looking for a size cliff that does not exist.*

What works now, and why it is not an answer. A *takeover* — when the guest re-points a range we already hold, we release and re-install at the same key inside one operation. It works (measured 7 rows versus 0, zero stale refusals versus 32, zero `Xid` versus one). But it is capped, and **that cap is load-bearing**: without it the settlement re-proposes the superseded row and ping-pongs host operations forever. **Which of two live aliases should win is unproven.**

⊘ **UPDATED 2026-08-20** (`cca2eb4b`)**, and the update is part of the question.** The cap used to be **4 per frame, reset by nothing,** and that key was wrong in a way that produced a *hardware fault of our own making*: a device-lifetime budget spent by early actors left a later process's leaves fabricated with no host backing, and the copy engine wrote into one — `Xid 31 FAULT_PDE` at `0x7e59_c6000000`, traced to a frame our own log had marked `CAPPED` (§8.11). It is now keyed by `(phys, taking va)`: a ping-pong between one pair of aliases is still bounded at 2× CAP, while a genuinely new owner gets its own budget. ⚠ **A separate defect of the same shape was found in the same campaign** — a join left behind by an *exited* process, named by no address-table row at all, which the per-address-space takeover could not reach by construction. That one is now reclaimed when nothing names the frame. *Both are workarounds for the missing event this question asks about.*

⊘⊘ **CORRECTED 2026-09-11: THE PREMISE OF THE QUESTION BELOW IS AN INSTRUMENT DEFECT, AND THE QUESTION SURVIVES ANYWAY — SMALLER AND SHARPER.** *"Both transports counted zero"* is true and means nothing: `INVALIDATE_TLB` RPC `fn=200` *cannot* fire (its caller is gated on a vGPU predicate false for us by construction) and the `MEM_OP` method is UVM's transport, not RM's. **The census had no row for the transport RM actually uses**, a raw BAR0 store. §6.1, §4.4.

★ **And a 2026-09-11 source audit answers part of the question with a rule.** A release *is* sometimes signalled: an unmap that *sparsifies instead of unmapping* is a genuine revocation, and its caller supplies the invalidate. ⇒ **The observable is not "a page-table entry changed" but "real backing appeared or disappeared."** ⊘ **What remains uncovered is specific and worse than the original question assumed**: a VA-space *destruct* releases page-table levels with **no invalidate anywhere in it**, and `VASPACE_RELEASE_ENTRIES` is a downgrade with no invalidate that frees PDE

memory for reuse *while our host mappings stand*. **Those are release events that emit nothing**, which is the question below, now with named instances.

The question. *Is there any protocol-visible event, on a GSP-client part, that means "this GPU VA range's backing is no longer referenced"?* We have measured that TLB invalidates do not appear on this path at all — both transports counted zero — and that `MAP_MEMORY_DMA` is stubbed out through the entire HAL chain on GSP clients. If the honest answer is *"no such event exists and you must reference-count what you observe"*, that is a useful answer and we will take it. **What we want to avoid is building a reclamation model on a second event that also never fires.**

Q6 — Is a guest-memory region lock implementable on arm64 at all?

The mechanism on x86. To read a guest page table safely we want to make a guest-physical range temporarily unwritable and have the resulting fault be *decodable* by the VMM, so that a guest write during a walk is stalled rather than racing.

The reading that says it cannot work on arm64. On a stage-2 permission fault caused by the guest's own *page-table walker*, ESR's ISV bit is 0 — the syndrome carries no instruction information — so KVM cannot hand the VMM a decodable MMIO exit and instead injects an abort into the guest. **If that reading is right, the region lock does not exist on arm64**, and our own documents record the contradiction in writing: *"'arm64 kept' and 'the capability is refused on arm64' cannot both hold."* Three mutually exclusive standings were written the same day and none was chosen.

The questions. Is the ISV=0 reading correct and complete for current kernels? Is there *any* arm64 mechanism that gives a hypervisor a decodable, resumable stall on a guest-page-table write — stage-2 permission plus a software walker, hardware dirty-state management, something in the SMMU, or an approach we have not considered? And if not: **what do other hypervisors that need to read guest page tables safely do on arm64?**

⊘ **One honest note that may make this cheaper than it looks.** Our own analysis concluded the region lock may have **no members at all** — that is, no call site actually needs it — in which case the contradiction is moot and the right move is to build nothing. **We would rather learn that the primitive is unnecessary than pick one of three standings.**

Q7 — What host-side state, fixed at boot and stable thereafter, varies performance by 9×?

The measurement. Same workload, same binary, same box, three consecutive boots: the guest's submit-side cost was **85.32, 9.41 and 64.39 ms — a 9.1× spread with the middle boot fastest**. Across twelve boots that day it spanned **6.3 to 216 ms**. **Within any one boot the iterations agree to** ±2%, so the value is decided at boot and then holds. A different statistic on the same twelve boots held to ±0.8%.

Ruled out. A session ramp from accumulating host state: the sequence is *non-monotone*, and free memory and low-order page availability *improved* after the slow boot while the cost stayed high.

What we suspect but have not shown. The dominant cost in the slow case is pinning guest RAM page by page, and the per-operation count tracks *physical contiguity* of the backing — so host page-allocator fragmentation at the moment the guest's memory backing is created is the obvious candidate. **The question is what to measure to confirm or kill it**, and what a device model can do about it: is there a way to obtain, or to ask the kernel to preserve, physically contiguous backing for a large memfd on a long-uptime host, short of `hugetlbfs` reservations at boot?

⊘ **One trap already paid for here.** `MADV_HUGEPAGE` on a **memfd** returns **0** and delivers **zero** huge pages: a memfd is shmem, and shmem transparent huge pages are a *separate* sysfs knob that commonly ships disabled. *The syscall's return value is not the measurement*; the mapped-huge-page counter is. A prescribed fix in one of our own design documents was inert as written for exactly this reason.

Q8 — We are the GSP. The guest kernel driver trusts the GSP completely. Is that a confused deputy?

The structure. The open NVIDIA kernel driver applies essentially no validation to GSP RPC replies — it is talking to firmware on a device it already trusts with full DMA. **In this architecture, we occupy that position**, and the data we return is derived, in part, from what the guest's own *userspace* asked for.

The question, which nothing in our audit set currently asks: can guest userspace steer the values we hand the guest kernel such that the guest kernel violates an invariant its own source assumes? The interesting cases are not memory-safety-obvious ones; they are fields the driver uses as sizes, counts, indices or capability bits without re-checking, where the value's provenance is ultimately an unprivileged request.

One instance of this class we found and closed, offered as the shape to look for. At driver 580 the guest reads an element-count field *we choose* directly into a copy loop whose staging area is a fixed small number of elements, with the live ring metadata as the very next byte. A count one larger than the staging area overwrites metadata the guest is actively using; at the ring's reachable maximum it writes 188 416 bytes past a kernel allocation. **That is guest-kernel heap corruption reachable from a value we pick.** It is now refused by a bound derived from the layout rather than a literal, checked on both the send and receive sides.

We do not believe that is the only one. The general question for a reader who knows this driver: **which fields in the GSP→CPU direction does the kernel driver consume without validation, and which of them can an emulated GSP choose freely?**

Q9 — Smaller, sharply-posed items

- **Driver-version drift.** Our supported cross-product is currently **one point**: one GPU (GA106), one guest driver version, one host driver version, one guest at a time. **The supported drift range is unmeasured**, and the discriminating experiment is cheap — pin the host, walk the guest driver back through earlier majors, and record *what* refuses. Nobody has run it. *Is there a documented compatibility contract between an NVIDIA guest driver and the GSP firmware it talks to, or is version-lockstep the only safe assumption?*
- **The second GPU generation will fail before it starts.** Our second architecture's page-table format currently delegates to a *test double with an invented encoding*, while the first is checked against NVIDIA's own generator. **Compile the second generation's format against the vendor source as an oracle *before* booting it**, or the boot reports "it broke" without reporting where. *Is the GMMU page-table format stable across Ampere and Ada, and where is the authoritative encoding in the open modules?*
- **A negative fixture that has never been replayed.** The predecessor implementation parsed **378 GSP ring elements out of arbitrary guest RAM and answered success** — a guest-reachable defect, kept as a recorded trace. The rewrite has a named refusal vocabulary that *should* reject every one of them. **No test replays the fixture against it**, so the immunity is argued from types and not measured. *This is the cheapest open item in this section and we are listing it to be embarrassed into closing it.*
- **No teardown or driver-reload boot has ever been run**, and our core has **no reset entry point of any kind**: on a guest driver reload the graph, every live process with its sandboxes, arenas and host handles, both routing maps and the guest-physical window all survive into the next driver's life with nothing reclaimed. The recommendation to spend the first successful boot on a teardown was made on 2026-08-05; many dozens of successful boots later, no such run is recorded. *Nothing surfaces this incidentally, which is exactly why it is still open.*
- **What kills the fourth CUDA process, when the first three now pass identically?** [measured] `w367`, 2026-08-20 (§8.11): three sequential guest CUDA processes complete with *identical* doorbell counts and 8/8 completions each, host `Xid` 0. The fourth reports `avail=False`, *"CUDA unknown error"*. **Only three processes ever existed for four workloads**, so the fourth never reached the device; the guest kernel logged nothing while it ran. **It fails above the `ioctl` boundary**, which is precisely where our instruments stop — our differential covers `/dev/nvidia*` `ioctl` traffic and nothing else. *What state does `libcuda` or the guest kernel driver keep that is per-boot rather than per-process, and exhausts or wedges after three contexts?* ⚠ $n = 1$ boot, and the run's logs are not committed.
- **A quadratic control plane, measured.** Deriving process boundaries is $O(\text{live objects})$ per event, so N events cost $O(N^2)$: **1 000 events 0.85 s, 8 000 events 54.8 s** on a debug build, against a cap of 2^{18} live handles. Deleting the per-event clone saves ∼24% and leaves the curve quadratic. *The unmeasured quantity that decides whether this is a denial-of-service or a real workload's ceiling is: how many RM objects does a CUDA or PyTorch process hold live simultaneously?* We have never measured it, and an earlier claim that PyTorch startup reaches the curve benignly was an inference and is withdrawn.

★★★ **And one meta-question, which is the one we would most like an outsider's view on.** Every question above is stated with a named instrument and that instrument's known limits. This project's central finding — measured roughly twenty-five times — is that **an instrument's null and the system's success are the same reading**, and that the failures are *asymmetric*: a broken instrument almost always makes the system look *better* than it is, because a mutant that is not run, a capture that is empty, a gate quantified over the wrong set, and a universal over an empty set all read as green.

Our working countermeasures are: **plant one reachable violation per class before trusting any audit** (a scan that misses its own plant is a broken instrument, not a clean bill of health); **discriminate on counts, never on a diff** (a vocabulary diff came back empty in both directions while the explaining quantity sat in the log at 32 against a baseline of 16); and **pre-register three-valued falsifiers**, because the third outcome — *the arm never fired at all* — is

usually the likely one and a two-valued test cannot see it.

Is there a better discipline than this? We arrived at it by paying for each rule individually and we do not know whether it is the state of the art or a reinvention of something with a literature.

A Measurement method

Every number in this paper that is not attributed to a document was produced by one of the following, run against the kayfabe tree at `5c367a38` and `/workspace/nvidia-gpu-passthrough` at `c86b69d`. **No compilation, no test execution, no GPU and no network access was involved**: the box has no GPU and 16 GB of free disk, so a `cargo build` was ruled out before writing began rather than skipped afterwards. Two things that need *no* compilation *were* run and are marked [measured] in the text: `scripts/claim_ledger.py --gate` and `cargo fmt --all --check`.

⚠ **And one hazard this revision hit that previous ones did not: the tree moved underneath it.** A commit landed mid-session that changed the answer to §8.5 — the pin was `034154fd` when the front matter was drafted and `5c367a38` when it was finished, and the LLM-wall paragraph was rewritten twice as a result. **Everything is re-measured at the final pin**, but a reader should treat *"pinned at a commit"* as a claim about a moving ref unless the working tree was also clean — which the project itself learned when a rung reported *"byte-identical to master"* and `master` had advanced 30 files underneath it.

⊘ **AND IT KEPT MOVING AFTER THE DOCUMENT WAS COMMITTED, WHICH IS THE STRONGER FORM OF THE SAME HAZARD.** Twelve further commits landed between the pin (`5c367a38`, 15:36) and this paper's first reading (`56dc01f3`, 20:32), and two of them falsified headline claims. The corrections are folded in above the text they correct rather than appended, and are listed as §10 rows 43–45. ⚠ **No structural quantity in this paper was re-measured at** `56dc01f3` — the commands in the table below were run at `5c367a38` and their outputs are pinned there. ⇒ *"Re-measured at the final pin" is a claim with a timestamp, and the timestamp expires.*

Quantity	Command
implementation lines	`find crates -path '*/src/*' -name '*.rs' | xargs wc -l`
test lines	the same over `tests`, `crates/*/tests`, `fuzz`
per-crate lines	the same, per `crates/<name>`
`#[test]` sites	`grep -rc '#[test]'` over workspace members, excluding the detached `crates/kayfabe-abi/gen` and the `trybuild` ui fixtures
`unsafe` relaxations	`grep -rhoE 'unsafe ({|fn |impl |trait )'` `crates/kayfabe-linux-raw/src/*_unsafe.rs | wc -l` — the same expression the CI ratchet uses
dependency edges	every `Cargo.toml` read in full, cross-checked against `Cargo.lock`
commit counts and dates	`git rev-list --count HEAD`, `git log --format=%ci`
document staleness	`git log -1 --format=%ci` per file, compared against the commit that invalidated the claim

The five figures are TikZ, drawn from the measurements above and from a function-by-function read of the call chains they depict; they are vector, not traced from an image. Figure 2 in particular is generated from the manifest edge list rather than from any design document, which is why it disagrees with the hexagon picture about which crates are "ports".

Two method rules carried forward, both prompted by the figures being where the drift was worst. Figure 3 names *symbols* rather than `file:line` pins, as the repository itself did to its own 105 pins in `3fd1455`. And Figure 2's edge list is re-derived from scratch every revision rather than edited — it was re-derived by script this time, which is how the `mocks` → `abi` edge and the new topological ordering were picked up without being looked for.

One method rule carried forward. Every claim carried over from the previous revision was re-measured rather than trusted, *including the ones this paper itself originated*. That is the only reason rows 27, 28, 30 and 33 exist — all four are claims this document asserted with no hedge, and all four had gone stale in the *understating* direction, which nobody audits for. **Re-derive, do not restate**, because a restated number and a measured number are indistinguishable on the page.

★ **One method rule added at this revision, and it earned two corrections immediately.** *The brief is a source like any other and is re-derived, not obeyed.* Two figures supplied to this revision as settled facts were checked

against the tree and found wrong (§10 rows 31–32); one was a number that appears in no document, and one was a blanket where the exceptions carried the actual argument. Neither is a criticism of whoever wrote the brief — both were close to right, and the second was a fair summary of an earlier document that a later measurement had superseded. **The rule is that an instruction and a citation decay the same way.**

★★★ **The claim ledger was run against this tree and it is RED.** `scripts/claim_ledger.py --gate` exits **1** at `5c367a38` with **1 166 unattributed / 90 conflated / 36 bare-hardware** against bars of 381 / 66 / 17 (§7.3). The previous revision reported this gate green with all three bars holding; that is no longer the case, and the bars have not moved since 2026-07-31. ⊘ **And the result is weaker than it looks in both directions**: the ledger's universe is `*.rs` and `*.md`, so **this file is not in it** — the gate says nothing whatever about the whitepaper — and its universe now includes the committed bench-trace READMEs, so part of the growth is denominator and not regression. *Which part, this paper did not measure.*

One closing note on how to use this paper. If you are here to find what is wrong with kayfabe, start at **§11** — the open questions are written to be attacked by someone with no access to this tree, and each carries the evidence needed to attack it. Then, in order: §8.11 (the paper's headline blocker, fixed and measured passing five hours after this document was committed), §8.5 (the LLM wall — ⊘ **this line used to read** *"the CUDA runtime does not initialise, and the fix is unbooted"*, which contradicted that very section *at this pin*: the wall broke, the model still produces no token, and the cause named there was refuted, §10 rows 44–45), §8.4 (the performance picture, and why one statistic grades two flags backwards), §8.3 (what "first compute" does and does not mean — every green is a relaxed green and the shipping default is not among the relaxations), §8.13 (what contact with hardware bought, as a conjunction rather than either half), §3.1 (the oracle is positively wrong about a class of rows and a larger class is unmeasured), §8.17 (an invariant broken on the production path under a green suite), §8.18 (there is no tenant axis, and the thesis is multi-tenancy), §7.3 and §7.7 (the evidence machinery, wrong five times and currently red), and §8.8 (an independent oracle that was independently written and identically wrong). Then the code — starting with `VerbPlan::gated_doorbell`, `Spine::apply`, `rmrpc::translate` and `kayfabe_fwd::plan_ce`. Do not start with `README.md`, and do not use `crate_maturity_map.md` for anything (§10 row 29).

And one note on how to read its confidence, which this revision has the sharpest version of. Each of the five revisions of this document has found its predecessor wrong in several places, and **for four consecutive revisions the falsified claim was the one the predecessor led with**: a disconnected crate, then the bridge, then the guest reaching the bridge, then the execution plane that *"cannot be faked"*. All four were crossed. **Worse, this revision found that three of its predecessor's headline claims were false *on the day it was published***, not merely overtaken later (§10, rows 35–37) — so the failure mode is not update latency, it is that **nobody re-derives the sentence everybody already believes.**

This revision's lead claim was that the remaining wall is *no longer* a question about what NVIDIA's protocol wants — it is our own object model, reached through closed CUDA userspace. It is the fifth answer to "what is the critical path", **and it changed twice in the hours this document took to write**, the second time because a milestone had been graded on a proxy.

⊘ **AND IT CHANGED A THIRD TIME, THREE MINUTES AFTER THIS DOCUMENT WAS COMMITTED, AND A FOURTH FIVE HOURS AFTER THAT.** *"Our own object model"* is refuted — every refusal in it is more numerous in the boot that passes (§10 row 44). *"A second CUDA context sequentially in one boot still hangs"* is fixed — three sequential processes pass (row 43, §8.11). **The fifth answer lasted three minutes and there is no sixth**: what `torch._C._cuda_init()` fails on, and what kills a *fourth* CUDA process, are both **unidentified**. ⇒ **The base rate did not merely hold; it accelerated. Treat every unhedged sentence here exactly the way the previous four deserved to be treated** — including this one, and including that one.